Quantitative Portfolio Management: Theory and Practice

Part 1: Chapters 1–4

Extended lecture notes and class companion

EDHEC Business School • Prof. Raman Uppal

Personal study notes based on the course slides, 2026–2027

Last updated: November 3, 2026

Abstract

These notes follow the course slides chapter by chapter and are designed to be used *during* class. Each chapter contains: (i) "In plain words" explanations, (ii) the theory with full formulas and step-by-step derivations, (iii) the intuition behind every expression, (iv) the Python exercises from the slides with their expected output and interpretation, (v) takeaways and self-test questions for the in-class quizzes and the handwritten final exam. The appendix contains question-by-question guides to the assignments, a formula sheet and an English–Italian glossary. Grey boxes mark material that is *not* on the slides.

Contents

How to use these notes **2**

Lecture log **4**

Chapter 0 – Syllabus and course organisation **5**

1 Managing Financial Data **8**

1.1 What we want to do in Chapter 1 8

1.2 Overview of the course: mean–variance efficient portfolios 8

1.2.1 One risky asset ($N = 1$) 8

1.2.2 Many risky assets ($N > 1$) 9

1.3 Estimation error in mean–variance weights (focus) 10

1.3.1 Why $\mathbb{E}[\boldsymbol{R}]$ is hard to estimate 11

1.3.2 Why $\mathbb{V}[\boldsymbol{R}]$ is hard to estimate 11

1.3.3 Why poor estimates are disastrous 11

1.4 Portfolio weights over time and out-of-sample returns 13

1.5 Notation 14

1.6 Convention regarding data: tidy vs. wide 14

1.7 Prices and returns 15

1.7.1 Multiperiod (compound) returns 16

1.7.2 Annualised returns 16

1.7.3 Continuous compounding (log returns) 16

1.7.4 Returns with dividends 19

1.7.5 Distribution of returns 19

1.7.6 Are returns Normal? Skewness and kurtosis 19

1.8 Obtaining financial and macroeconomic data 21

1.8.1 Stock-price data (Yahoo! Finance) 21

1.8.2 From prices to returns 22

1.8.3 Stock-characteristics data 23

1.8.4 Macroeconomic data 24

1.9 Takeaways of Chapter 1 25

1.10 Self-test questions (quiz/exam practice) 26

1.11 To do for next class 27

2 Performance Measurement **28**

2.1 What we want to do in Chapter 2 . . . 28
2.2 Static, myopic, and dynamic portfolios . . . 28
2.3 Backtesting: in-sample vs. out-of-sample performance . . . 29
2.3.1 Rolling vs. expanding window . . . 29
2.4 Measures of portfolio performance . . . 30
2.4.1 Measures of mean returns . . . 30
2.4.2 Measures of the riskiness of returns . . . 31
2.4.3 Measures combining mean and risk . . . 32
2.5 Transaction costs and turnover . . . 35
2.6 Review of hypothesis testing . . . 37
2.7 Test of the difference in Sharpe ratios (focus) . . . 38
2.7.1 The delta method . . . 38
2.7.2 Step 1: distribution of $\hat{\mu}_n$ and $\hat{\sigma}_n^2$ . . . 39
2.7.3 Step 2: distribution of $\widehat{SR}_n$ . . . 39
2.7.4 Step 3: distribution of $\widehat{SR}_n - \widehat{SR}_m$ . . . 40
2.7.5 Bootstrapping . . . 42
2.7.6 Block bootstraps for time series . . . 42
2.7.7 Ledoit–Wolf (2008) bootstrap p-value for ΔSR . . . 43
2.7.8 Multiple testing and p-hacking . . . 45
2.8 Takeaways of Chapter 2 . . . 45
2.9 Self-test questions (quiz/exam practice) . . . 46
2.10 To do for next class . . . 47

3 Mean–Variance Portfolios that Ignore Estimation Error **49**

3.1 What we want to do in Chapter 3 . . . 49
3.2 Universe selection . . . 49
3.3 Portfolio choice in pictures: the Nobel-prize ideas . . . 50
3.4 Portfolio diversification . . . 50
3.4.1 Systematic vs. unsystematic risk . . . 50
3.4.2 Mean and variance of a portfolio . . . 51
3.4.3 Diversification with N assets . . . 51
3.5 Mean–variance frontier without a risk-free asset (focus) . . . 53
3.5.1 The optimisation problem . . . 53
3.5.2 Solving it step by step (Lagrangian) . . . 53
3.5.3 Two-fund separation and the GMV portfolio . . . 55
3.5.4 Worked example with $N = 3$ . . . 55
3.6 Portfolio frontier with a risk-free asset . . . 56
3.6.1 The problem . . . 56
3.6.2 Worked example: three risky assets + $R_f = 6.5\%$, $\mu_{targ} = 30\%$ . . . 57
3.6.3 Equivalent approach: maximise mean–variance utility . . . 58

3.6.4 The market portfolio . . . 59
3.7 Risk–return of individual assets and frontier portfolios (focus) . . . 60
3.7.1 The beta relation . . . 61
3.8 Portfolio optimisation with respect to a benchmark . . . 62
3.9 Other constraints and transaction costs . . . 63
3.10 Summary of modern portfolio theory . . . 64
3.11 Self-test questions . . . 64
3.12 To do for next class . . . 65

4 Mean–Variance Portfolios that Adjust for Estimation Error 66
4.1 What we want to do in Chapter 4 . . . 66
4.2 The effect of estimation error on portfolio weights (focus) . . . 66
4.2.1 Error in $\mathbb{E}[\boldsymbol{R}]$ . . . 66
4.2.2 Error in $V[\boldsymbol{R}]$ . . . 67
4.2.3 A two-asset example . . . 68
4.3 Performance of sample-based mean–variance portfolios . . . 69
4.3.1 Methodology for out-of-sample evaluation (focus) . . . 69
4.3.2 Evidence: DeMiguel, Garlappi and Uppal (2009) . . . 70
4.4 Models that adjust for estimation error: the menu . . . 71
4.5 Bayesian approaches . . . 71
4.6 Short-sale-constrained portfolios (focus) . . . 73
4.6.1 Why short-sale constraints shrink the means . . . 74
4.7 Global-minimum-variance portfolios (focus) . . . 74
4.7.1 Variant 1: the GMV portfolio . . . 75
4.7.2 Variant 3: GMV with short-sale constraints (Jagannathan–Ma 2003) . . . 76
4.7.3 Variant 4: Ledoit–Wolf shrinkage of the covariance matrix . . . 78
4.7.4 Variant 5: norm-constrained portfolios (DGNU 2009) . . . 80
4.7.5 Performance of Ledoit–Wolf and norm-constrained portfolios . . . 81
4.8 The big comparison at a glance . . . 81
4.9 Appendix: numerical optimisation with `scipy` . . . 81
4.9.1 Mean–variance utility (with risk-free asset) . . . 82
4.9.2 GMV: analytic, numerical, with bounds . . . 82
4.10 Takeaways of Chapter 4 . . . 83
4.11 Self-test questions . . . 84
4.12 To do for next class . . . 85

References (Chapter 1) 86

A Assignment guides 87
A.1 Submission checklist (applies to every assignment) . . . 87
A.2 Assignment 1 (Lectures 1–2; due 11 November) . . . 87
A.3 Assignment 2 (Lectures 3–4; due 16 November) . . . 91

B Formula sheet (Chapters 1–4) **96**

C Glossary (English – Italian) **98**

How to use these notes

Purpose. A companion to keep open during class: the slides say *what*, these notes also say *why* and *how*. Each chapter follows the slides in order, so you can find the page the professor is on.

The boxes.

Box	Meaning
In plain words	The idea explained simply, without formulas. Read this first if you are lost.
Key result (▶)	Formula or fact you must know for the quiz and the exam.
Definition	A precise definition.
Intuition	Why the formula makes economic sense (the professor always asks this).
Additional note	Derivations, numerical checks and code explanations that are *not* on the slides.
Worked example	A numerical example solved step by step (many are not on the slides).
Common mistakes	Errors students typically make in code and in exam answers.
Code listings	Python from the slides (cleaned), with the expected output in a separate frame.

The course in one page. The whole course is built around **one formula**, the optimal mean–variance portfolio

$$\boldsymbol{w} = \frac{1}{\gamma} V^{-1} (\boldsymbol{\mu} - R_f \mathbf{1}_N) \qquad \text{(weights = risk tolerance} \times \text{inverse covariance} \times \text{risk premia).}$$

1. **Chapter 1 (data):** how to get prices, turn them into returns, and estimate the two inputs $\boldsymbol{\mu}$ and V. Main lesson: $\hat{\boldsymbol{\mu}}$ is very noisy, $\hat{V}$ is better but hard to invert when N is large.
2. **Chapter 2 (performance):** how to judge a strategy honestly (out of sample), which numbers to look at (Sharpe ratio, risk measures, turnover) and how to test whether a difference is real or luck.
3. **Chapter 3 (theory):** what the formula gives if we *knew* $\boldsymbol{\mu}$ and V: efficient frontier, tangency portfolio, beta relation.
4. **Chapter 4 (practice):** what happens with *estimated* inputs (disaster), and the fixes: Bayesian shrinkage, constraints, GMV, Ledoit–Wolf, norm constraints. Common thread: **shrinkage**.
5. **After Part 1:** the rest of the course uses economic models to get better inputs (covered in Part 2 of these notes).

Lectures, assignments and exam questions.

Sessions	Chapters	Assignment	Due	Exam question
1–2	1 Data, 2 Performance	#1	11 Nov	Q1
3–4	3 MV theory, 4 Estimation error	#2	16 Nov	Q2
5–6	5 Black–Litterman, 6 Parametric policies	#3	25 Nov	Q3
7–8	7 Volatility timing, 8 Unsystematic risk	#4	30 Nov	Q4

Study routine that matches the grading.

1. **Before class:** read the "In plain words" boxes of the chapter (10 minutes).
2. **During class:** follow the slides here; mark what the professor stresses.
3. **After class:** do the self-test questions at the end of the chapter (they mimic the quizzes).
4. **Weekly:** rewrite by hand, without looking, the "write by hand" code at the end of each chapter (the exam is code on paper).
5. **Before the exam:** the Formula sheet and the Glossary at the end of these notes.

Lecture log

Sess.	Date	Chapter	Due (22:00 CET)	Notes
1	02 Nov	Ch. 0 Syllabus; Ch. 1 Financial data		✓
2	03 Nov	Ch. 2 Performance measurement	#1: 11 Nov	✓
3	04 Nov	Ch. 3 MV portfolios ignoring estimation error		✓
4	05 Nov	Ch. 4 MV portfolios adjusting for estimation error	#2: 16 Nov	✓
5	16 Nov	Ch. 5 Black–Litterman		
6	17 Nov	Ch. 6 Parametric portfolio policies	#3: 25 Nov	
7	18 Nov	Ch. 7 Volatility-timed factor portfolios		
8	19 Nov	Ch. 8 Systematic and unsystematic risk	#4: 30 Nov	
Exam	16 Dec	Final exam, 9:00, closed-book, on paper		

Chapter 0 – Syllabus and course organisation

0.1 Instructor

- Raman Uppal, Professor of Finance, EDHEC Business School.
- Email: `Raman.Uppal@edhec.edu` — start the subject with "`QPM-2026-2027`"; replies within 24 hours.
- Office hours: by appointment and by email. Homepage: `https://www.ramanuppal.com`.

0.2 Objectives and learning outcomes

The objective is to study cutting-edge methods to construct **optimal equity portfolios that perform well out of sample**, combining *finance* (portfolio management), *econometrics* (empirical finance) and *data science* (Python).

LO1 Use Python to obtain data from Yahoo! Finance, Nasdaq Data Link (Quandl), Ken French Data Library, Open Source Asset Pricing.

LO2 Use Python to evaluate the performance of portfolios.

LO3 Use Python to construct mean–variance efficient portfolios with short-sale constraints, position limits, LASSO and elastic-net constraints, and transaction costs.

LO4 Use Python to improve performance by adjusting for estimation error in means, variances, covariances, and by using conditioning information.

Prerequisites: calculus and linear algebra; Markowitz optimisation; basic econometrics; Python. The course is *not* an introduction to Python, not a machine-learning course, and does not discuss AI/LLMs.

0.3 Assessment

► **Key result: grading**

- **15%** — four assignments and eight in-class online quizzes.

$$\text{Grade}_j = [\text{Quiz } j \text{ grade}] \times \mathbb{1}\{\text{corresponding assignment submitted}\}.$$

The lowest quiz score is dropped. Assignments are *not graded* (AI is allowed): full credit if submitted on time.

- **85%** — final exam: handwritten, in class, closed-book, four questions matching the four assignments; tests portfolio theory and the ability to **write Python code by hand**.

Quizzes cannot be made up; late assignments get 0.

0.4 Assignment rules (failure ⇒ grade 0)

- Groups of exactly 3 or 4 students (diversity encouraged).
- Email to `Raman.Uppal@edhec.edu`, with subject line "`QPM 2026-2027: Assignment n`", where $n \in \{1, 2, 3, 4\}$.
- Executable Jupyter notebook named `QPM-2026-Assignment-n-A-B-C-D.ipynb` (A–D = last names).
- Section 0 (Markdown): line 1 submission date; line 2 "QPM 2026-2027: Assignment n"; line 3 group members alphabetically, LAST NAME in capitals; line 4 comments/challenges.
- Section k: copy Question k, then the answer; include all imports.
- Before sending: *Kernel → Restart Kernel and Run All Cells.*
- Mention any outside help (no penalty).

0.5 Class rules

Attendance mandatory; double-badging ⇒ 0 for the whole course for both students (to miss a class, email the HUB and copy the professor). Bring a laptop (quizzes) and a name card. No phones; laptop only for the course. Expected workload: about 2 hours of study per hour of lecture.

0.6 Course map and difficulty

Part	Chapter	Theory	Coding
A. Preliminaries	1 Managing financial data	Low	Low
	2 Performance measurement	Low	Low
B. Modern PM	3 MV ignoring estimation error	Low	Medium
	4 MV adjusting for estimation error	Medium	Medium
C. Post-modern PM	5 Black–Litterman	High	Medium
	6 Parametric portfolio policies	Medium	Medium
	7 Volatility-timed factor portfolios	Medium	Medium
	8 Systematic & unsystematic risk	Medium	Medium

Topics in detail.

- **Ch. 2**: out-of-sample performance metrics; VaR, Expected Shortfall, Maximum Drawdown; backtesting and manipulation-proof measures; transaction and price-impact costs; bootstrapped p-values (Ledoit–Wolf 2008); p-hacking and multiple testing.
- **Ch. 3**: Markowitz theory (Merton 1972; Roll 1980; Huang–Litzenberger 1988); in-sample performance; short-sale, position (LASSO, elastic-net) and leverage constraints; transaction costs; higher moments; out-of-sample performance.

- **Ch. 4**: Bayesian approaches (Jorion 1985–1992); three-fund portfolio (Kan–Zhou 2007); GMV portfolio; short-sale constraints (Jagannathan–Ma 2003); covariance shrinkage (Ledoit–Wolf 2003–2017); norm-constrained portfolios (DeMiguel, Garlappi, Nogales, Uppal 2009).
- **Ch. 5**: Black–Litterman (1990–1992), intuition (He–Litterman 1999), derivation, numerical example.
- **Ch. 6**: single/multi-factor models; fundamental indexation (Arnott–Hsu–Moore 2005); risk parity (Roncalli 2013); parametric portfolio policies (Brandt–Santa-Clara–Valkanov 2009).
- **Ch. 7**: volatility timing (Kirby–Ostdiek 2012; Moreira–Muir 2017, 2019); net of costs (Barroso–Detzel 2021); out of sample (Cederburg et al. 2020); both (DeMiguel, Martín-Utrera, Uppal 2024).
- **Ch. 8**: asset pricing with unsystematic risk (Dello-Preite, Uppal, Zaffaroni, Zviadadze 2025); portfolio choice with unsystematic risk (Raponi, Uppal, Zaffaroni 2023).

0.7 Resources

- Slides (main material). Red = important; blue = links.
- **Tidy Finance with Python** (Scheuch, Voigt, Weiss, Frey 2024) — main Python reference. Tidy-data principles used throughout.
- Bali, Engle, Murray (2016), *Empirical Asset Pricing* — empirical finance.
- Moreira (2021), *Quantitative Investing* (online, with Python).
- EDHEC MOOC "Investment Management with Python and Machine Learning" (Coursera) — pre-course Chapter 0.

Chapter 1

Managing Financial Data

1.1 What we want to do in Chapter 1

- Take a big-picture view of the whole course.
- Study the main kinds of data used to construct optimal portfolios.
- Learn to obtain this data with Python and store it efficiently.

In plain words

Chapter 1 sets the stage. The whole course is about one formula that tells you how much to put in each stock. The formula needs two inputs: the *expected returns* and the *covariance matrix*. Both must be estimated from data, and this chapter shows (i) how to get and clean the data and (ii) why those estimates are much noisier than you would think. Everything else in the course is a way to deal with this noise.

The two questions of the course are: **Q1.** How should we construct optimal portfolios? **Q2.** How should we measure portfolio performance?

1.2 Overview of the course: mean–variance efficient portfolios

The course focuses on **mean–variance efficient** weights, which trade off optimally the portfolio's return *mean* and return *variance*. For most of the course we do *not* focus on higher moments (skewness, kurtosis), other risk measures (VaR, ES) or general expected-utility objectives.

1.2.1 One risky asset ($N = 1$)

An investor splits wealth between one risky asset (e.g. the market) and the risk-free asset. The optimal weight in the risky asset is

▶ **Key result: optimal weight,** $N = 1$

$$w_1 = \frac{1}{\gamma} \frac{\mathbb{E}[R_1] - R_f}{\mathbb{V}[R_1]},$$

w_1: fraction of wealth in the risky asset; γ: risk aversion; $\mathbb{E}[R_1]$: expected gross return; R_f: gross risk-free return; $\mathbb{V}[R_1] = \sigma^2_{R_1} = \sigma_{R_{11}}$: return variance.

Intuition

1. $\gamma \uparrow \Rightarrow\ w_1 \downarrow$ (more risk-averse, less risky asset).
2. Risk premium $\mathbb{E}[R_1] - R_f \uparrow \Rightarrow\ w_1 \uparrow$.
3. Risk $\mathbb{V}[R_1] \uparrow \Rightarrow\ w_1 \downarrow$.

The professor insists: *every formula in the course must make intuitive sense.*

Additional note (not on the slides)

Where the formula comes from. The mean–variance investor maximises

$$\max_w \underbrace{R_f + w(\mathbb{E}[R_1] - R_f)}_{\text{portfolio mean}} - \frac{\gamma}{2} \underbrace{w^2\, \mathbb{V}[R_1]}_{\text{portfolio variance}} .$$

First-order condition: $\mathbb{E}[R_1] - R_f - \gamma w\, \mathbb{V}[R_1] = 0 \Rightarrow w = \frac{1}{\gamma}\frac{\mathbb{E}[R_1]-R_f}{\mathbb{V}[R_1]}$. The objective is concave in w (second derivative $-\gamma\mathbb{V}[R_1] < 0$), so this is a maximum. *Example:* $\mathbb{E}[R_1]-R_f = 6\%$, $\sigma = 20\%$, $\gamma = 3$: $w = \frac{1}{3} \cdot \frac{0.06}{0.04} = 0.5$, i.e. 50% in the market.

1.2.2 Many risky assets ($N > 1$)

▶ **Key result: the key expression of the course**

$$\boldsymbol{w} = \frac{1}{\gamma}\, \mathbb{V}[\boldsymbol{R}]^{-1} (\mathbb{E}[\boldsymbol{R}] - R_f\, \mathbf{1}_N),$$

with

$$\boldsymbol{w} = \begin{pmatrix} w_1 \\ \vdots \\ w_N \end{pmatrix}, \quad \mathbb{E}[\boldsymbol{R}] = \begin{pmatrix} \mathbb{E}[R_1] \\ \vdots \\ \mathbb{E}[R_N] \end{pmatrix}, \quad \mathbf{1}_N = \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix}, \quad \mathbb{V}[\boldsymbol{R}] = \begin{pmatrix} \sigma_{R_{11}} & \cdots & \sigma_{R_{1N}} \\ \vdots & \ddots & \vdots \\ \sigma_{R_{N1}} & \cdots & \sigma_{R_{NN}} \end{pmatrix},$$

where $\sigma_{R_{nn}} = \sigma^2_{R_n}$ is the variance of asset n and $\sigma_{R_{nm}}$ the covariance between n and m. The weight $1 - \boldsymbol{w}^\top \mathbf{1}_N$ goes to the risk-free asset.

In plain words

Compared with $N = 1$, the only new thing is V^{-1} instead of $1/\sigma^2$. It does two jobs: it penalises volatile assets (like $1/\sigma^2$) and it accounts for *correlations*: if two assets move together, holding both is like holding one, so the formula avoids double-counting them, and it may even short one to hedge the other.

Worked example: two risky assets by hand

Excess means $\boldsymbol{\mu} - R_f \mathbf{1}_N = (0.06,\ 0.04)^\top$, volatilities $(0.20,\ 0.10)$, correlation 0.5, risk aversion $\gamma = 3$.

Step 1: covariance matrix. $\sigma_{12} = \rho\sigma_1\sigma_2 = 0.5 \times 0.2 \times 0.1 = 0.01$:

$$V = \begin{pmatrix} 0.04 & 0.01 \\ 0.01 & 0.01 \end{pmatrix}.$$

Step 2: inverse of a 2×2 matrix. Swap the diagonal, change the sign of the off-diagonal, divide by the determinant $ad - bc = 0.04 \times 0.01 - 0.01^2 = 0.0003$:

$$V^{-1} = \frac{1}{0.0003} \begin{pmatrix} 0.01 & -0.01 \\ -0.01 & 0.04 \end{pmatrix} = \begin{pmatrix} 33.33 & -33.33 \\ -33.33 & 133.33 \end{pmatrix}.$$

Step 3: multiply. $V^{-1}(\boldsymbol{\mu} - R_f \mathbf{1}_N) = (33.33 \cdot 0.06 - 33.33 \cdot 0.04,\ -33.33 \cdot 0.06 + 133.33 \cdot 0.04) = (0.667,\ 3.333)$.

Step 4: divide by γ. $\boldsymbol{w} = (0.222,\ 1.111)$. The weights sum to 1.333, so the investor *borrows* 33% at the risk-free rate ($w_f = 1 - 1.333 = -0.333$).

Reading the result. Asset 1 has the higher premium (6% vs. 4%) but gets the *smaller* weight: it is four times as risky (0.04 vs. 0.01) and, because the two assets are correlated, asset 2 already captures much of asset 1's exposure. This is why you cannot choose weights by looking at assets one at a time.

Additional note (not on the slides)

Same derivation in matrix form: $\max_{\boldsymbol{w}}\ \boldsymbol{w}^\top(\mathbb{E}[\boldsymbol{R}] - R_f\mathbf{1}_N) - \frac{\gamma}{2}\boldsymbol{w}^\top\mathbb{V}[\boldsymbol{R}]\boldsymbol{w}$. FOC: $(\mathbb{E}[\boldsymbol{R}] - R_f\mathbf{1}_N) - \gamma\mathbb{V}[\boldsymbol{R}]\boldsymbol{w} = \mathbf{0} \Rightarrow$ the formula above. Note that the $N = 1$ case is just the scalar version: $\mathbb{V}^{-1}$ becomes $1/\sigma^2$.

1.3 Estimation error in mean–variance weights (focus)

Since $\boldsymbol{w} = \frac{1}{\gamma}\mathbb{V}[\boldsymbol{R}]^{-1}(\mathbb{E}[\boldsymbol{R}] - R_f\mathbf{1}_N)$, the whole challenge is to **estimate $\mathbb{E}[\boldsymbol{R}]$ and $\mathbb{V}[\boldsymbol{R}]$ precisely**.

In plain words

If the inputs are wrong, the weights are wrong. Expected returns are tiny compared with the noise around them (a stock may average 8% a year but move ±30%), so after 10 years you still do not know the mean well. The covariance matrix has thousands of entries for a large N, and inverting it magnifies every small error.

1.3.1 Why $\mathbb{E}[\boldsymbol{R}]$ is hard to estimate

Precision of the mean does *not* improve by sampling more often over a fixed span:

$$R_{0,100} = \frac{P_{100}}{P_0} = \frac{P_{100}}{P_{99}} \times \frac{P_{99}}{P_{98}} \times \cdots \times \frac{P_1}{P_0} = \frac{P_{100}}{P_0}.$$

All intermediate prices cancel: only the first and last prices matter. What helps is a *longer* sample (more years), not a *denser* one.

1.3.2 Why $\mathbb{V}[\boldsymbol{R}]$ is hard to estimate

Number of parameters:

$$\underbrace{N}_{\text{variances}} + \underbrace{\tfrac{N^2-N}{2}}_{\text{covariances}} = \frac{N(N+1)}{2}. \qquad N = 100 \;\Rightarrow\; 5050 \text{ parameters.}$$

1.3.3 Why poor estimates are disastrous

- $\widehat{\mathbb{E}[\boldsymbol{R}]}$ has large error;
- $\widehat{\mathbb{V}[\boldsymbol{R}]}^{-1}$ has large error (high **condition number**);
- multiplying by $\mathbb{V}^{-1}$ **magnifies** the error in $\mathbb{E}[\boldsymbol{R}]$;
- $\Rightarrow$ badly estimated $\boldsymbol{w} \Rightarrow$ poor out-of-sample performance.

Definition: Condition number

$\kappa(\mathbb{V}) = \lambda_{\max}/\lambda_{\min}$, the ratio of the largest to the smallest eigenvalue of a symmetric positive-definite matrix. A large κ means $\mathbb{V}$ is close to singular, and small errors in inputs produce large errors in $\mathbb{V}^{-1}\boldsymbol{x}$.

Additional note (not on the slides)

Why small eigenvalues cause big weights. Write $\mathbb{V} = \sum_i \lambda_i \boldsymbol{q}_i \boldsymbol{q}_i^\top$, so $\mathbb{V}^{-1} = \sum_i \lambda_i^{-1} \boldsymbol{q}_i \boldsymbol{q}_i^\top$. The directions with the smallest (and most under-estimated) eigenvalues get multiplied by the largest factors $1/\lambda_i$. With T close to N, the sample eigenvalues spread out: the smallest are biased towards zero, so $\widehat{\mathbb{V}}^{-1}$ explodes in those directions. If $N > T$, $\widehat{\mathbb{V}}$ is singular and cannot even be inverted.

Worked example: condition number of a 2×2 matrix

Two assets with the same variance σ^2 and correlation ρ. The eigenvalues of $V = \sigma^2 \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$ are $\sigma^2(1+\rho)$ (direction "both assets together") and $\sigma^2(1-\rho)$ (direction "long one, short the other"). So

$$\kappa = \frac{1+\rho}{1-\rho}: \qquad \rho = 0.5 \Rightarrow \kappa = 3, \qquad \rho = 0.9 \Rightarrow \kappa = 19, \qquad \rho = 0.99 \Rightarrow \kappa = 199.$$

The more similar the assets, the smaller the variance of the long–short direction, and V^{-1} multiplies that direction by $1/(\sigma^2(1-\rho))$, which is huge. Any small difference in estimated means along that direction is turned into an enormous long–short bet. This is exactly the $+635\%/-535\%$ example of Chapter 4 (correlation 0.99).

Python exercise 1: why the sample $\mathbb{V}[\boldsymbol{R}]$ is ill-conditioned

N identical assets, 20% annual volatility, pairwise correlation 0.3, $T = 60$ months. True GMV weights $= 1/N$ (assets identical). Compare $\kappa(\mathbb{V})$ vs. $\kappa(\widehat{\mathbb{V}})$ and estimated GMV weights.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(1)
T = 60                    # 5 years of monthly returns
sd = 0.20 / np.sqrt(12)   # 20% annual volatility
rho = 0.3                 # correlation between all pairs
Ns = [5, 10, 25, 50]      # numbers of assets
# True covariance matrices, simulated returns, sample covariances
V = {N: sd**2 * (rho*np.ones((N, N)) + (1-rho)*np.eye(N)) for N in Ns}
R = {N: rng.multivariate_normal(np.zeros(N), V[N], size=T) for N in Ns}
Vhat = {N: np.cov(R[N], rowvar=False) for N in Ns}

# GMV weights from the sample covariance matrix
w = {N: np.linalg.solve(Vhat[N], np.ones(N)) for N in Ns}
w = {N: w[N] / w[N].sum() for N in Ns}
table = pd.DataFrame({
    "cond(V)":    [np.linalg.cond(V[N]) for N in Ns],
    "cond(Vhat)": [np.linalg.cond(Vhat[N]) for N in Ns],
    "true w":     [1/N for N in Ns],
    "min w":      [w[N].min() for N in Ns],
    "max w":      [w[N].max() for N in Ns]}, index=Ns)
print(table.round(2))
```

	cond(V)	cond(Vhat)	true w	min w	max w
5	3.14	7.22	0.20	0.05	0.38
10	5.29	22.12	0.10	-0.04	0.28
25	11.71	61.01	0.04	-0.13	0.26
50	22.43	1864.26	0.02	-0.39	0.47

Interpretation: with $N = 50$, $T = 60$, $\kappa(\widehat{\mathbb{V}})$ is about 80 times $\kappa(\mathbb{V})$; true weights are all 0.02 but estimated ones range from -0.39 to 0.47.

Additional note (not on the slides)

Code points to remember for the handwritten exam.

- Equicorrelation matrix: $\sigma^2[\rho\,\mathbf{1}\mathbf{1}^\top + (1-\rho)I]$.
- GMV weights: $\boldsymbol{w}_{GMV} = \dfrac{\mathbb{V}^{-1}\mathbf{1}_N}{\mathbf{1}_N^\top \mathbb{V}^{-1}\mathbf{1}_N}$. In code, `np.linalg.solve(V, ones)` computes $\mathbb{V}^{-1}\mathbf{1}_N$ without explicitly inverting (more stable than `np.linalg.inv`); dividing by the sum normalises to $\mathbf{1}_N^\top \boldsymbol{w} = 1$.
- `np.cov(X, rowvar=False)`: columns are variables (assets).
- `np.linalg.cond`: condition number (2-norm).

The solution (rest of the course). Find better estimators of $\mathbb{E}[\boldsymbol{R}]$ and $\mathbb{V}[\boldsymbol{R}]$ that reduce estimation error and dimension. $\mathbb{E}[\boldsymbol{R}]$ and $\mathbb{V}[\boldsymbol{R}]$ matter beyond portfolios too: cost of capital needs $\mathbb{E}[\boldsymbol{R}]$, risk management needs $\mathbb{V}[\boldsymbol{R}]$.

1.4 Portfolio weights over time and out-of-sample returns

We choose weights at each date $t \in \{1, \ldots, T\}$: $\boldsymbol{w}_t = (w_{t,1}, \ldots, w_{t,N})^\top$, with transpose $\boldsymbol{w}_t^\top = [w_{t,1}, \ldots, w_{t,N}]$.

$$\sum_{n=1}^{N} w_{t,n} = \boldsymbol{w}_t^\top \mathbf{1}_N, \qquad \mathbb{E}[R_{t,p}] = \sum_{n=1}^{N} w_{t,n}\mathbb{E}[R_{t,n}] = \boldsymbol{w}_t^\top \mathbb{E}[\boldsymbol{R}_t].$$

▶ **Key result: out-of-sample return**

Weights $\boldsymbol{w}_t$ use information up to t; performance depends on returns at $t+1$:

$$R_{t+1,p} = \boldsymbol{w}_t^\top \boldsymbol{R}_{t+1} = \sum_{n=1}^{N} w_{t,n} R_{t+1,n}.$$

This is **out-of-sample** performance. Our choice of $\boldsymbol{w}_t$ depends on our views about $R_{t+1,n}$.

In plain words

"Out of sample" means: decide today using only what you know today, then see what happens tomorrow. Any backtest that uses tomorrow's information to decide today's weights is cheating (look-ahead bias) and will look far better than reality.

What can weights depend on? The asset's expected return and risk (variance, skewness, kurtosis, downside risk, VaR, ES, tail risk), which in turn may depend on: past returns of the asset and of other assets; firm characteristics (size, profitability, ...); covariances/correlations with other assets; sensitivity to macroeconomic factors.

Data dimensions. Across N assets (**cross-section**), over T dates (**time series**), or both (**panel**).

1.5 Notation

- N, $n \in \{1, \ldots, N\}$: number of assets.
- T, $t \in \{1, \ldots, T\}$: number of observations.
- K, $k \in \{1, \ldots, K\}$: number of factors.
- $T^{\text{est}} \leq T$: length of the **estimation window**. Not standard across papers: also M (months, usually 60 or 120), D (days, usually 30, 60, 90), or τ.

Our notation	Alternatives in the literature
$\boldsymbol{w}$	$\boldsymbol{x}$, $\boldsymbol{\theta}$
γ	$\gamma = 1/\tau$, τ = risk tolerance
$\mathbb{V}[\boldsymbol{R}]$ or V_R	V, Σ_R, Σ, C
$\mathbb{E}[\boldsymbol{R}]$ or E_R	μ_R, μ
R_f (gross)	r_f (net)
$\mathbf{1}_N$	$\mathbf{1}$, $\boldsymbol{e}$, $\boldsymbol{\iota}$
T^{est}	M, D, τ

1.6 Convention regarding data: tidy vs. wide

Tidy-data principles (Tidy Finance): **(1)** every column is a variable; **(2)** every row is an observation; **(3)** every cell is a single value.

Tidy (long): $T \times N$ rows

date	asset	price
1	1	$P_{1,1}$
1	2	$P_{1,2}$
$\vdots$	$\vdots$	$\vdots$
T	N	$P_{T,N}$

Wide: dates in rows, assets in columns

	$n = 1$	$\cdots$	$n = N$
$t = 1$	$P_{1,1}$	$\cdots$	$P_{1,N}$
$\vdots$	$\vdots$	$\ddots$	$\vdots$
$t = T$	$P_{T,1}$	$\cdots$	$P_{T,N}$

▶ **Key result: rule**

Store data in **tidy** format; **pivot to wide** only for matrix calculations (covariances, correlations, $\mathbb{V}^{-1}$).

Python exercise 2: wide and tidy data

```
import numpy as np
import pandas as pd
pd.set_option("display.precision", 4)
rng = np.random.default_rng(2)
dates = pd.date_range("2024-01-31", periods=24, freq="ME")
ret = rng.normal(0.01, 0.06, size=(24, 3))
```

```
prices_wide = pd.DataFrame(100*np.cumprod(1 + ret, axis=0),
    index=dates, columns=["AAA", "BBB", "CCC"])
prices_wide.index.name = "date"
# Wide -> long (tidy): one row per (date, symbol)
prices = (prices_wide.reset_index()
    .melt(id_vars="date", var_name="symbol", value_name="price")
    .sort_values(["symbol", "date"]))

# Wrong: pct_change() runs across the stocks
prices["ret_wrong"] = prices["price"].pct_change()
# Right: compute returns within each stock
g = prices.groupby("symbol")["price"]
prices["ret"] = g.pct_change()
print(prices.iloc[22:26])
# Long -> wide, as needed for covariance matrices
ret_wide = prices.pivot(index="date", columns="symbol", values="ret")
print(ret_wide.dropna().corr().round(2))
```

row	date	symbol	price	ret_wrong	ret
22	2025-11-30	AAA	72.2815	-0.0818	-0.0818
23	2025-12-31	AAA	73.3244	0.0144	0.0144
24	2024-01-31	BBB	97.8635	0.3347	NaN
25	2024-02-29	BBB	109.4097	0.1180	0.1180

Interpretation: in row 24 the "wrong" return of 33.47% is the change from the last price of AAA to the first of BBB — not a return. With `groupby`, the first return of each stock is correctly `NaN`.

Additional note (not on the slides)

Key pandas verbs: `melt` (wide→long), `pivot` (long→wide), `groupby(...).pct_change()` (returns within each asset), `sort_values(["symbol","date"])` (needed before `pct_change`). With `yfinance` multi-ticker downloads, `.stack(level="Ticker")` also converts wide to long.

1.7 Prices and returns

▶ **Key result: return definitions**

$$\text{Gross return:} \quad R_{t,n} = \frac{P_{t,n}}{P_{t-1,n}}$$

$$\text{Net return:} \quad r_{t,n} = R_{t,n} - 1 = \frac{P_{t,n}}{P_{t-1,n}} - 1$$

$$\text{Excess return:} \quad R_{t,n} - R_{t,0} = \frac{P_{t,n}}{P_{t-1,n}} - \frac{P_{t,0}}{P_{t-1,0}}$$

$R_{t,0}$: reference asset, usually risk-free ($R_{t,f}$). Excess gross returns = excess net returns (the -1's cancel).

1.7.1 Multiperiod (compound) returns

$$R_{t,n}(h) = \frac{P_{t,n}}{P_{t-h,n}} = \frac{P_{t,n}}{P_{t-1,n}} \times \cdots \times \frac{P_{t-h+1,n}}{P_{t-h,n}} = \prod_{i=0}^{h-1} R_{t-i,n}.$$

1.7.2 Annualised returns

Returns are quoted per year. Monthly net returns are annualised by multiplying by 12; annualised gross = 1+ annualised net. With one-year returns over h years:

$$\underbrace{R_{t,n}(h)^{1/h} = \Big[\prod_{i=0}^{h-1} R_{t-i,n}\Big]^{1/h}}_{\text{geometric mean (exact)}} \approx \underbrace{\frac{1}{h}\sum_{i=0}^{h-1} R_{t-i,n}}_{\text{arithmetic mean}}.$$

Additional note (not on the slides)

Arithmetic vs. geometric mean. Always (AM–GM inequality) arithmetic $\geq$ geometric, and approximately

$$\bar{r}_{\text{arith}} - \bar{r}_{\text{geom}} \approx \tfrac{1}{2}\sigma^2.$$

Classic example: $+50\%$ then -50%: arithmetic mean = 0%, but wealth goes $1 \to 1.5 \to 0.75$, geometric mean = $\sqrt{0.75} - 1 \approx -13.4\%$.

1.7.3 Continuous compounding (log returns)

Let $p_{t,n} = \ln P_{t,n}$. Then $\ln R_{t,n} = p_{t,n} - p_{t-1,n}$ and

$$\ln R_{t,n}(h) = \ln R_{t,n} + \ln R_{t-1,n} + \cdots + \ln R_{t-h+1,n} \tag{1.1}$$

$$= p_{t,n} - p_{t-h,n}. \tag{1.2}$$

▶ **Key result: two insights**

1. **Eq.** (1.2): only the first and last prices matter. The 100-year return is identical using two prices, annual, monthly or daily data. ⇒ extra observations do *not* help estimate the mean (they *do* help for the variance). Estimates of expected returns are very imprecise; estimates of variances are much more precise.
2. **Eq.** (1.1): multiperiod log return = *sum* of one-period log returns ⇒ easy statistics (sums, not products).

In plain words

Two kinds of returns, two jobs. Simple (gross) returns are what you earn: a portfolio's simple return is the weighted average of the assets' simple returns. Log returns are convenient over time: the 10-year log return is just the sum of the yearly ones. Rule: across assets use simple returns; over time use log returns.

Disadvantage of log returns. Gross portfolio returns aggregate linearly, log portfolio returns do not:

$$R_{t,p} = \sum_n w_{t,n} R_{t,n}, \qquad \ln R_{t,p} \neq \sum_n w_{t,n} \ln R_{t,n},$$

but over short intervals $\ln R_{t,p} \approx \sum_n w_{t,n} \ln R_{t,n}$.

Intuition

- **Cross-section** (allocating across assets) $\Rightarrow$ use gross returns $R_{t,p}$ or gross excess returns $R_{t,p} - R_{t,0}$.
- **Time series** $\Rightarrow$ use log returns $\ln R_{t,p}$ or log excess returns.
- This course focuses on the **cross-section**.

Python exercise 3: sampling more often does not help estimate the mean

2,000 samples of 10 years of daily log returns, true $\mu = 8\%$, $\sigma = 20\%$ p.a.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(3)
M = 2000                 # number of simulated 10-year samples
years, d = 10, 252       # years per sample, days per year
mu, sigma = 0.08, 0.20   # true annual mean and volatility
r = rng.normal(mu/d, sigma/np.sqrt(d), size=(M, years*d))
# k-day log returns: sum k consecutive daily log returns
k = {"daily": 1, "monthly": 21, "annual": 252}
rk = {f: r.reshape(M, -1, k[f]).sum(axis=2) for f in k}
n = {f: d // k[f] for f in k}   # observations per year

# Annualized estimates in each of the M samples
mean_hat = {f: rk[f].mean(axis=1) * n[f] for f in k}
vol_hat = {f: rk[f].std(axis=1, ddof=1) * np.sqrt(n[f]) for f in k}
table = pd.DataFrame({
    "T": [rk[f].shape[1] for f in k],
    "SE of mean": [mean_hat[f].std() for f in k],
    "SE of vol":  [vol_hat[f].std() for f in k]}, index=list(k))
print(table.round(4))
```

```
             T  SE of mean  SE of vol
daily     2520      0.0632     0.0028
monthly    120      0.0632     0.0126
annual      10      0.0632     0.0460
```

- SE of the mean = 6.32% at every frequency = $\sigma/\sqrt{\text{years}} = 0.20/\sqrt{10}$.
- SE of volatility falls from 4.6% (annual) to 0.28% (daily).
- Even with 10 years of daily data, a 95% CI for the mean is $8\% \pm 12.6\%$: we cannot tell if the mean is positive.

Worked example: how many years do we need?

The standard error of an annual mean is $\sigma/\sqrt{\text{years}}$. For a single stock ($\sigma = 30\%$) and for the market ($\sigma = 16\%$):

Years of data	5	10	25	100
SE, single stock ($\sigma = 30\%$)	13.4%	9.5%	6.0%	3.0%
SE, market ($\sigma = 16\%$)	7.2%	5.1%	3.2%	1.6%

A 95% confidence interval is about ± 2 SE. Even with 100 years of data, the market's mean is known only to within $\pm 3.2\%$, while the equity premium itself is about 5–7%. For volatility, by contrast, a few years of daily data are enough.

Additional note (not on the slides)

The math behind it. With T years sampled n times per year, each period has mean μ/n and variance σ^2/n. The annualised mean estimator $n \cdot \frac{1}{nT}\sum r_i$ has variance $n^2 \cdot \frac{1}{(nT)^2} \cdot nT \cdot \frac{\sigma^2}{n} = \frac{\sigma^2}{T}$: independent of n. For volatility, $\text{SE}(\hat{\sigma}) \approx \sigma/\sqrt{2nT}$, which shrinks as n grows. Check: $0.20/\sqrt{2 \cdot 2520} = 0.0028$. ✓

Python exercise 4: simple and log returns

10 years of monthly gross returns, two assets, mean 0.8%, vol 5% per month.

```
import numpy as np
rng = np.random.default_rng(4)
T = 120   # 10 years of monthly returns on 2 assets
R = 1 + rng.normal(0.008, 0.05, size=(T, 2))  # gross
r = np.log(R)                                 # log
# 1. Product of gross returns = exp(sum of log returns)
print("prod(R):", R[:, 0].prod().round(6))
print("exp(sum r):", np.exp(r[:, 0].sum()).round(6))
# 2. Annualized arithmetic mean vs geometric mean
arith = 12 * (R[:, 0] - 1).mean()
geom = R[:, 0].prod()**(12/T) - 1
half_var = 12 * (R[:, 0] - 1).var() / 2
print("arithmetic:", arith.round(4), "geometric:", geom.round(4),
      "sigma^2/2:", half_var.round(4))
# 3. Portfolio log return vs weighted sum of log returns
w = np.array([0.6, 0.4])
gap_m = np.abs(np.log(R @ w) - r @ w).mean()
Ra = R.reshape(10, 12, 2).prod(axis=1)  # annual gross
gap_a = np.abs(np.log(Ra @ w) - np.log(Ra) @ w).mean()
print("mean gap, monthly:", gap_m.round(5), "annual:", gap_a.round(5))
```

```
prod(R): 2.079249
exp(sum r): 2.079249
arithmetic: 0.0892 geometric: 0.0759 sigma^2/2: 0.0157
mean gap, monthly: 0.00066 annual: 0.00742
```

1. Product of gross returns = exp($\sum$ log returns) exactly.
2. Arithmetic (8.92%) exceeds geometric (7.59%) by 1.33%, close to $\sigma^2/2 = 1.57\%$.
3. Log-aggregation gap: 0.07% monthly vs. 0.74% annual (about 11 times larger).

1.7.4 Returns with dividends

$$R_{t,n} = \frac{P_{t,n} + D_{t,n}}{P_{t-1,n}}, \qquad \ln R_{t,n} = \ln(P_{t,n} + D_{t,n}) - \ln P_{t-1,n}.$$

1.7.5 Distribution of returns

Returns are random variables, characterised by their **joint**, **conditional** or **unconditional** distribution.

Unconditional: IID lognormal. Convenient assumption: gross returns IID lognormal $\Leftrightarrow$ $\ln R_{t,n} \sim \mathcal{N}(\mu_n, \sigma_n^2)$. Lognormal has lower bound 0 $\Rightarrow$ respects **limited liability**; $\ln R$ ranges over $(-\infty, +\infty)$.

▶ **Key result: lognormal moments**

If $\ln X \sim \mathcal{N}(\mu, \sigma^2)$: $\mathbb{E}[X^m] = \exp(m\mu + \frac{1}{2}m^2\sigma^2)$. Hence

$$\begin{aligned}
\mathbb{E}[R_n] &= \exp(\mu_n + \tfrac{1}{2}\sigma_n^2) \\
\mathbb{E}[R_n^2] &= \exp(2\mu_n + 2\sigma_n^2) \\
\mathbb{V}[R_n] &= \mathbb{E}[R_n^2] - (\mathbb{E}[R_n])^2 = \exp(2\mu_n + \sigma_n^2)(\exp(\sigma_n^2) - 1).
\end{aligned}$$

Intuition

$\mathbb{E}[R] = e^{\mu+\sigma^2/2} > e^{\mu}$: the mean of the gross return exceeds the exponential of the mean log return by the factor $e^{\sigma^2/2}$ (Jensen's inequality). Taking logs: $\ln \mathbb{E}[R] = \mu + \sigma^2/2$ — the same $\sigma^2/2$ gap between arithmetic and geometric means seen in Exercise 4.

1.7.6 Are returns Normal? Skewness and kurtosis

At short horizons returns are *not* Normal: weak skewness, strong **excess kurtosis** (fat tails).

$$S[X] = \mathbb{E}\left[\frac{(X-\mu)^3}{\sigma^3}\right], \qquad K[X] = \mathbb{E}\left[\frac{(X-\mu)^4}{\sigma^4}\right].$$

Normal: $S = 0$, $K = 3$; excess kurtosis $= K - 3$. Sample estimators (dividing by T):

$$\hat{S} = \frac{1}{\hat{\sigma}^3}\frac{1}{T}\sum_{t=1}^{T}(x_t - \hat{\mu})^3, \quad \hat{K} = \frac{1}{\hat{\sigma}^4}\frac{1}{T}\sum_{t=1}^{T}(x_t - \hat{\mu})^4, \quad \hat{\mu} = \frac{1}{T}\sum x_t, \quad \hat{\sigma}^2 = \frac{1}{T}\sum(x_t - \hat{\mu})^2.$$

Under Normality, for large T: $\hat{S} \sim \mathcal{N}(0, 6/T)$, $\hat{K} \sim \mathcal{N}(3, 24/T)$.

Additional note (not on the slides)

Jarque–Bera test combines both:

$$JB = T\left[\frac{\hat{S}^2}{6} + \frac{(\hat{K}-3)^2}{24}\right] \overset{H_0}{\sim} \chi^2_2.$$

Large JB (small p-value) $\Rightarrow$ reject Normality. It is just the sum of the two squared z-statistics implied by the asymptotic distributions above.

In plain words

Real daily returns have many more "crash" and "boom" days than a bell curve predicts (fat tails). Kurtosis measures this: 3 for a Normal, often 5–10 for daily stock returns. Over a month, many daily shocks add up and the distribution looks more Normal, but not completely.

Python exercise 5: testing whether returns are Normal

10,500 daily returns from Student-t with 5 d.o.f. (fat tails), scaled to 1% daily vol; monthly = sum of 21 daily log returns.

```
import numpy as np
import pandas as pd
from scipy import stats
rng = np.random.default_rng(5)
T, nu = 21*500, 5    # 500 months of days; t with 5 df
z = rng.standard_t(nu, size=T) / np.sqrt(nu/(nu-2))
daily = 0.01 * z     # 1% daily volatility, fat tails
monthly = daily.reshape(-1, 21).sum(axis=1)
x = {"daily": daily, "monthly": monthly}

# Moves larger than 4 standard deviations
n4 = {f: np.sum(np.abs(x[f] - x[f].mean()) > 4*x[f].std()) for f in x}
table = pd.DataFrame({
    "T": [len(x[f]) for f in x],
    "skew": [stats.skew(x[f]) for f in x],
    "kurt": [stats.kurtosis(x[f], fisher=False) for f in x],
    "JB p-value": [stats.jarque_bera(x[f]).pvalue for f in x],
    "4-sigma": [n4[f] for f in x],
    "if Normal": [len(x[f])*2*stats.norm.sf(4) for f in x]},
    index=list(x))
print(table.round(3))
```

```
             T    skew   kurt  JB p-value  4-sigma  if Normal
daily    10500  -0.104  7.256       0.000       40      0.665
monthly    500   0.076  2.932       0.751        0      0.032
```

- Daily: kurtosis 7.3 (SE under Normality $\sqrt{24/T} \approx 0.05$); 40 four-sigma days vs. fewer than one expected.
- Monthly: kurtosis 2.9, JB $p = 0.75$ $\Rightarrow$ cannot reject Normality: sums of many fat-tailed draws are close to Normal (CLT).

- Caveat: in real data volatility is time-varying, so monthly returns are also fat-tailed (less than daily).

Additional note (not on the slides)

`np.sqrt(nu/(nu-2))`: the variance of a Student-t with ν d.o.f. is $\nu/(\nu-2)$, so dividing standardises it to unit variance. `stats.kurtosis(..., fisher=False)` returns K (Normal = 3); the default `fisher=True` returns *excess* kurtosis (Normal = 0).

1.8 Obtaining financial and macroeconomic data

Three kinds of data: (1) stock prices, (2) stock characteristics, (3) macroeconomic data. Code adapted from an earlier edition of Tidy Finance (the current edition uses its own `tidyfinance` package and `polars`).

1.8.1 Stock-price data (Yahoo! Finance)

```
import pandas as pd
import numpy as np
import yfinance as yf

prices = (yf.download(tickers="AAPL", start="2000-01-01", end="2024-12-31",
                      auto_adjust=False,        # keep "Adj Close"
                      multi_level_index=False)  # one level of column names
  .reset_index()
  .assign(symbol="AAPL")
  .rename(columns={"Date": "date", "Open": "open", "High": "high",
                   "Low": "low", "Close": "close",
                   "Adj Close": "adjusted", "Volume": "volume"}))
prices.head()
```

	date	open	high	low	close	adjusted	volume	symbol
0	2000-01-03	0.936384	1.004464	0.907924	0.999442	0.848323	535796800	AAPL
1	2000-01-04	0.966518	0.987723	0.903460	0.915179	0.776801	512377600	AAPL

▶ **Key result: adjusted prices**

Adjusted prices correct for corporate actions (stock splits, dividends) that change quoted prices but have no direct impact on investors. **Use adjusted prices** to compute returns. Yahoo rescales adjusted prices after every dividend/split, so values downloaded today differ from older downloads.

Apple splits: 2:1 (1987, 2000, 2005), 7:1 (2014), 4:1 (2020). Opening price on 2000-01-04: \$0.9665 (split-adjusted); on 2023-01-04: \$126.98.

Python exercise 6: adjusting prices for splits and dividends

250 days, start at \$400, 4-for-1 split on day 150, \$1 dividend on day 200. Adjustment factor on each event date multiplies *all earlier* prices: split $\to 1/4$; dividend $\to P_t/(P_t + D)$.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(6)
n = 250                                          # trading days
true_ret = rng.normal(0.0004, 0.015, size=n)  # total returns
true_ret[0] = 0.0
split_day, ratio = 150, 4                        # 4-for-1 split
div_day, div = 200, 1.00                         # $1 dividend per (new) share
G = np.cumprod(1 + true_ret)
day = np.arange(n)
shares = np.where(day >= split_day, ratio, 1)
close = (400*G/shares - div*(G/G[div_day])*(day >= div_day))

# Adjustment factor on each event date; multiplies all prices BEFORE it
f = np.ones(n)
f[split_day] = 1 / ratio
f[div_day] = close[div_day] / (close[div_day] + div)
adj = np.append(np.cumprod(f[::-1])[::-1][1:], 1.0)
df = pd.DataFrame({"close": close, "adjusted": close*adj})
df["raw_ret"] = df["close"].pct_change()
df["adj_ret"] = df["adjusted"].pct_change()
print(df.loc[[split_day, div_day]].round(4))
err = np.abs(df["adj_ret"].values[1:] - true_ret[1:]).max()
print("Largest error of adjusted returns:", err)
```

```
        close  adjusted  raw_ret  adj_ret
150  102.2968  101.4969  -0.7536  -0.0143
200  126.8846  126.8846  -0.0280  -0.0203
Largest error of adjusted returns: 3.8e-16
```

- Split date: raw "return" -75.4%; adjusted -1.43% (true).
- Dividend date: raw -2.80% misses the \$1 dividend; adjusted -2.03% is the true total return.
- Yahoo uses the similar factor $1 - D/P_{t-1}$: very close, not identical, to total returns.

Additional note (not on the slides)

`np.cumprod(f[::-1])[::-1]` is a *reverse cumulative product*: entry t is the product of all factors from t to the end. Shifting by one (`[1:]`, append 1) means each price is multiplied by the factors of all events that happen *after* it — exactly the backward adjustment.

1.8.2 From prices to returns

$$r_t = \frac{P_t}{P_{t-1}} - 1.$$

```
returns = (prices
  .sort_values("date")
  .assign(ret=lambda x: x["adjusted"].pct_change())
  .get(["symbol", "date", "ret"]))
```

First return is `NaN`; e.g. AAPL 2000-01-04: -8.43%.

Many stocks.

1. Download the tickers of the index constituents (e.g. DJIA).
2. Pass the list to `yf.download(tickers=[...])`.
3. Convert to long format with `.stack(level="Ticker")`.
4. Compute returns with `groupby("symbol")`.

Learn to do it for S&P 500, Nasdaq, DAX (see Brugière 2020, Sec. 9.5).

Python exercise 7: survivorship bias

Using *today's* constituents to compute past returns keeps only survivors. 1,000 stocks, 120 months, true mean 0.8%, vol 12% per month; delisted once price $< 30\%$ of start (the return in that month is included).

```
import numpy as np
rng = np.random.default_rng(7)
N, T = 1000, 120
ret = rng.normal(0.008, 0.12, size=(N, T))
price = np.cumprod(1 + ret, axis=1)
dead = np.maximum.accumulate(price < 0.3, axis=1)   # once dead, always dead
listed = np.hstack([np.ones((N, 1), bool), ~dead[:, :-1]])
survivor = ~dead[:, -1]
ret_listed = np.where(listed, ret, np.nan)
all_firms = np.nanmean(ret_listed, axis=0).mean()
survivors = ret[survivor].mean()
print("Share of firms surviving:", survivor.mean().round(3))
print("Mean return, all listed firms:", all_firms.round(4))
print("Mean return, survivors only:", survivors.round(4))
```

```
Share of firms surviving:      0.663
Mean return, all listed firms: 0.0079   # ~ true 0.008
Mean return, survivors only:   0.0132   # survivorship bias
```

▶ **Key result: survivorship bias**

Survivors earn 1.32%/month vs. true 0.8%: overstatement of about 6% per year. For backtests use the constituents **at each past date** (e.g. CRSP), not today's.

1.8.3 Stock-characteristics data

Sample moments (simplest estimators of $\mathbb{E}[\boldsymbol{R}]$ and $\mathbb{V}[\boldsymbol{R}]$):

$$\hat{\mu}_{R_n} = \frac{1}{T}\sum_{t=1}^{T} R_{t,n}, \quad \hat{\sigma}_{R_{nn}} = \frac{1}{T}\sum_{t=1}^{T}(R_{t,n} - \hat{\mu}_{R_n})^2, \quad \hat{\sigma}_{R_{nm}} = \frac{1}{T}\sum_{t=1}^{T}(R_{t,n} - \hat{\mu}_{R_n})(R_{t,m} - \hat{\mu}_{R_m}).$$

Note: pandas `.var()` and `.cov()` divide by $T-1$, not T (`ddof=1` by default; NumPy `np.var` uses `ddof=0`).

Alternatively, use firm **characteristics** — size, value (book-to-market), profitability, investment — to estimate $\mathbb{E}[R_n]$ and $\mathbb{V}[R_n]$. The source is **Ken French's data library**, which we read with the package `pandas-datareader`.

```
import pandas_datareader as pdr
start_date = "1960-01-01"
end_date = "2022-12-31"

factors_ff3_monthly_raw = pdr.DataReader(
  name="F-F_Research_Data_Factors", data_source="famafrench",
  start=start_date, end=end_date)[0]
factors_ff3_monthly = (factors_ff3_monthly_raw
  .divide(100)                          # percent -> decimal
  .reset_index(names="month")
  .assign(month=lambda x: pd.to_datetime(x["month"].astype(str)))
  .rename(str.lower, axis="columns")
  .rename(columns={"mkt-rf": "mkt_excess"}))
```

- FF3: `mkt_excess`, `smb` (size), `hml` (value), `rf`.
- FF5 (`"F-F_Research_Data_5_Factors_2x3"`): adds `rmw` (profitability) and `cma` (investment). Same code, different name.
- Also available: daily FF factors, 10 industry portfolios, q-factors (Hou, Xue, Zhang 2015).

Other sources. PyAnomaly, getFamaFrenchFactors; **Open Source Asset Pricing** (330+ predictors; Chen–Zimmermann 2022); `investpy` (Investing.com, non-US stocks, funds, ETFs, FX, bonds, crypto); Gu–Kelly–Xiu (2020) for ML; Open Source Bond Asset Pricing (Dickerson–Mueller–Robotti 2023); PyCleanTrace for TRACE bond data (Dick-Nielsen–Poulsen 2019).

EDGAR. SEC filing database, free. Python APIs: SEC Edgar Downloader, python-edgar, OpenEDGAR, EDGAR Tools, SEC EDGAR API; commercial sec-api.io. Non-US equivalents: France — Registre du commerce et des sociétés; UK — Companies House; Canada — SEDAR+; others for Germany, Ireland, Italy, Switzerland.

1.8.4 Macroeconomic data

Macro variables are used to predict the market equity premium. **Welch and Goyal (2008)** re-examine many predictors:

Code	Definition
`dp`	log dividend–price ratio: $\ln D_{12} - \ln P$
`dy`	log dividend yield: $\ln D_{12} - \ln P_{t-1}$
`ep`	log earnings–price ratio: $\ln E_{12} - \ln P$
`de`	log dividend payout: $\ln D_{12} - \ln E_{12}$
`tbl`, `lty`, `ltr`	3-month T-bill rate, long-term yield, long-term govt bond return
`tms`	term spread: `lty` − `tbl`
`dfy`	default spread: BAA − AAA yields
`svar`, `infl`, `ntis`, `b/m`	stock variance, inflation, net equity expansion, book-to-market
`rp_div`	target: next-month log excess return incl. dividends

```
sheet_id = "1bM7vCWd3WOt95Sf9qjLPZjoiafgF_8EG"
sheet_name = "macro_predictors.xlsx"
macro_predictors_link = ("https://docs.google.com/spreadsheets/d/" + sheet_id
                         + "/gviz/tq?tqx=out:csv&sheet=" + sheet_name)
macro_predictors = (
  pd.read_csv(macro_predictors_link, thousands=",")
  .assign(
```

```
    month=lambda x: pd.to_datetime(x["yyyymm"], format="%Y%m"),
    IndexDiv=lambda x: x["Index"] + x["D12"],
    logret=lambda x: np.log(x["IndexDiv"]) - np.log(x["IndexDiv"].shift(1)),
    Rfree=lambda x: np.log(x["Rfree"] + 1),
    rp_div=lambda x: (x["logret"] - x["Rfree"]).shift(-1),
    dp=lambda x: np.log(x["D12"]) - np.log(x["Index"]),
    dy=lambda x: np.log(x["D12"]) - np.log(x["Index"].shift(1)),
    ep=lambda x: np.log(x["E12"]) - np.log(x["Index"]),
    de=lambda x: np.log(x["D12"]) - np.log(x["E12"]),
    tms=lambda x: x["lty"] - x["tbl"],
    dfy=lambda x: x["BAA"] - x["AAA"])
  .get(["month", "rp_div", "dp", "dy", "ep", "de", "svar", "b/m",
        "ntis", "tbl", "lty", "ltr", "tms", "dfy", "infl"])
  .query("month >= @start_date and month <= @end_date")
  .dropna())
```

Additional note (not on the slides)

`.shift(-1)` on `rp_div` aligns the *next* month's excess return with *this* month's predictors: a regression $rp_{t+1} = a + b\,x_t + \varepsilon_{t+1}$ is then a genuine predictive regression (no look-ahead bias).

FRED (St. Louis Fed, 817,000 series):

```
cpi_monthly = (pdr.DataReader(name="CPIAUCNS", data_source="fred",
                              start=start_date, end=end_date)
  .reset_index(names="month")
  .rename(columns={"CPIAUCNS": "cpi"})
  .assign(cpi=lambda x: x["cpi"] / x["cpi"].iloc[-1]))  # rebase to last month = 1
```

Any series: just change the key (e.g. gold-ore PPI: `PCU2122212122210`).

WRDS / CRSP / Compustat (optional, not covered in class): SQL (PostgreSQL) database; Tidy Finance explains access and storage in SQLite.

1.9 Takeaways of Chapter 1

1. Key formula: $\boldsymbol{w} = \frac{1}{\gamma}\mathbb{V}[\boldsymbol{R}]^{-1}(\mathbb{E}[\boldsymbol{R}] - R_f\mathbf{1}_N)$; the whole course is about estimating $\mathbb{E}[\boldsymbol{R}]$ and $\mathbb{V}[\boldsymbol{R}]$ well.
2. $\mathbb{E}[\boldsymbol{R}]$: imprecise, because only the first and last prices matter; SE $= \sigma/\sqrt{\text{years}}$ regardless of frequency.
3. $\mathbb{V}[\boldsymbol{R}]$: $N(N+1)/2$ parameters; the sample matrix is ill-conditioned when N is large relative to T; $\mathbb{V}^{-1}$ magnifies errors in means.
4. Performance must be judged **out of sample**: $R_{t+1,p} = \boldsymbol{w}_t^\top \boldsymbol{R}_{t+1}$.
5. Tidy data; pivot to wide only for matrix algebra; compute returns *within* each asset (`groupby`).
6. Gross returns aggregate across assets; log returns aggregate over time. Arithmetic − geometric mean $\approx \sigma^2/2$.
7. Daily returns are fat-tailed; monthly closer to Normal.

8. Use adjusted prices; beware survivorship bias.

Common mistakes to avoid

1. Computing returns with `pct_change()` on a tidy column without `groupby("symbol")`: the first "return" of each stock is computed from the previous stock's last price.
2. Using the raw `Close` price instead of the `Adj Close`: splits show up as -50% or -75% "returns" and dividends are missed.
3. Annualising volatility by multiplying by 12 instead of $\sqrt{12}$ (only means and variances scale with time).
4. Forgetting that `pandas .std()` uses $T-1$ and `np.std` uses T.
5. Averaging log returns across assets to get a portfolio return (only simple returns aggregate across assets).
6. Using today's index constituents to study the past (survivorship bias).
7. Using `np.linalg.inv` on a near-singular matrix without noticing: always look at the condition number or use `np.linalg.solve`.

1.10 Self-test questions (quiz/exam practice)

1. If risk aversion doubles, what happens to w_1? *(It halves.)*
2. How many parameters in $\mathbb{V}[\boldsymbol{R}]$ for $N = 50$? *(1275.)*
3. Going from monthly to daily data over the same 10 years, does the SE of the mean fall? *(No; only the SE of volatility falls.)*
4. Why is $\ln R_{t,p} \neq \sum_n w_{t,n} \ln R_{t,n}$? When is the approximation good? *(Log is non-linear; good over short intervals.)*
5. Excess gross vs. excess net returns? *(Identical.)*
6. Kurtosis of a Normal? SE of $\hat{K}$ with $T = 10{,}000$? *(3; $\sqrt{24/10000} \approx 0.049$.)*
7. A stock has a 2-for-1 split. What is the raw return on that day if the true return is 0? *(-50%.)*
8. Write by hand the Python code to compute returns from a tidy DataFrame with columns `symbol, date, adjusted`.

```
df = df.sort_values(["symbol", "date"])
df["ret"] = df.groupby("symbol")["adjusted"].pct_change()
```

9. Write the code for GMV weights given a covariance matrix `V`.

```
ones = np.ones(V.shape[0])
w = np.linalg.solve(V, ones)
w = w / w.sum()
```

10. Write the code for MV weights given `mu`, `V`, `rf`, `gamma`.

```
w = np.linalg.solve(V, mu - rf * np.ones(len(mu))) / gamma
```

1.11 To do for next class

- Read the *Financial Data* section of Tidy Finance with Python.
- Start Assignment #1 (covers Ch. 1 and Ch. 2; due 11 Nov, 22:00 CET).

Chapter 2

Performance Measurement

2.1 What we want to do in Chapter 2

- Learn how to measure the performance of a portfolio strategy and why it must be **out of sample**: in-sample returns suffer from **look-ahead bias**.
- Study many performance metrics. The key one, used for the rest of the course, is the **Sharpe ratio**.
- Test whether the difference between the Sharpe ratios of two portfolios is **statistically significant** (delta method and bootstrap).

In plain words

Chapter 2 is about *how to keep score*. Before comparing strategies we must agree on (1) how to compute returns honestly (out of sample), (2) which numbers to look at (mainly the Sharpe ratio, plus risk measures and turnover), and (3) how to tell whether one strategy is *really* better or just lucky (statistical tests).

2.2 Static, myopic, and dynamic portfolios

Type	Definition	Example (25-year-old saving for retirement at 65)
Static	Form the portfolio at $t = 0$ and hold it until T.	Chooses a portfolio once and holds it for 40 years.
Myopic	Rebalance at each date, optimising *only one period ahead* (ignore future rebalancing).	Rebalances every year, optimising only for the next year.
Dynamic	Rebalance at each date, *anticipating* future rebalancing.	Rebalances every year, knowing she will rebalance again next year.

▶ **Key result: the course studies myopic portfolios**

- **Not static**: static portfolios are suboptimal relative to myopic (and dynamic) ones.
- **Not dynamic**: they require estimating many more parameters, so they perform poorly out of sample (see Merton 1971, 1990; Campbell–Viceira 2002).
- Ignoring estimation error, myopic is suboptimal relative to dynamic, but in practice estimation error dominates.

2.3 Backtesting: in-sample vs. out-of-sample performance

Notation. $R_{t,p}$ is the gross return at date t of portfolio p (convention: $R_{\text{date,asset}}$). A myopic portfolio $\boldsymbol{w}_{0,p}$ chosen at $t = 0$ is evaluated by its return $R_{1,p}$.

▶ **Key result: weights must depend only on past information**

$\boldsymbol{w}_{0,p}$ may depend on information at $t = 0, -1, -2, \ldots$ but **never** on information from $t = 1, 2, \ldots$

- If $\boldsymbol{w}_{0,p}$ uses only past information $\Rightarrow$ $R_{1,p}$ is an **out-of-sample** return.
- If $\boldsymbol{w}_{0,p}$ also uses future information $\Rightarrow$ $R_{1,p}$ is an **in-sample** return (not realistic: look-ahead bias).

The sequence of weights $\boldsymbol{w}_p = \{\boldsymbol{w}_{0,p}, \boldsymbol{w}_{1,p}, \ldots, \boldsymbol{w}_{T-1,p}\}$ generates the out-of-sample returns $\boldsymbol{R}_p = \{R_{1,p}, R_{2,p}, \ldots, R_{T,p}\}$.

$t = 0$	$t = 1$	$t = 2$	$\cdots$	$t = T - 1$	$t = T$
$\boldsymbol{w}_{0,p}$	$R_{1,p}$, $\boldsymbol{w}_{1,p}$	$R_{2,p}$, $\boldsymbol{w}_{2,p}$	$\cdots$	$R_{T-1,p}$, $\boldsymbol{w}_{T-1,p}$	$R_{T,p}$

2.3.1 Rolling vs. expanding window

- **Rolling window**: fixed-length lookback. E.g. a 5-year window: the decision in 2020 uses 2015–2019, in 2021 uses 2016–2020, in 2022 uses 2017–2021. Use it if the distant past is *not* relevant or is different from the recent past.
- **Expanding window**: fixed start, growing length. The decision in 2020 uses 2015–2019, in 2021 uses 2015–2020, in 2022 uses 2015–2021. Use it if the distant past is *similar* to the recent past.
- **Exponentially declining weights** on past data: a compromise.

```
# pandas: rolling, expanding, exponentially weighted (EWMA) estimates
mu_roll = ret.rolling(window=60).mean()        # last 60 months
mu_exp  = ret.expanding(min_periods=60).mean() # all data since the start
vol_ewm = ret.ewm(halflife=12).std()           # weights decay with half-life 12 months
# IMPORTANT: to use them as forecasts at t for t+1, shift by one period
mu_forecast = ret.rolling(60).mean().shift(1)
```

Additional note (not on the slides)

`rolling(60).mean()` at date t *includes* the return at t. To form weights at $t-1$ for the return at t without look-ahead bias, use `.shift(1)`, or loop over `R.iloc[t-60:t]` as in the backtest skeleton of Notebook 00.

2.4 Measures of portfolio performance

Three groups: (1) mean returns; (2) risk of returns; (3) mean and risk combined.

2.4.1 Measures of mean returns

1. **Total mean return** of the portfolio.
2. **Systematic component**: the reward for bearing systematic risk, from a factor model.
3. **Alpha**: the part of the mean return unexplained by the factor model. If the model is correct, $\alpha = 0$. (In Chapter 8: part of alpha is a reward for unsystematic risk.)
4. **Outperformance frequency**: the average fraction of times the portfolio has a higher cumulative return than the benchmark within twelve months from the start of each such period.

Return factors (long–short portfolios): $R_{mkt} - R_f$ (market); R_{smb} small minus big (size); R_{hml} high minus low book-to-market (value); R_{umd} up minus down (momentum); R_{rmw} robust minus weak profitability; R_{cma} conservative minus aggressive investment.

▶ **Key result: factor models of expected returns**

$$\begin{aligned}
\text{CAPM (Sharpe 1964):} &\quad E[R_p] - R_f = \beta_{p,mkt} E[R_{mkt} - R_f] \\
\text{FF3 (1993):} &\quad \ldots + \beta_{p,smb} E[R_{smb}] + \beta_{p,hml} E[R_{hml}] \\
\text{Carhart 4F (1997):} &\quad \text{FF3} + \beta_{p,umd} E[R_{umd}] \\
\text{FF5 (2015):} &\quad \text{FF3} + \beta_{p,rmw} E[R_{rmw}] + \beta_{p,cma} E[R_{cma}] \\
\text{FF6 (2018):} &\quad \text{FF5} + \beta_{p,umd} E[R_{umd}]
\end{aligned}$$

Also: q-factor models (Hou, Xue, Zhang 2015, 2017; Hou, Mo, Xue, Zhang 2019); K statistical (*latent*) factors from PCA of the covariance matrix (Connor–Goldberg–Korajczyk 2010; Kozak–Nagel–Santosh 2018, 2020). Dello-Preite, Uppal, Zaffaroni, Zviadadze (2025): once mean returns are adjusted for *unsystematic* risk, the only systematic factor needed is the market (or market and size).

Additional note (not on the slides)

How betas and alphas are estimated in practice: a time-series OLS regression of the portfolio's excess return on the factors,

$$R_{t,p} - R_{t,f} = \alpha_p + \beta_{p,mkt}(R_{t,mkt} - R_{t,f}) + \beta_{p,smb}R_{t,smb} + \beta_{p,hml}R_{t,hml} + \varepsilon_{t,p}.$$

The intercept is the alpha; its t-statistic tests $\alpha_p = 0$.

```
import statsmodels.api as sm
X = sm.add_constant(ff3[["mkt_excess", "smb", "hml"]])
fit = sm.OLS(port_excess, X).fit()
alpha, betas = fit.params["const"], fit.params[1:]
print(fit.tvalues["const"])        # t-stat of alpha
```

2.4.2 Measures of the riskiness of returns

1. **Volatility**: standard deviation of returns.
2. **Semi-volatility**: like volatility, but only observations below the mean:

$$\text{Semi-vol} = \sqrt{\frac{1}{T}\sum_{t:\, x_t < \hat{\mu}} (x_t - \hat{\mu})^2}.$$

3. **Skewness** and 4. **Kurtosis** (Chapter 1).
5. **Average maximum drawdown**: time-series average of the maximum percentage loss of portfolio value $V(\tau)$ over any period $\tau_1 \to \tau_2$ within the last twelve months:

$$MDD = \frac{1}{T-12}\sum_{t=12}^{T-1} \max_{t-11 \le \tau_1 < \tau_2 \le t} \left\{0,\, 1 - \frac{V(\tau_2)}{V(\tau_1)}\right\} \times 100.$$

6. **Value at Risk (VaR)**: the loss that is exceeded with a given probability (1% or 5%) over a given horizon, in normal market conditions. E.g. a one-week 99% VaR of \$10m: a 1% chance of losing more than \$10m in a week, i.e. once every 100 weeks, about once every two years.
7. **Conditional VaR (CVaR)** at q%: the expected return in the worst q% of cases. Also called average VaR, **expected shortfall**, expected tail loss, superquantile. More sensitive than VaR to the *shape* of the tail.

Additional note (not on the slides)

In terms of the return quantile q_α (e.g. $\alpha = 1\%$): $\text{VaR}_\alpha = -q_\alpha$ and $\text{CVaR}_\alpha = -E[R \mid R \le q_\alpha]$, so CVaR $\ge$ VaR always. VaR says nothing about *how much* you lose beyond the threshold; CVaR does. Two tails with the same VaR can have very different CVaRs.

Worked example: VaR and CVaR under Normality

Monthly excess return with mean 0.5% and volatility 4%, Normal. The 1% quantile of a standard Normal is $z_{0.01} = -2.326$.

- 1% quantile of the return: $0.005 - 2.326 \times 0.04 = -8.8\%$, so $\text{VaR}_{1\%} = 8.8\%$ ("in 1 month out of 100 we lose more than 8.8%").
- CVaR for a Normal: $\text{CVaR}_\alpha = \sigma\,\varphi(z_\alpha)/\alpha - \mu$, with φ the standard Normal density: $0.04 \times 0.02665/0.01 - 0.005 = 10.2\%$ ("when we are in the worst 1% of months, we lose 10.2% on average").

Compare with Python exercise 1: strategy A (Normal) has VaR 9.0% and CVaR 10.4%, close to these values; strategy B (crash risk) has VaR 20.7%: the Normal formula would underestimate its risk by more than half.

2.4.3 Measures combining mean and risk

▶ **Key result: the Sharpe ratio (main measure of the course)**

$$SR_p = \frac{E[R_p - R_f]}{\sqrt{V[R_p - R_f]}} = \frac{\text{excess mean return}}{\text{total portfolio risk}}.$$

For IID returns over h periods the numerator scales with h and the denominator with $\sqrt{h}$, so SR scales with $\sqrt{h}$: **annualise a monthly Sharpe ratio by multiplying by $\sqrt{12} \approx 3.46$.**

In plain words

The Sharpe ratio answers: "how much extra return do I get for each unit of risk I take?" An annual SR of 0.5 means an expected excess return of half a standard deviation per year. The stock market's long-run SR is around 0.4–0.5. Because returns grow with time and volatility with its square root, the SR grows with the square root of the horizon.

Correct annualisation (Lo 2002). $\sqrt{12}$ is correct only if returns have no serial correlation. With first-order autocorrelation ρ (AR(1)), the multiplier for q periods is

$$\eta(q) = \frac{q}{\sqrt{q + 2\sum_{k=1}^{q-1}(q-k)\rho^k}}.$$

Positive autocorrelation $\Rightarrow$ multiplier smaller than $\sqrt{12}$ (the $\sqrt{12}$ rule overstates the SR); negative $\Rightarrow$ larger.

Additional note (not on the slides)

Why. The variance of the sum of q correlated returns is $V\left[\sum_{i=1}^{q} x_i\right] = \sigma^2\left[q + 2\sum_{k=1}^{q-1}(q-k)\rho_k\right]$: there are $q-k$ pairs at lag k. The mean of the sum is $q\mu$. So the q-period SR is $\frac{q\mu}{\sigma\sqrt{q+2\sum(q-k)\rho_k}} = \eta(q)\, SR_1$. For an AR(1), $\rho_k = \rho^k$. With $\rho = 0$, $\eta(q) = \sqrt{q}$.

Limitations of the Sharpe ratio.

- Appropriate for *efficient portfolios*, not individual assets (it ignores correlations).
- Not appropriate when returns have high skewness and kurtosis (only two moments).
- Ignores the full distribution (the **Omega** measure uses it).
- Still, it is the main measure used throughout the course.

Measure	Formula	Notes
Sortino	$\frac{E[R_p] - R_{\text{target}}}{\sqrt{DV}}$, $DV = \frac{1}{T}\sum \min(R_{t,p} - R_{\text{target}}, 0)^2$	Penalises only *downside* deviations from the target.
Treynor	$\frac{E[R_p - R_f]}{\beta_{p,mkt}}$, $\beta_{p,mkt} = \frac{C[R_p, R_{mkt}]}{V[R_{mkt}]}$	Risk = systematic only; ignores unsystematic risk. SR and TR rank, but do not say *by how much* one is better.
M² / RAP	$E[R_p - R_f] \times \frac{\sqrt{V[R_B]}}{\sqrt{V[R_p - R_f]}} = SR_p\sqrt{V[R_B]}$	Modigliani–Modigliani. Excess return rescaled to the benchmark's volatility; same ranking as SR; useful to plot cumulative returns at equal volatility.
Information ratio	$\frac{E[R_p - R_B]}{\sqrt{V[R_p - R_B]}}$	Benchmark is a risky index; denominator is *tracking error*. SR for absolute, IR for relative performance.
Jensen's alpha	$(R_p - R_f) - (R_{\text{model}} - R_f)$	Realised minus model-implied excess return (CAPM alpha, FFC four-factor alpha, ...).
Certainty equivalent	depends on utility	Adjusts also for higher moments, but requires choosing the risk aversion. Conclusions usually similar to SR.

Additional note (not on the slides)

Certainty-equivalent return for a mean–variance investor (as in DeMiguel, Garlappi and Uppal 2009, which comes back in Chapter 4): $\text{CEQ}_p = \hat{\mu}_p - \frac{\gamma}{2}\hat{\sigma}_p^2$, the risk-free return that the investor finds as good as the risky strategy.

Python exercise 1: same Sharpe ratio, different risks

50 years of monthly excess returns. A: Normal. B: small gains most months, rare large losses (like selling insurance). Both rescaled to mean 0.5% and vol 4%.

```
import numpy as np
import pandas as pd
from scipy import stats
rng = np.random.default_rng(21)
T = 600   # 50 years of monthly excess returns
a = rng.normal(0, 1, T)                              # Strategy A: Normal
crash = rng.random(T) < 0.03                         # Strategy B: 3% crash months
b = np.where(crash, -8.0, 0.5) + rng.normal(0, 0.5, T)
x = {"A": a, "B": b}
x = {k: 0.005 + 0.04*(x[k] - x[k].mean())/x[k].std() for k in x}  # same mean & vol

wealth = {k: np.cumprod(1 + x[k]) for k in x}
peak = {k: np.maximum.accumulate(wealth[k]) for k in x}
q = {k: np.quantile(x[k], 0.01) for k in x}
table = pd.DataFrame({
    "Sharpe":  [np.sqrt(12)*x[k].mean()/x[k].std() for k in x],
    "Sortino": [np.sqrt(12)*x[k].mean() / np.sqrt(np.mean(np.minimum(x[k], 0)**2)) for k in x],
    "skew":    [stats.skew(x[k]) for k in x],
    "VaR 1%":  [-q[k] for k in x],
    "CVaR 1%": [-x[k][x[k] <= q[k]].mean() for k in x],
    "max DD":  [(1 - wealth[k]/peak[k]).max() for k in x]}, index=list(x))
print(table.round(3))
```

```
   Sharpe  Sortino    skew  VaR 1%  CVaR 1%  max DD
A   0.433    0.661  -0.162   0.090    0.104   0.593
B   0.433    0.469  -4.584   0.207    0.218   0.452
```

- Identical Sharpe ratios by construction.
- Sortino, skewness, VaR and CVaR all reveal B's **crash risk**: its 1% VaR is more than twice A's.
- Max drawdown happens to be larger for A in this sample: **no single risk measure tells the whole story**.

Additional note (not on the slides)

Code to remember. Maximum drawdown in two lines: `wealth = np.cumprod(1+r)`, `peak = np.maximum.accumulate(wealth)`, then `(1 - wealth/peak).max()`. Note that in this exercise the Sortino target is 0 (`np.minimum(x, 0)`).

Python exercise 2: annualising the Sharpe ratio

100,000 years of monthly AR(1) excess returns with $\rho \in \{-0.2, 0, 0.3\}$.

```
import numpy as np
import pandas as pd
from scipy.signal import lfilter
rng = np.random.default_rng(22)
T = 1_200_000
eps = rng.normal(0, 0.04, T)
rhos = [-0.2, 0.0, 0.3]
```

```
x = {r: 0.005 + lfilter([1], [1, -r], eps) for r in rhos}   # AR(1): x_t = rho x_{t-1} + eps_t
xa = {r: x[r].reshape(-1, 12).sum(axis=1) for r in rhos}     # annual returns
sr_m = {r: x[r].mean()/x[r].std() for r in rhos}

k = np.arange(1, 12)
lo = {r: 12/np.sqrt(12 + 2*np.sum((12-k)*r**k)) for r in rhos}  # Lo (2002) multiplier
table = pd.DataFrame({
    "sqrt(12) x monthly SR": [np.sqrt(12)*sr_m[r] for r in rhos],
    "Lo multiplier x SR":    [lo[r]*sr_m[r] for r in rhos],
    "actual annual SR":      [xa[r].mean()/xa[r].std() for r in rhos]}, index=rhos)
print(table.round(3))
```

```
      sqrt(12) x monthly SR  Lo multiplier x SR  actual annual SR
-0.2                  0.428               0.515             0.517
 0.0                  0.437               0.437             0.439
 0.3                  0.419               0.317             0.317
```

With $\rho = 0.3$, $\sqrt{12}$ overstates the annual SR by about a third (0.419 vs. 0.317); with $\rho < 0$ it understates it. The Lo multiplier is close to the truth in both cases.

Additional note (not on the slides)

`lfilter([1], [1, -r], eps)` computes the recursion $y_t = \varepsilon_t + \rho\, y_{t-1}$ quickly (a loop would be slow for 1.2 million observations). Positive autocorrelation is typical of illiquid assets and hedge funds with smoothed returns, so their annualised Sharpe ratios are often overstated.

2.5 Transaction costs and turnover

Definition: Turnover

When rebalancing at $t+1$, the trade in asset n is $|w_{t+1,n} - w_{t^+,n}|$, where $w_{t+1,n}$ is the **new** weight and $w_{t^+,n}$ is the **old** date-t weight evaluated at date-$(t+1)$ prices (it has drifted with returns).

$$\text{Turnover}_{t+1} = \sum_{n=1}^{N} |w_{t+1,n} - w_{t^+,n}|, \qquad \overline{\text{Turnover}} = \frac{1}{T}\sum_{t=0}^{T-1}\sum_{n=1}^{N} |w_{t+1,n} - w_{t^+,n}| \times 12.$$

▶ **Key result: weights just before rebalancing**

After returns realise, the old weights drift to

$$w_{t^+,n} = \frac{w_{t,n}\,(1 + r_{t+1,n})}{1 + r_{t+1,p}}, \qquad r_{t+1,p} = \sum_n w_{t,n} r_{t+1,n}.$$

Additional note (not on the slides)

Why. Dollar holding in n grows by $(1 + r_{t+1,n})$, total wealth by $(1 + r_{t+1,p})$; the new share of wealth is the ratio. Consequence: even **1/N has positive turnover**, because you must sell winners and buy losers to return to equal weights. A buy-and-hold portfolio has zero turnover.

In plain words

Numerical example. 1/N with two assets: 50/50. Next month asset A gains 10%, asset B is flat. Now A is $0.55/1.05 = 52.4\%$ and B is 47.6%. To go back to 50/50 you sell 2.4% of A and buy 2.4% of B: turnover $= |0.5 - 0.524| + |0.5 - 0.476| = 4.8\%$. With a cost of 50 bp per unit traded you pay $0.005 \times 0.048 = 0.024\%$ of wealth.

Industry turnover ratio. $\dfrac{\min(\text{securities sold},\ \text{securities purchased})}{\text{average net assets}} \times 100$. Example: bought \$10m, sold \$9m, average assets \$100m $\Rightarrow$ 9%. It counts only one side, while the academic turnover counts both.

▶ Key result: returns net of proportional transaction costs

With proportional cost κ (no fixed costs):

$$W_{t+1,p} = W_{t,p}\, R_{t+1,p}\Big(1 - \kappa \sum_{n=1}^{N} |w_{t+1,n} - w_{t^+,n}|\Big),$$

$$r^{\text{net}}_{t+1,p} = R_{t+1,p}\Big(1 - \kappa \sum_{n=1}^{N} |w_{t+1,n} - w_{t^+,n}|\Big) - 1.$$

Typical κ: 50 bp (Balduzzi–Lynch 1999, based on NYSE studies); 10–20 bp for large stocks today; 0–5 bp for a hedge fund. See French (2008).

Python exercise 3: turnover and break-even transaction cost

10 assets, 20 years; true means change every month; we observe a noisy forecast. A = 1/N; B = 1/N + tilt towards assets with the highest forecasts.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(23)
N, T = 10, 240
mu = rng.normal(0.008, 0.01, size=(T, N))          # true means
f = rng.normal(0, 0.04, size=(T, 1))               # market factor
R = mu + f + rng.normal(0, 0.05, size=(T, N))      # returns
signal = mu + rng.normal(0, 0.01, size=(T, N))     # noisy forecast
w = {"A": np.full((T, N), 1/N),
     "B": 1/N + 2*(signal - signal.mean(axis=1, keepdims=True))}
rp = {s: (w[s]*R).sum(axis=1) for s in w}
```

```
w_plus = {s: w[s]*(1 + R)/(1 + rp[s])[:, None] for s in w}   # drifted weights
turn = {s: np.abs(w[s][1:] - w_plus[s][:-1]).sum(axis=1) for s in w}

kappas = [0, 10, 25, 50, 100]                                # in basis points
net = {(s, k): (1 - k/1e4*turn[s])*(1 + rp[s][1:]) - 1 for s in w for k in kappas}
sr = {key: np.sqrt(12)*net[key].mean()/net[key].std() for key in net}
print("Average annual turnover:", {s: round(float(12*turn[s].mean()), 2) for s in w})
table = pd.DataFrame({s: [sr[(s, k)] for k in kappas] for s in w}, index=kappas)
table.index.name = "cost (bp)"
print(table.round(3))
```

```
Average annual turnover: {'A': 0.47, 'B': 3.69}
cost (bp)   SR_A   SR_B
0          0.784  0.958
10         0.781  0.935
25         0.777  0.900
50         0.770  0.842
100        0.755  0.725
```

B trades about 8 times more than 1/N. Before costs B wins (0.96 vs. 0.78); the advantage disappears between 50 and 100 bp. **The value of a strategy depends on its turnover.**

Limitations of this three-step approach (optimise → measure turnover → multiply by $\bar{\kappa}$):

1. Costs should be included *when optimising*, not after;
2. the relevant turnover is that of the cost-aware portfolio (smaller);
3. use the cost of each stock at each date, $\kappa_{t,n}$, not an average $\bar{\kappa}$.

Later in the course: optimise with costs, use $\kappa_{t,n}$, and **net trades across factors** before computing costs (DeMiguel, Martín-Utrera, Nogales, Uppal 2020; DeMiguel, Martín-Utrera, Uppal 2024). Turnover also creates capital-gains taxes (not studied; see Dybvig–Koo 1996, DeMiguel–Uppal 2005).

Turnover by style. Higher for active (vs. passive), growth (vs. value), quantitative (vs. qualitative). **Controlling turnover**: account for costs when rebalancing; long horizon; buy-and-hold; tax-efficient vehicles.

2.6 Review of hypothesis testing

Steps. (1) State the hypothesis; (2) choose a test statistic and its distribution; (3) set the significance level; (4) state the decision rule; (5) collect data; (6) compute the statistic; (7) statistical decision; (8) investment decision.

Test of a mean. $H_0 : \mu = \mu_0$ vs. $H_1 : \mu \neq \mu_0$. If $\bar{X} \sim \mathcal{N}(\mu_0, \sigma^2/T)$ under H_0:

$$z = \frac{\bar{X} - \mu_0}{\sigma/\sqrt{T}} \sim \mathcal{N}(0,1), \qquad P(-1.96 \leq z \leq 1.96) = 0.95.$$

Reject H_0 at 5% (two-sided) if $|z| > 1.96$; one-sided ($H_1 : \mu < \mu_0$) if $z < -1.645$.

Definition: p-value, Type I/II errors, power

- **p-value**: probability of a result at least as extreme as the observed one, assuming H_0 is true; the lowest level at which H_0 can be rejected. Two-sided: $p = 2\,\Phi(-|z|)$.
- **Type I error**: reject H_0 when it is true (probability α, the level of the test).
- **Type II error**: do not reject H_0 when it is false (probability β).
- **Power** $= 1-\beta$: probability of rejecting H_0 when an alternative is true. Fix α (usually 5%) and maximise power. Low power $\Rightarrow$ the test cannot discriminate between H_0 and H_1.

In plain words

A test asks: "if the boring hypothesis were true (e.g. no difference), how surprising would my data be?" The z-statistic measures the surprise in standard errors. Beyond about 2 standard errors (1.96) we call it too surprising and reject. The p-value is the probability of being at least that surprised by pure chance.

Worked example. $H_0 : \mu = 15$, $\bar{X} = 13$, $\sigma = 3.0456$, $T = 30$:

$$z = \frac{13-15}{3.0456/\sqrt{30}} = -3.597, \qquad p = 2 \times P[z < -3.597] = 2 \times 0.000161 = 0.000322.$$

$|z| > 1.96 \Rightarrow$ reject H_0. Reporting the p-value avoids arbitrary conventional levels.

2.7 Test of the difference in Sharpe ratios (focus)

We test $H_0 : SR_n - SR_m = 0$ (portfolio n vs. benchmark m). Same logic as the test of a mean, with $\bar{X} \to \widehat{SR}_n - \widehat{SR}_m$ and $\mu_0 \to 0$:

$$z = \frac{\widehat{SR}_n - \widehat{SR}_m}{\sqrt{V[\widehat{SR}_n - \widehat{SR}_m]}}.$$

All we need is the variance of the difference. Two methods: the delta method and the bootstrap.

2.7.1 The delta method

▶ **Key result: delta method**

If $\sqrt{T}(\hat{\theta} - \theta_0) \xrightarrow{d} \mathcal{N}(0, \Omega)$ and f is differentiable with gradient $\nabla f(\theta_0)$, then

$$\sqrt{T}\{f(\hat{\theta}) - f(\theta_0)\} \xrightarrow{d} \mathcal{N}(0, V), \qquad V = \nabla f(\theta_0)^\top \, \Omega \, \nabla f(\theta_0), \qquad \widehat{se}\{f(\hat{\theta})\} = \sqrt{\hat{V}/T}.$$

Test $H_0 : f(\theta_0) = c$ with $z = \frac{f(\hat{\theta})-c}{\widehat{se}} \approx \mathcal{N}(0,1)$, $p = 2\Phi(-|z|)$.

Intuition

In a small enough range, any differentiable f is approximately *linear* (first-order Taylor expansion), and a linear function of a Normal variable is Normal. As T grows the estimator's variance shrinks, so the linear approximation is applied over a smaller range and becomes more accurate.

In plain words

We know how noisy a sample mean and a sample variance are. The Sharpe ratio is a *function* of the two (mean divided by square root of variance). The delta method converts the noise of the inputs into the noise of the output by linearising the function (a first-order Taylor expansion). Steps 1–3 do exactly that, first for one SR, then for the difference of two.

2.7.2 Step 1: distribution of $\hat{\mu}_n$ and $\hat{\sigma}_n^2$

For IID Normal excess returns, with $\boldsymbol{u} = (\mu_n, \sigma_n^2)^\top$:

$$\sqrt{T}(\hat{\boldsymbol{u}} - \boldsymbol{u}) \to \mathcal{N}(\mathbf{0}_2,\ \Omega)\,, \qquad \Omega = \begin{pmatrix} \sigma_n^2 & 0 \\ 0 & 2\sigma_n^4 \end{pmatrix} \quad \Rightarrow \quad V[\hat{\mu}_n] \overset{a}{=} \frac{\sigma_n^2}{T}, \quad V[\hat{\sigma}_n^2] \overset{a}{=} \frac{2\sigma_n^4}{T}.$$

2.7.3 Step 2: distribution of $\widehat{SR}_n$

$f(\boldsymbol{u}) = \mu_n/\sqrt{\sigma_n^2}$. Gradient and quadratic form:

$$f_u = \begin{pmatrix} \dfrac{1}{\sigma_n} \\ -\dfrac{\mu_n}{2\sigma_n^3} \end{pmatrix}, \qquad f_u^\top \Omega f_u = \frac{\sigma_n^2}{\sigma_n^2} + \frac{\mu_n^2}{4\sigma_n^6}\, 2\sigma_n^4 = 1 + \frac{1}{2} SR_n^2.$$

▶ **Key result: standard error and CI of the Sharpe ratio (IID Normal)**

$$\sqrt{T}(\widehat{SR}_n - SR_n) \to \mathcal{N}\Big(0,\ 1 + \tfrac{1}{2} SR_n^2\Big), \qquad SE[\widehat{SR}_n] = \sqrt{\frac{1}{T}\Big(1 + \frac{1}{2}\widehat{SR}_n^2\Big)},$$

$$z = \frac{\widehat{SR}_n - SR_0}{SE[\widehat{SR}_n]} \approx \mathcal{N}(0,1), \qquad CI_{95\%} = \widehat{SR}_n \pm 1.96\sqrt{\frac{1}{T}\Big(1 + \frac{1}{2}\widehat{SR}_n^2\Big)}.$$

Robust variants replace σ^2/T by a HAC long-run variance.

Additional note (not on the slides)

Numerical example. Monthly $SR = 0.15$ (annual ≈ 0.52), $T = 120$ months: $SE = \sqrt{(1 + 0.5 \cdot 0.0225)/120} = 0.0918$ monthly, i.e. ≈ 0.32 annualised. The 95% CI for the annual SR is about 0.52 ± 0.62: it includes zero. Ten years of data are not enough to prove that a Sharpe ratio of 0.5 is positive. (Note: the SR inside the formula must be at the *same frequency* as T.)

2.7.4 Step 3: distribution of $\widehat{SR}_n - \widehat{SR}_m$

IID Normal excess returns $R_{t,n}, R_{t,m}$ with covariance σ_{nm} and correlation $\rho_{nm} = \sigma_{nm}/(\sigma_n\sigma_m)$. $\boldsymbol{u} = (\mu_n, \mu_m, \sigma_n^2, \sigma_m^2)^\top$:

$$\Omega = \begin{pmatrix} \sigma_n^2 & \sigma_{nm} & 0 & 0 \\ \sigma_{nm} & \sigma_m^2 & 0 & 0 \\ 0 & 0 & 2\sigma_n^4 & 2\sigma_{nm}^2 \\ 0 & 0 & 2\sigma_{nm}^2 & 2\sigma_m^4 \end{pmatrix}, \qquad f(\boldsymbol{u}) = \frac{\mu_n}{\sigma_n} - \frac{\mu_m}{\sigma_m}, \qquad f_u = \begin{pmatrix} 1/\sigma_n \\ -1/\sigma_m \\ -\mu_n/(2\sigma_n^3) \\ \mu_m/(2\sigma_m^3) \end{pmatrix}.$$

Additional note (not on the slides)

Doing the multiplication. Ω is block-diagonal, so the quadratic form splits into a "means" block and a "variances" block:

$$\text{means: } \frac{\sigma_n^2}{\sigma_n^2} + \frac{\sigma_m^2}{\sigma_m^2} - 2\frac{\sigma_{nm}}{\sigma_n\sigma_m} = 2 - 2\rho_{nm},$$

$$\text{variances: } \frac{\mu_n^2}{4\sigma_n^6}2\sigma_n^4 + \frac{\mu_m^2}{4\sigma_m^6}2\sigma_m^4 - 2\frac{\mu_n\mu_m}{4\sigma_n^3\sigma_m^3}2\sigma_{nm}^2 = \tfrac{1}{2}(SR_n^2 + SR_m^2 - 2SR_nSR_m\rho_{nm}^2).$$

▶ **Key result: Jobson–Korkie (1981) test with Memmel (2003) correction**

$$V\big[\widehat{SR}_n - \widehat{SR}_m\big] = \frac{1}{T}\Big[2 - 2\hat{\rho}_{nm} + \frac{1}{2}\big(\widehat{SR}_n^2 + \widehat{SR}_m^2 - 2\widehat{SR}_n\widehat{SR}_m\hat{\rho}_{nm}^2\big)\Big],$$

$$z = \frac{\widehat{SR}_n - \widehat{SR}_m}{\sqrt{V[\widehat{SR}_n - \widehat{SR}_m]}}.$$

If $\rho_{nm} = 0$: $V = \frac{1}{T}\big[(1 + \frac{1}{2}SR_n^2) + (1 + \frac{1}{2}SR_m^2)\big]$, the sum of the two variances.

Intuition

The term $2 - 2\rho_{nm}$ dominates. **The higher the correlation between the two strategies, the smaller the variance of the difference**, so the easier it is to detect a given difference in Sharpe ratios. Comparing two highly correlated strategies (e.g. MV vs. a slight variant) is much easier than comparing two unrelated ones.

Worked example: applying the JKM test

Ten years of monthly data ($T = 120$). Monthly Sharpe ratios $\widehat{SR}_n = 0.15$ (annual 0.52) and $\widehat{SR}_m = 0.10$ (annual 0.35); correlation $\hat{\rho} = 0.8$.

$$V = \frac{1}{120}\Big[2 - 1.6 + \tfrac{1}{2}\big(0.0225 + 0.01 - 2 \times 0.15 \times 0.10 \times 0.64\big)\Big] = \frac{0.4 + 0.0067}{120} = 0.00339,$$

so $SE = \sqrt{0.00339} = 0.0582$. $z = (0.15 - 0.10)/0.0582 = 0.86$, $p = 2\Phi(-0.86) = 0.39$: **not significant**. An annual SR difference of 0.17 over ten years cannot be distinguished from luck.

With $\hat{\rho} = 0$ instead: $SE = 0.130$ and $z = 0.39$, even weaker. High correlation between the two strategies is what makes the test more powerful (the $2 - 2\rho$ term).

Limitations of the delta method (JKM). Not valid with **non-normality** (heavy tails, skewness) or **time dependence** (autocorrelation, volatility clustering). See the QuantPy article recommended in the slides. Alternative: **bootstrap**.

Python exercise 4: when is the JKM test reliable?

5,000 samples of 10 years; two strategies with the *same* true SR (H_0 true), correlation 0.5. Case IID vs. AR(1) with $\rho = 0.3$.

```
import numpy as np
from scipy.signal import lfilter
rng = np.random.default_rng(24)
M, T = 5000, 120
rho_nm = 0.5
z1 = rng.normal(size=(M, T))
z2 = rho_nm*z1 + np.sqrt(1-rho_nm**2)*rng.normal(size=(M, T))
ar = {"IID": 0.0, "AR(1)": 0.3}
n = {c: 0.005 + 0.04*lfilter([1], [1, -ar[c]], z1, axis=1) for c in ar}
m = {c: 0.005 + 0.04*lfilter([1], [1, -ar[c]], z2, axis=1) for c in ar}

srn = {c: n[c].mean(1)/n[c].std(1, ddof=1) for c in ar}
srm = {c: m[c].mean(1)/m[c].std(1, ddof=1) for c in ar}
rho = {c: np.array([np.corrcoef(n[c][i], m[c][i])[0, 1] for i in range(M)]) for c in ar}
var = {c: (2 - 2*rho[c] + 0.5*(srn[c]**2 + srm[c]**2
       - 2*srn[c]*srm[c]*rho[c]**2))/T for c in ar}
zst = {c: (srn[c] - srm[c])/np.sqrt(var[c]) for c in ar}
print("Rejection rate at 5% (should be 0.05):",
      {c: round(float(np.mean(np.abs(zst[c]) > 1.96)), 3) for c in ar})
```

```
Rejection rate at 5% (should be 0.05): {'IID': 0.053, 'AR(1)': 0.15}
```

IID: rejects $\approx 5\%$, as it should. AR(1): rejects 15%, **three times too often** (size distortion). Hence the need for tests robust to serial dependence: the block bootstrap of Ledoit and Wolf (2008).

2.7.5 Bootstrapping

Definition: Bootstrap

Estimate the sampling distribution of an estimator by **resampling the data with replacement**. Useful when parametric assumptions are in doubt, when parametric inference is impossible, or when standard-error formulas are complex.

Intuition. From one sample of French heights you get one mean. Draw a new sample of the same size N from your data with replacement (e.g. from $[1, 2, 3, 4, 5]$ you might get $[2, 5, 4, 4, 1]$), compute its mean, repeat 1,000–10,000 times. The histogram of these bootstrap means estimates the sampling distribution of the mean. Works for almost any statistic. Reference: Efron and Tibshirani (1994).

Worked example: one bootstrap round by hand

Data (5 monthly returns, in %): $[2,\ -1,\ 3,\ 0,\ 1]$, mean 1.0. Draw 5 positions with replacement, e.g. $(3, 1, 5, 5, 2)$: the bootstrap sample is $[3,\ 2,\ 1,\ 1,\ -1]$, mean 1.2. Repeat 5,000 times: the standard deviation of the 5,000 means is the bootstrap standard error; the 2.5% and 97.5% quantiles give a 95% confidence interval. For a Sharpe ratio difference you do the same with *pairs* of returns and compute $\widehat{SR}_n - \widehat{SR}_m$ in each round.

2.7.6 Block bootstraps for time series

The IID bootstrap destroys the time ordering, so it cannot reproduce serial correlation. Solution: resample **blocks** of consecutive observations.

Method	How blocks are formed
Block bootstrap	Non-overlapping blocks, e.g. $(X_1, X_2), (X_3, X_4), \ldots, (X_9, X_{10})$; draw blocks with replacement (e.g. 3,1,5,2,5).
Moving block (MBB)	Overlapping blocks $(X_1, X_2), (X_2, X_3), \ldots, (X_9, X_{10})$.
Stationary	**Random** block lengths (geometric, mean ℓ) and random starting points (Politis–Romano).
Circular block	Data treated as a circle: blocks wrap around from the end to the start, so edge points are sampled as often as middle points (Politis–Romano 1992). **Recommended**; used by Ledoit–Wolf (2008).

In plain words

Financial returns have memory (calm and turbulent periods cluster). If you shuffle single months you destroy that memory and underestimate uncertainty. So you shuffle *blocks* of consecutive months: inside each block the memory survives. And you always move the two strategies' returns together, month by month, to keep their correlation.

Resample **pairs** $(R_{t,n}, R_{t,m})$ together, to preserve the correlation between the two strategies.

2.7.7 Ledoit–Wolf (2008) bootstrap p-value for ΔSR

$H_0 : SR_n = SR_m$ vs. $H_1 : SR_n \neq SR_m$. Studentised statistic $S = \widehat{\Delta SR}/\widehat{SE}(\widehat{\Delta SR})$, with a HAC or bootstrap SE.

► **Key result: practical recipe**

1. Compute $\widehat{SR}_n, \widehat{SR}_m$, $\widehat{\Delta SR}$ and $S_{obs} = \widehat{\Delta SR}/\widehat{SE}$, with $\widehat{SE}$ from the delta method using a HAC (Newey–West) long-run covariance.
2. Choose block length ℓ (rule of thumb $\ell \propto T^{1/3}$; LW calibrate it); circular or stationary bootstrap.
3. For $b = 1, \ldots, B$: draw a bootstrap series of *pairs* by concatenating random blocks, truncated at T; compute $\widehat{\Delta SR}^{*(b)}$ and the **centred** studentised statistic

$$S_b^* = \frac{\widehat{\Delta SR}^{*(b)} - \widehat{\Delta SR}}{\widehat{SE}^{*(b)}}.$$

   Centring is needed because the bootstrap samples come from data in which H_0 does not hold exactly.
4. Two-sided p-value: $\hat{p} = \Big(1 + \sum_{b=1}^{B} \mathbb{1}\{|S_b^*| \geq |S_{obs}|\}\Big)/(B+1)$.

Sample code (optional in the slides; key functions)

```
import numpy as np
from numpy.random import default_rng

def sharpe(x):
    x = np.asarray(x); s = x.std(ddof=1)
    return np.nan if s == 0 else x.mean()/s

# IID bootstrap of the mean and percentile CI
rng = default_rng(42)
data = rng.normal(0.0, 1.0, size=200)
B = 5000
boot_means = np.array([rng.choice(data, size=data.size, replace=True).mean()
                       for _ in range(B)])
se_mean = boot_means.std(ddof=1)
ci_mean = np.quantile(boot_means, [0.025, 0.975])
```

```
def block_bootstrap_pairs(a, b, ell, rng):
    # Circular block bootstrap: blocks of length ell may wrap around the end
    n = len(a)
    num_blocks = int(np.ceil(n/ell))
    starts = rng.integers(0, n, size=num_blocks)
    idx = np.concatenate([np.arange(s, s+ell) % n for s in starts])[:n]
    return a[idx], b[idx]

def _newey_west_variance_of_mean(series, max_lag=None):
    x = np.asarray(series); n = x.size; x = x - x.mean()
    if max_lag is None:
        max_lag = int(np.floor(4*(n/100)**(2/9)))
    gamma0 = np.dot(x, x)/n
    cov_k = lambda k: np.dot(x[:-k], x[k:])/n
    w = lambda k: 1 - k/(max_lag + 1.0)              # Bartlett weights
    s = gamma0 + 2.0*sum(w(k)*cov_k(k) for k in range(1, max_lag+1))
    return s/n

def delta_sr_se_hac(a, b, max_lag=None):
    a = np.asarray(a); b = np.asarray(b)
    muA, muB = a.mean(), b.mean(); sA, sB = a.std(ddof=1), b.std(ddof=1)
    # influence function of SR_a - SR_b
    d = (a - muA)/sA - (b - muB)/sB \
        - (muA/(2*sA**3))*((a - muA)**2 - sA**2) \
        + (muB/(2*sB**3))*((b - muB)**2 - sB**2)
    return np.sqrt(_newey_west_variance_of_mean(d, max_lag))

def delta_sr_pvalue_block(a, b, ell, B=5000, rng=None):
    rng = default_rng() if rng is None else rng
    dObs = sharpe(a) - sharpe(b)
    TObs = dObs/delta_sr_se_hac(a, b)
    TBoot = []
    for _ in range(B):
        rA, rB = block_bootstrap_pairs(a, b, ell, rng)
        seB = delta_sr_se_hac(rA, rB)
        if np.isfinite(seB) and seB != 0:
            TBoot.append((sharpe(rA) - sharpe(rB) - dObs)/seB)   # centred
    return np.mean(np.abs(np.asarray(TBoot)) >= np.abs(TObs))

# ell = max(5, round(n**(1/3)))
```

Additional note (not on the slides)

- **Influence function.** The vector `d` is, observation by observation, the first-order (delta-method) contribution of each date to $\widehat{SR}_a - \widehat{SR}_b$. Its sample mean has variance $\approx V[\widehat{\Delta SR}]$; using Newey–West on `d` makes the SE robust to autocorrelation and heteroskedasticity. With IID Normal data it reproduces the JKM formula.
- **Newey–West.** Long-run variance $= \gamma_0 + 2\sum_{k=1}^{L}(1 - \frac{k}{L+1})\gamma_k$: autocovariances up to lag L with declining (Bartlett) weights, which guarantee a positive variance.
- The slides also show bootstrapping a quantile, the **BCa** (bias-corrected and accelerated) interval, and the stationary bootstrap (restart a new block with probability $p = 1/\ell$ at each step).
- Challenge in the slides: reproduce Tables 1 and 2 of Ledoit–Wolf (2008).

2.7.8 Multiple testing and p-hacking

Python exercise 5: the best of 200 useless strategies

200 strategies, 10 years of monthly returns, all with **true mean zero**. Pick the one with the highest t-statistic.

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(25)
K, T = 200, 120
x = rng.normal(0, 0.04, size=(K, T))                # no skill at all
t = x.mean(axis=1) / (x.std(axis=1, ddof=1)/np.sqrt(T))
best = np.argmax(t)
p_naive = 2*stats.norm.sf(t[best])
p_bonf = min(1, K*p_naive)                          # Bonferroni correction
print("Best strategy: annual SR =", round(np.sqrt(12)*t[best]/np.sqrt(T), 2),
      " t-stat =", round(t[best], 2))
print("Naive p-value:", round(p_naive, 4))
print("Bonferroni p-value:", round(p_bonf, 3))
print("Strategies with |t| > 1.96:", np.sum(np.abs(t) > 1.96))
```

```
Best strategy: annual SR = 0.73   t-stat = 2.31
Naive p-value: 0.0208
Bonferroni p-value: 1
Strategies with |t| > 1.96: 11
```

▶ **Key result: multiple testing**

The best useless strategy has an annual SR of 0.73 and naive $p = 0.02$: it looks "significant". 11 of 200 pass at 5%, about the 10 expected by chance. After **Bonferroni** ($p \times K$), $p = 1$: no evidence of skill. See Harvey, Liu, and Zhu (2015).

Additional note (not on the slides)

Note the identity used in the code: annual SR $= \sqrt{12}\, t/\sqrt{T}$, since $t = \widehat{SR}_{\text{monthly}}\sqrt{T}$. Harvey, Liu and Zhu (2015) argue that, given the hundreds of factors already tried, a new factor should clear $t > 3$, not $t > 1.96$. Bonferroni is conservative (it controls the probability of *even one* false discovery); less strict alternatives such as Holm or Benjamini–Hochberg exist.

2.8 Takeaways of Chapter 2

1. The course studies **myopic** portfolios evaluated **out of sample**: weights at t use only information up to t; performance is $R_{t+1,p} = \boldsymbol{w}_t^\top \boldsymbol{R}_{t+1}$.
2. Rolling windows when the past is less relevant; expanding when it is similar; EWMA in between.
3. Many metrics; the **Sharpe ratio** is the main one. Annualise with $\sqrt{12}$ only if returns are not autocorrelated (else Lo 2002).

4. Same Sharpe ratio does not mean same risk: look at Sortino, skewness, VaR, CVaR, drawdowns.
5. Turnover uses *drifted* weights w_{t^+}; net return $= R_{t+1,p}(1 - \kappa \,\text{Turnover}) - 1$. High-turnover strategies can lose their edge after costs.
6. $SE(\widehat{SR}) = \sqrt{(1 + SR^2/2)/T}$: Sharpe ratios are estimated imprecisely.
7. JKM test for $SR_n - SR_m$: the variance shrinks with the correlation between the strategies; it over-rejects with autocorrelation $\Rightarrow$ use the Ledoit–Wolf circular block bootstrap.
8. Trying many strategies inflates false discoveries: correct for multiple testing.

Common mistakes to avoid

1. Computing the Sharpe ratio on total returns instead of *excess* returns.
2. Using weights estimated with data that include the evaluation month (look-ahead).
3. Using $\sqrt{12}$ to annualise when returns are clearly autocorrelated.
4. Computing turnover with $|w_{t+1} - w_t|$ instead of $|w_{t+1} - w_{t^+}|$: this ignores the drift of weights and gives 1/N zero turnover (it is not zero).
5. Concluding that strategy A "beats" B because its SR is higher, without a test: with 10 years the SE of an annual SR is about 0.3.
6. In the bootstrap, resampling the two strategies independently (this destroys their correlation) or forgetting to centre the statistic.
7. Reporting the best of many backtests without a multiple-testing correction.

2.9 Self-test questions (quiz/exam practice)

1. Why does the course not study dynamic portfolios? *(Too many parameters to estimate $\Rightarrow$ poor out-of-sample performance.)*
2. A monthly Sharpe ratio is 0.2 with IID returns. Annual SR? *($0.2\sqrt{12} = 0.69$.)* If $\rho = 0.3$, is the true annual SR higher or lower? *(Lower.)*
3. Which measure uses a risky benchmark and tracking error? *(Information ratio.)*
4. Which measure uses only systematic risk in the denominator? *(Treynor.)*
5. Why does 1/N have positive turnover? *(Weights drift with returns; rebalancing back to $1/N$ requires trading.)*
6. Strategy gross return 1%, turnover 0.5, $\kappa = 50$ bp. Net return? *($1.01 \times (1 - 0.005 \times 0.5) - 1 = 0.7475\%$.)*
7. $T = 240$ months, monthly $SR = 0.1$. SE of $\widehat{SR}$? *($\sqrt{1.005/240} = 0.0647$.)*
8. Two strategies with $\rho_{nm} = 0.9$ vs. $\rho_{nm} = 0$: in which case is a given ΔSR easier to detect? *($\rho = 0.9$: variance of the difference much smaller.)*
9. Why centre the bootstrap statistic at $\widehat{\Delta SR}$? *(The bootstrap data do not satisfy H_0 exactly.)*
10. You test 50 strategies and the best has $p = 0.01$. Bonferroni p? *(0.5.)*

11. Write by hand: Sharpe ratio, Sortino ratio, max drawdown, 5% VaR and CVaR of a return array `r`.

```
sr = np.sqrt(12) * r.mean() / r.std(ddof=1)
sortino = np.sqrt(12) * r.mean() / np.sqrt(np.mean(np.minimum(r, 0)**2))
wealth = np.cumprod(1 + r); mdd = (1 - wealth / np.maximum.accumulate(wealth)).max()
q = np.quantile(r, 0.05); var5 = -q; cvar5 = -r[r <= q].mean()
```

12. Write by hand the JKM z-statistic for two return arrays `a`, `b`.

```
T = len(a)
sa, sb = a.mean()/a.std(ddof=1), b.mean()/b.std(ddof=1)
rho = np.corrcoef(a, b)[0, 1]
var = (2 - 2*rho + 0.5*(sa**2 + sb**2 - 2*sa*sb*rho**2)) / T
z = (sa - sb) / np.sqrt(var)
p = 2 * stats.norm.sf(abs(z))
```

13. Write by hand the turnover of a strategy with weight matrix `W` ($T \times N$) and returns `R`.

```
rp = (W * R).sum(axis=1)
W_plus = W * (1 + R) / (1 + rp)[:, None]
turnover = np.abs(W[1:] - W_plus[:-1]).sum(axis=1)
```

2.10 To do for next class

- Read "Working with Data" in Moreira (2021), *Quantitative Investing*, Chapter 5.
- Work on Assignment #1 (Ch. 1–2; due 11 Nov, 22:00 CET).

References (Chapter 2)

- Balduzzi, P., and A. W. Lynch. 1999. Transaction costs and predictability. *JFE* 52(1): 47–78.
- Campbell, J. Y., and L. M. Viceira. 2002. *Strategic Asset Allocation.* Oxford UP.
- Carhart, M. M. 1997. On persistence in mutual fund performance. *JF* 52(1): 57–82.
- Dello-Preite, M., R. Uppal, P. Zaffaroni, and I. Zviadadze. 2025. Cross-sectional asset pricing with unsystematic risk. SSRN 4135146.
- DeMiguel, V., A. Martín-Utrera, F. J. Nogales, and R. Uppal. 2020. A transaction-cost perspective on the multitude of firm characteristics. *RFS* 33(5): 2180–2222.
- DeMiguel, V., A. Martín-Utrera, and R. Uppal. 2024. A multifactor perspective on volatility-managed portfolios. *JF* 79(6): 3859–3891.
- Efron, B., and R. J. Tibshirani. 1994. *An Introduction to the Bootstrap.* CRC.
- Fama, E. F., and K. R. French. 1993; 2015; 2018. *JFE* 33: 3–56; 116: 1–22; 128: 234–252.
- French, K. R. 2008. The cost of active investing. *JF* 63(4): 1537–1573.
- Harvey, C. R., Y. Liu, and H. Zhu. 2015. . . . and the cross-section of expected returns. *RFS* 29(1): 5–68.
- Jobson, J. D., and B. M. Korkie. 1981. Performance hypothesis testing with the Sharpe and Treynor measures. *JF*: 889–908.
- Ledoit, O., and M. Wolf. 2008. Robust performance hypothesis testing with the Sharpe ratio. *JEF* 15: 850–859.
- Lo, A. W. 2002. The statistics of Sharpe ratios. *FAJ* 58(4): 36–52.
- Memmel, C. 2003. Performance hypothesis testing with the Sharpe ratio. SSRN 412588.

- Politis, D. N., and J. P. Romano. 1992. A circular block-resampling procedure for stationary data. Wiley.
- Sharpe, W. 1964. Capital asset prices. *JF* 19(3): 425–442.

Chapter 3

Mean–Variance Portfolios that Ignore Estimation Error

In plain words

This is the "textbook" theory of portfolio choice (Markowitz, Tobin, Sharpe). We *pretend* that we know the true expected returns $\boldsymbol{\mu}$ and the true covariance matrix V. Under this assumption we find the best possible portfolios. Chapter 4 then asks what happens when $\boldsymbol{\mu}$ and V must be estimated from data (answer: things go badly wrong).

3.1 What we want to do in Chapter 3

- Derive the **efficient frontier** *without* a risk-free asset (Markowitz).
- Derive it *with* a risk-free asset (Tobin): the **tangency portfolio**.
- Understand **diversification**, the role of covariances, and where the **beta** relation (CAPM-like) comes from.
- Extensions: optimisation relative to a **benchmark**, extra **constraints**, **transaction costs**.

3.2 Universe selection

Before choosing assets we must choose the **universe** we pick them from. It depends on:

- **Region**: a French investor may use France, the EU or developed markets; regulation may restrict an Indian investor to Indian assets; some focus on emerging markets.
- **Breadth**: large caps only, riskier firms only, or everything. Norway's sovereign fund (NBIM) holds over 10,000 companies, about 1.5% of all listed shares in the world, across equities, fixed income, real estate and infrastructure.
- **Asset class**: equities, fixed income, derivatives (futures, options), commodities (gold, oil, corn), real estate (REITs), crypto.
- **Risk preferences** (treasuries vs. high credit risk), **liquidity** (insurers can hold illiquid assets), views on **inflation and growth**.

In this course the universe is equities (mostly US evidence, but the results apply to other developed markets too).

3.3 Portfolio choice in pictures: the Nobel-prize ideas

All pictures are in the mean (μ) vs. standard deviation (σ) plane. Investors want to go *up* (more return) and *left* (less risk).

Who	Idea
Markowitz (1990)	Do not evaluate assets in isolation: look at their *correlations*. Combining assets gives a **frontier** that lies to the left of individual assets. More assets $\Rightarrow$ a better (further left) frontier.
Tobin (1981)	Add a risk-free asset: every efficient portfolio is a mix of (1) the risk-free asset and (2) one portfolio of risky assets, the **tangency portfolio** T. This is **two-fund separation**.
Sharpe (1990)	In equilibrium the tangency portfolio is the **market portfolio** ($T = M$) $\Rightarrow$ CAPM: *only systematic risk is priced*.
Samuelson (1970)	Multi-period: if R_f and the slope of the tangency line (the Sharpe ratio) do not change, the optimal portfolio does not change.
Merton (1997)	If R_f and the Sharpe ratio change over time, the optimal portfolio = Markowitz portfolio + **intertemporal hedging** demands against these changes.

In plain words

The picture to keep in mind (draw it yourself: it is the most important graph of the course). Horizontal axis: volatility σ; vertical axis: expected return μ.

- Single assets are scattered dots.
- The **frontier** is a curve shaped like a "$\subset$" lying on its side (a hyperbola), to the *left* of the dots: combining assets gives less risk for the same return. Its leftmost point is the **GMV** portfolio; the upper half is the **efficient frontier**.
- From the risk-free rate R_f on the vertical axis, draw the steepest straight line that still touches the frontier: it touches at the **tangency portfolio** T. This line is the **capital market line**; its slope is the maximum Sharpe ratio.
- Every investor holds a point on this line: closer to R_f if risk-averse, beyond T (borrowing) if risk-tolerant.

3.4 Portfolio diversification

3.4.1 Systematic vs. unsystematic risk

- **Systematic** risk: common to many assets $\Rightarrow$ cannot be diversified away.
- **Unsystematic** (idiosyncratic) risk: specific to one asset $\Rightarrow$ diversifiable.
- Assume (for now; revisited in Chapter 8) that unsystematic risk earns no reward $\Rightarrow$ hold diversified portfolios $\Rightarrow$ only systematic risk matters and only it is compensated.

3.4.2 Mean and variance of a portfolio

With dollar amounts $V_1, \dots, V_N$ and total $V = \sum V_n > 0$, the weights are $w_n = V_n/V$ with $\sum_n w_n = \boldsymbol{w}^\top \mathbf{1}_N = 1$ (a portfolio with $V = 0$ is an *arbitrage portfolio*).

► **Key result: portfolio moments**

$$R_p = \boldsymbol{w}^\top \boldsymbol{R}, \qquad \mu_p = \boldsymbol{w}^\top \boldsymbol{\mu} = \sum_n w_n \mathbb{E}[R_n],$$

$$\sigma_p^2 = \boldsymbol{w}^\top V \boldsymbol{w} = \sum_{n=1}^{N} \sum_{m=1}^{N} w_n w_m \sigma_{nm}, \qquad \sigma_p = \sqrt{\sigma_p^2}.$$

Two assets: $\sigma_p^2 = w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1w_2\sigma_{12}$.

In plain words

The "table trick". Write a table with $w_1R_1, \dots, w_NR_N$ on the rows and on the columns. Cell (n, m) is $w_n w_m \sigma_{nm}$. The portfolio variance is the **sum of all the cells**. The diagonal cells are variances (N of them); the off-diagonal cells are covariances ($N^2 - N$ of them). So the variance of a sum is *not* the sum of the variances: you need the covariances.

---

	w_1R_1	w_2R_2	$\cdots$	w_NR_N
w_1R_1	$w_1^2\sigma_1^2$	$w_1w_2\sigma_{12}$	$\cdots$	$w_1w_N\sigma_{1N}$
w_2R_2	$w_2w_1\sigma_{21}$	$w_2^2\sigma_2^2$	$\cdots$	$w_2w_N\sigma_{2N}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
w_NR_N	$w_Nw_1\sigma_{N1}$	$w_Nw_2\sigma_{N2}$	$\cdots$	$w_N^2\sigma_N^2$

Worked example: the minimum-variance mix of two assets

With two assets and weight w in asset 1, $\sigma_p^2(w) = w^2\sigma_1^2 + (1-w)^2\sigma_2^2 + 2w(1-w)\sigma_{12}$. Setting the derivative to zero gives

$$w^* = \frac{\sigma_2^2 - \sigma_{12}}{\sigma_1^2 + \sigma_2^2 - 2\sigma_{12}}.$$

Example: $\sigma_1 = 20\%$, $\sigma_2 = 30\%$, $\rho = 0.2$, so $\sigma_{12} = 0.012$: $w^* = (0.09 - 0.012)/(0.04 + 0.09 - 0.024) = 0.736$, and $\sigma_p = 18.1\%$, *lower than the safer asset alone* (20%). Diversification creates a portfolio less risky than any of its components, as long as ρ is not too high. (If $\rho = 1$ there is no benefit; if $\rho = -1$ risk can be eliminated completely.)

3.4.3 Diversification with N assets

Equal weights $w_n = 1/N$: each of the N variance cells is $\frac{1}{N^2}\sigma_n^2$; each of the $N^2 - N$ covariance cells is $\frac{1}{N^2}\sigma_{nm}$. Adding them up:

▶ **Key result: the diversification formula**

$$\sigma_p^2 = \frac{1}{N}\underbrace{\Big(\frac{1}{N}\sum_n \sigma_n^2\Big)}_{\text{average variance}} + \Big(1-\frac{1}{N}\Big)\underbrace{\Big(\frac{1}{N^2-N}\sum_n\sum_{m\neq n}\sigma_{nm}\Big)}_{\text{average covariance}} \xrightarrow{N\to\infty} \text{average covariance.}$$

For a well-diversified portfolio, **covariances determine risk; individual variances barely matter**. When adding an asset, look at its covariance (correlation) with the portfolio, not at its variance.

In plain words

Each stock's own "noise" gets weight $1/N^2$ and there are only N of them, so in total they count $1/N$ and vanish. But the "common movement" (all stocks go down when the market goes down) is in all $N^2 - N$ pairs and never disappears. That floor is market risk.

Python exercise 1: diversification in practice

500 stocks, 10 years monthly; each return = market × beta (0.5–1.5) + firm-specific noise.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(31)
N_all, T = 500, 120
beta = rng.uniform(0.5, 1.5, N_all)
f = rng.normal(0.006, 0.045, T)                  # market
e = rng.normal(0, 0.09, size=(T, N_all))         # firm-specific
R = 0.002 + np.outer(f, beta) + e                # stock returns
V = np.cov(R, rowvar=False)
avg_var = np.mean(np.diag(V))
avg_cov = (V.sum() - np.trace(V)) / (N_all*(N_all - 1))   # off-diagonal average

Ns = [1, 2, 5, 10, 25, 50, 100, 500]
vol = {N: np.mean([R[:, rng.choice(N_all, N, replace=False)].mean(axis=1).std(ddof=1)
                   for _ in range(200)]) for N in Ns}
formula = {N: np.sqrt(avg_var/N + (1 - 1/N)*avg_cov) for N in Ns}
table = pd.DataFrame({"simulated vol": [np.sqrt(12)*vol[N] for N in Ns],
                      "formula vol": [np.sqrt(12)*formula[N] for N in Ns]}, index=Ns)
print(table.round(3))
print("Limit (sqrt of average covariance):", round(float(np.sqrt(12*avg_cov)), 3))
```

```
N    simulated vol  formula vol
1            0.358        0.359
2            0.279        0.281
5            0.221        0.221
10           0.197        0.197
25           0.181        0.181
50           0.175        0.176
100          0.173        0.173
500          0.170        0.170
Limit (sqrt of average covariance): 0.17
```

Volatility falls from 36% (one stock) to 20% (10 stocks) but only to 17% (500 stocks). **The 17% floor is $\sqrt{\text{average covariance}}$**: diversification removes firm-specific risk, not market risk. Most of the benefit comes from the first 10–20 stocks.

3.5 Mean–variance frontier without a risk-free asset (focus)

Definition: Frontier portfolio

Given a target expected return, the portfolio with the smallest risk is a **frontier portfolio**. The set of all of them in the (σ, μ) plane is the **portfolio frontier**; its upper half is the **efficient frontier**.

Notation. $\mathbf{1}_N$ vector of ones; I_N identity; $\boldsymbol{w}$ weights; $\boldsymbol{\mu} = (\mathbb{E}[R_1], \ldots, \mathbb{E}[R_N])^\top$; V the symmetric $N \times N$ covariance matrix (often written Σ in papers).

3.5.1 The optimisation problem

$$\min_{\boldsymbol{w}} \ \boldsymbol{w}^\top V \boldsymbol{w} \qquad \text{s.t.} \quad (1)\ \boldsymbol{w}^\top \mathbf{1}_N = 1, \qquad (2)\ \boldsymbol{w}^\top \boldsymbol{\mu} = \mu_{targ}.$$

In plain words

Find the weights with the lowest variance such that (1) they add up to 100% and (2) the portfolio earns the target return, e.g. 15%. It is a quadratic objective with linear constraints, so it has a closed-form solution (when there are no other constraints).

3.5.2 Solving it step by step (Lagrangian)

Step 1: Lagrangian. With multipliers λ_w, λ_R:

$$L = \boldsymbol{w}^\top V \boldsymbol{w} + \lambda_w(1 - \boldsymbol{w}^\top \mathbf{1}_N) + \lambda_R(\mu_{targ} - \boldsymbol{w}^\top \boldsymbol{\mu}).$$

Step 2: FOC in $\boldsymbol{w}$.

$$\frac{\partial L}{\partial \boldsymbol{w}} = 2V\boldsymbol{w} - \lambda_w \mathbf{1}_N - \lambda_R \boldsymbol{\mu} = \mathbf{0} \ \Rightarrow \ \boldsymbol{w} = V^{-1}\Big(\frac{\lambda_w}{2}\mathbf{1}_N + \frac{\lambda_R}{2}\boldsymbol{\mu}\Big). \tag{1}$$

Step 3: FOCs in the multipliers give back the constraints: $1 = \boldsymbol{w}^\top \mathbf{1}_N$ (2) and $\mu_{targ} = \boldsymbol{w}^\top \boldsymbol{\mu}$ (3).

Step 4: substitute (1) into (2) and (3). Define the three scalars

▶ **Key result: the three "a" numbers**

$$a_1 = \boldsymbol{\mu}^\top V^{-1}\boldsymbol{\mu}, \qquad a_2 = \boldsymbol{\mu}^\top V^{-1}\mathbf{1}_N, \qquad a_3 = \mathbf{1}_N^\top V^{-1}\mathbf{1}_N, \qquad D = a_1 a_3 - a_2^2.$$

Then

$$\begin{pmatrix} 1 \\ \mu_{targ} \end{pmatrix} = \begin{pmatrix} a_3 & a_2 \\ a_2 & a_1 \end{pmatrix} \begin{pmatrix} \lambda_w/2 \\ \lambda_R/2 \end{pmatrix} \;\Rightarrow\; \frac{\lambda_w}{2} = \frac{a_1 - a_2\mu_{targ}}{D}, \qquad \frac{\lambda_R}{2} = \frac{-a_2 + a_3\mu_{targ}}{D}.$$

Step 5: plug back into (1).

In plain words

What do a_1, a_2, a_3 mean? They are just numbers that summarise the assets, but each has an interpretation:

- $a_3 = \mathbf{1}_N^\top V^{-1}\mathbf{1}_N = 1/\sigma^2_{GMV}$: the higher a_3, the lower the risk of the safest portfolio.
- $a_2/a_3 = \mu_{GMV}$: the expected return of the safest portfolio.
- $D/a_3 = a_1 - a_2^2/a_3$ measures how much the expected returns differ across assets (after accounting for risk): it controls how steep the frontier is. If all assets had the same mean, $D = 0$ and the frontier would be a single point in the μ direction.

The frontier variance is a quadratic function of the target return: $\sigma_p^2 = \frac{1}{a_3} + \frac{a_3}{D}\left(\mu_{targ} - \mu_{GMV}\right)^2$. Each extra unit of target return above the GMV costs more and more variance.

▶ **Key result: frontier weights for target return μ_{targ}**

$$\boldsymbol{w} = \frac{1}{D}(a_1 V^{-1}\mathbf{1}_N - a_2 V^{-1}\boldsymbol{\mu}) + \frac{1}{D}(a_3 V^{-1}\boldsymbol{\mu} - a_2 V^{-1}\mathbf{1}_N)\,\mu_{targ}.$$

Closed form only without extra constraints. With $w_n \geq 0$ or $w_n \leq w_{\max}$ we must solve numerically (Chapter 4 appendix).

Additional note (not on the slides)

How to remember the 2×2 inverse. $\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. Here $\begin{pmatrix} a_3 & a_2 \\ a_2 & a_1 \end{pmatrix}^{-1} = \frac{1}{D}\begin{pmatrix} a_1 & -a_2 \\ -a_2 & a_3 \end{pmatrix}$, which gives the two multipliers directly. $D > 0$ as long as the assets do not all have the same mean.

3.5.3 Two-fund separation and the GMV portfolio

Define $\boldsymbol{w}_0$ (frontier portfolio with $\mu_{targ} = 0$) and $\boldsymbol{w}_1$ ($\mu_{targ} = 1$):

$$\boldsymbol{w}_0 = \frac{1}{D}(a_1 V^{-1}\mathbf{1}_N - a_2 V^{-1}\boldsymbol{\mu}), \qquad \boldsymbol{w}_1 = \boldsymbol{w}_0 + \frac{1}{D}(a_3 V^{-1}\boldsymbol{\mu} - a_2 V^{-1}\mathbf{1}_N).$$

▶ **Key result: two-fund separation (no risk-free asset)**

Every frontier portfolio is a combination of $\boldsymbol{w}_0$ and $\boldsymbol{w}_1$:

$$\boldsymbol{w}_p = \boldsymbol{w}_0 + (\boldsymbol{w}_1 - \boldsymbol{w}_0)\mu_{targ} = (1 - \mu_{targ})\boldsymbol{w}_0 + \mu_{targ}\boldsymbol{w}_1.$$

Any two frontier portfolios generate the entire frontier.

Its variance is $\sigma_p^2 = (1-\mu_{targ})^2 \boldsymbol{w}_0^\top V \boldsymbol{w}_0 + 2\mu_{targ}(1-\mu_{targ})\boldsymbol{w}_0^\top V \boldsymbol{w}_1 + \mu_{targ}^2 \boldsymbol{w}_1^\top V \boldsymbol{w}_1$; minimising it over μ_{targ} gives the **global minimum-variance (GMV)** portfolio.

Additional note (not on the slides)

Closed forms you can derive from the "a" numbers (Merton 1972):

$$\sigma_p^2(\mu_{targ}) = \frac{a_3\mu_{targ}^2 - 2a_2\mu_{targ} + a_1}{D} \quad \text{(a parabola in } (\mu, \sigma^2)\text{, a hyperbola in } (\mu, \sigma)\text{)},$$

$$\boldsymbol{w}_{GMV} = \frac{V^{-1}\mathbf{1}_N}{a_3}, \qquad \mu_{GMV} = \frac{a_2}{a_3}, \qquad \sigma_{GMV}^2 = \frac{1}{a_3}.$$

(Set $d\sigma_p^2/d\mu_{targ} = 0$: $\mu_{targ} = a_2/a_3$; plugging in gives $\sigma^2 = 1/a_3$.)

3.5.4 Worked example with $N = 3$

$$\boldsymbol{\mu} = \begin{pmatrix} 0.20 \\ 0.30 \\ 0.40 \end{pmatrix}, \quad V = \begin{pmatrix} 0.0625 & 0.0700 & 0.1050 \\ 0.0700 & 0.1225 & 0.0840 \\ 0.1050 & 0.0840 & 0.3600 \end{pmatrix}, \quad V^{-1} = \begin{pmatrix} 85.06 & -37.61 & -16.03 \\ -37.61 & 26.35 & 4.82 \\ -16.03 & 4.82 & 6.33 \end{pmatrix}.$$

$a_1 = 0.865$, $a_2 = 2.40$, $a_3 = 20.09$, $D = 11.63$, and

$$\boldsymbol{w}_0 = \begin{pmatrix} 2.48 \\ -0.96 \\ -0.52 \end{pmatrix}, \quad \boldsymbol{w}_1 = \begin{pmatrix} -5.18 \\ 4.37 \\ 1.82 \end{pmatrix}, \quad \boldsymbol{w}_1 - \boldsymbol{w}_0 = \begin{pmatrix} -7.66 \\ 5.32 \\ 2.34 \end{pmatrix}, \qquad \boldsymbol{w} = \boldsymbol{w}_0 + \mu_{targ}(\boldsymbol{w}_1 - \boldsymbol{w}_0).$$

μ_{targ}	weights (w_1, w_2, w_3)	σ_p	meaning
35%	$(-0.20,\ 0.90,\ 0.30)$	0.376	short \$0.20 of asset 1, buy \$0.90 of 2 and \$0.30 of 3
30%	$(0.18,\ 0.64,\ 0.18)$	0.3258	lower risk than asset 2 alone ($\sigma = 0.35$) for the same 30%
GMV ($\approx$11.9%)	$(1.56,\ -0.32,\ -0.24)$	0.223	lowest possible risk

Conclusion: asset 2 alone gives 30% with $\sigma = 0.35$; diversifying gives 30% with $\sigma = 0.3258$. To trace the whole frontier, loop over many μ_{targ}.

```
import numpy as np
ones = np.ones((3, 1))
mu = np.array([[0.20], [0.30], [0.40]])
V = np.array([[0.0625, 0.0700, 0.1050],
              [0.0700, 0.1225, 0.0840],
              [0.1050, 0.0840, 0.3600]])
Vinverse = np.linalg.inv(V)
a1 = mu.T @ Vinverse @ mu          # ".T" transpose, "@" matrix product
a2 = mu.T @ Vinverse @ ones
a3 = ones.T @ Vinverse @ ones
D = a1*a3 - a2**2                  # 0.8650, 2.3980, 20.0930, 11.6306
w0 = (a1*Vinverse@ones - a2*Vinverse@mu)/D
w1 = w0 + (a3*Vinverse@mu - a2*Vinverse@ones)/D

def solution(mu_targ):
    weights = w0 + mu_targ*(w1 - w0)
    vol = np.sqrt(weights.T @ V @ weights)[0, 0]
    return weights, vol
print(solution(0.35))   # [-0.2034, 0.9067, 0.2966], vol 0.3764
print(solution(0.30))   # [ 0.1797, 0.6407, 0.1797], vol 0.3258
```

Additional note (not on the slides)

Column vectors `np.ones((3,1))` produce 1×1 *matrices* for a_1, a_2, a_3 (hence `[0,0]` to extract the number). With 1-D arrays (`np.ones(3)`) they are plain numbers; both are fine, but be consistent.

3.6 Portfolio frontier with a risk-free asset

Two views of the same portfolio. \$40 risk-free, \$30 IBM, \$30 Merck. View 1: 40% R_f, 30% IBM, 30% Merck. View 2: 40% in the risk-free sub-portfolio, 60% in a risky sub-portfolio that is 50/50 IBM/Merck.

If a fraction a goes to a risky portfolio q and $1-a$ to R_f: $\mu_p = (1-a)R_f + a\mathbb{E}[R_q]$, $\sigma_p = |a|\sigma_q$ $\Rightarrow$ a **straight line** from R_f through q. The steepest line touches the frontier at the **tangency portfolio**.

3.6.1 The problem

With $w_f = 1 - \boldsymbol{w}^\top \mathbf{1}_N$ in the risk-free asset:

$$\min_{\boldsymbol{w}} \ \boldsymbol{w}^\top V \boldsymbol{w} \qquad \text{s.t.} \qquad \boldsymbol{w}^\top \boldsymbol{\mu} + (1 - \boldsymbol{w}^\top \mathbf{1}_N) R_f = \mu_{targ} \iff \boldsymbol{w}^\top (\boldsymbol{\mu} - R_f \mathbf{1}_N) = \mu_{targ} - R_f.$$

▶ **Key result: frontier weights with a risk-free asset**

$$\boldsymbol{w} = \underbrace{\left[\frac{\mu_{targ} - R_f}{(\boldsymbol{\mu} - R_f \mathbf{1}_N)^\top V^{-1} (\boldsymbol{\mu} - R_f \mathbf{1}_N)}\right]}_{\text{scaling: how much risk}} \underbrace{V^{-1}(\boldsymbol{\mu} - R_f \mathbf{1}_N)}_{\text{direction: which risky portfolio}} . \tag{8}$$

The higher μ_{targ}, the larger the position in the risky portfolio and the smaller in R_f. The scalar plays the role of risk tolerance $1/\gamma$.

Additional note (not on the slides)

Derivation (one constraint only). Let $\boldsymbol{e} = \boldsymbol{\mu} - R_f \mathbf{1}_N$ and $H = \boldsymbol{e}^\top V^{-1}\boldsymbol{e}$. Lagrangian $L = \boldsymbol{w}^\top V \boldsymbol{w} + \lambda(\mu_{targ} - R_f - \boldsymbol{w}^\top \boldsymbol{e})$. FOC: $2V\boldsymbol{w} = \lambda \boldsymbol{e} \Rightarrow \boldsymbol{w} = \frac{\lambda}{2} V^{-1}\boldsymbol{e}$. Plug into the constraint: $\frac{\lambda}{2} H = \mu_{targ} - R_f \Rightarrow \frac{\lambda}{2} = \frac{\mu_{targ} - R_f}{H}$. **Bonus:** $\sigma_p = \frac{\mu_{targ} - R_f}{\sqrt{H}}$, so $\frac{\mu_{targ} - R_f}{\sigma_p} = \sqrt{H}$: every portfolio on the line has the same Sharpe ratio $\sqrt{H} = \sqrt{(\boldsymbol{\mu} - R_f \mathbf{1}_N)^\top V^{-1} (\boldsymbol{\mu} - R_f \mathbf{1}_N)}$, the **maximum Sharpe ratio**.

3.6.2 Worked example: three risky assets + $R_f = 6.5\%$, $\mu_{targ} = 30\%$

$\boldsymbol{\mu} - R_f \mathbf{1}_N = (0.135, 0.235, 0.335)^\top$, $H = 0.6382$, $V^{-1}(\boldsymbol{\mu} - R_f \mathbf{1}_N) = (-2.7268,\ 2.7299,\ 1.0889)^\top$:

$$\boldsymbol{w} = \frac{0.30 - 0.065}{0.6382} \begin{pmatrix} -2.7268 \\ 2.7299 \\ 1.0889 \end{pmatrix} = \begin{pmatrix} -1.0041 \\ 1.0052 \\ 0.4010 \end{pmatrix},$$

$$w_f = 1 - (-1.0041 + 1.0052 + 0.4010) = 0.5979, \qquad \sigma_p = 0.2942.$$

Case for $\mu_p = 30\%$	Asset 2 alone	3 risky assets	3 risky + R_f
σ_p	0.3500	0.3256	0.2942

More assets ⇒ same return with less risk. Check: $\sigma_p = (0.30 - 0.065)/\sqrt{0.6382} = 0.235/0.7989 = 0.2942$ ✓.

▶ **Key result: capital market line**

All efficient portfolios with a risk-free asset lie on the line

$$\mu_p = R_f + SR_T\, \sigma_p, \qquad SR_T = \frac{\mathbb{E}[R_T] - R_f}{\sigma_T} = \sqrt{(\boldsymbol{\mu} - R_f \mathbf{1}_N)^\top V^{-1} (\boldsymbol{\mu} - R_f \mathbf{1}_N)}.$$

In the example: $\mu_p = 0.065 + 0.799\, \sigma_p$. With $\sigma_p = 0.2942$ we get $\mu_p = 0.30$ ✓. An investor who wants 40% needs $\sigma_p = (0.40 - 0.065)/0.799 = 0.419$ and will borrow at R_f to lever the tangency portfolio.

Tangency portfolio. Rescale the risky weights to sum to one:

$$\boldsymbol{w}_{tang} = \frac{\boldsymbol{w}}{\boldsymbol{w}^\top \mathbf{1}_N} = \frac{1}{0.4021} \begin{pmatrix} -1.0041 \\ 1.0052 \\ 0.4010 \end{pmatrix} = \begin{pmatrix} -2.4971 \\ 2.4998 \\ 0.9972 \end{pmatrix}.$$

It depends on R_f but **not** on μ_{targ}. Its Sharpe ratio is $\sqrt{H} = 0.799$.

```
Rf = 0.065
x = Vinverse @ (mu - Rf*ones)                # direction V^{-1}(mu - Rf 1)
def solutionWithRf(mu_targ, Rf):
    H = ((mu - Rf*ones).T @ Vinverse @ (mu - Rf*ones))[0, 0]
    w = (mu_targ - Rf)/H * x
    return w, np.sqrt(w.T @ V @ w)[0, 0]
w, vol = solutionWithRf(0.30, 0.065)         # [-1.0041, 1.0052, 0.4010], 0.2942
w_f = 1 - w.sum()                            # 0.5979
weightsTangency = x / (ones.T @ x)           # [-2.4972, 2.4999, 0.9972]
```

Python exercise 2: the estimated tangency portfolio

10 assets with known $\boldsymbol{\mu}$, V; estimate the tangency portfolio from 10 years of data.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(32)
N, T = 10, 120
mu = np.linspace(0.04, 0.12, N) / 12                    # true excess means
beta = np.linspace(0.6, 1.4, N)
V = 0.045**2*np.outer(beta, beta) + np.diag(np.full(N, 0.06**2))
x = np.linalg.solve(V, mu); w_true = x / x.sum()        # true tangency
R = rng.multivariate_normal(mu, V, size=T)
mu_hat, V_hat = R.mean(axis=0), np.cov(R, rowvar=False)
x = np.linalg.solve(V_hat, mu_hat); w_hat = x / x.sum()  # estimated tangency
sr = lambda w, m, S: np.sqrt(12) * (w @ m) / np.sqrt(w @ S @ w)
print("true max SR:", round(sr(w_true, mu, V), 2))
print("estimated, in-sample SR:", round(sr(w_hat, mu_hat, V_hat), 2))
print("estimated, true SR:", round(sr(w_hat, mu, V), 2))
```

```
true weights      -0.02  0.01  0.04  0.06  0.09  0.11  0.14  0.16  0.19  0.22
estimated weights -0.00 -0.21  0.40 -0.38 -0.18  0.83 -0.01  0.35  0.05  0.15
true max SR: 0.49   estimated, in-sample SR: 0.95   estimated, true SR: 0.34
```

► **Key result: promise vs. reality**

- **True SR** (true weights, true parameters): 0.49 — the best possible.
- **In-sample SR** (estimated weights, *estimated* parameters): 0.95 — **overfitting**: what the investor *believes*.
- **True SR of the estimated portfolio** (estimated weights, *true* parameters): 0.34 — what the investor *gets*.

The estimated weights look nothing like the true ones (-0.38 vs. $+0.06$ for asset 4). This gap is the subject of Chapter 4.

3.6.3 Equivalent approach: maximise mean–variance utility

▶ **Key result: MVU formulation**

$$\max_{\boldsymbol{w}} \ \mu_p - \frac{\gamma}{2}\sigma_p^2 = R_f + \boldsymbol{w}^\top(\boldsymbol{\mu} - R_f\mathbf{1}_N) - \frac{\gamma}{2}\boldsymbol{w}^\top V\boldsymbol{w}$$

$$\Rightarrow \quad (\boldsymbol{\mu} - R_f\mathbf{1}_N) - \gamma V\boldsymbol{w} = \mathbf{0} \quad \Rightarrow \quad \boxed{\boldsymbol{w} = \frac{1}{\gamma}V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)}. \tag{10}$$

Varying γ traces the whole frontier. Equations (8) and (10) coincide if

$$\frac{1}{\gamma} = \frac{\mu_{targ} - R_f}{(\boldsymbol{\mu} - R_f\mathbf{1}_N)^\top V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)}.$$

In plain words

Slide question: why do both sides represent the same economic quantity? Both measure how much risk the investor is willing to take. Choosing a high target return μ_{targ} is the same as being risk tolerant (small γ): in both cases you move further out along the capital market line. In the example, $\mu_{targ} = 30\%$ corresponds to $\gamma = 0.6382/0.235 = 2.7157$ — exactly the value used in the Chapter 4 appendix code.

The tangency portfolio does not depend on γ.

$$\boldsymbol{w}_{tang} = \frac{V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)}{\mathbf{1}_N^\top V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)}.$$

γ cancels in numerator and denominator ⇒ **all investors hold risky assets in the same proportions**, whatever their risk aversion. In equilibrium demand = supply, so the tangency portfolio is the **market portfolio**.

Example (used again in Chapter 4). $\gamma = 5$, $R_f = 1.01$, two assets with gross mean 1.09, vol 20%, correlation 0.99: MV weights $(0.201, 0.201)$, $w_f = 0.598$, tangency $(0.5, 0.5)$.

3.6.4 The market portfolio

$$w_n^{mkt} = \frac{\text{market cap of } n}{\text{total market cap}} = \frac{num_n \times P_n}{\sum_{m=1}^{N} num_m \times P_m}.$$

- If the CAPM holds, hold only the market (plus R_f): **passive investing**; little benefit from active funds.
- In practice one holds a **cap-weighted index** that tracks the market (origin of index funds and ETFs). Benefits: broad exposure, little trading (low costs), large firms are liquid, high capacity.
- Limitations: the CAPM may not hold; other factors (size, momentum) may earn more ⇒ factor indexes, other assets (Chapters 5–8).

3.7 Risk–return of individual assets and frontier portfolios (focus)

Questions: how does one asset contribute to portfolio risk? What exactly is systematic risk? How is it related to expected return? Four steps (with a risk-free asset).

▶ **Key result: four marginal contributions**

1. **Expected return.** $\mathbb{E}[R_p] = R_f + \sum_n w_n(\mathbb{E}[R_n] - R_f) \Rightarrow \dfrac{\partial \mathbb{E}[R_p]}{\partial w_n} = \mathbb{E}[R_n] - R_f$ (the risk premium).
2. **Total contribution to variance.** Row n of the table: $\sum_m w_n w_m \sigma_{nm} = w_n \sigma_{np}$, and $\sum_n w_n \sigma_{np} = \sigma_p^2$.
3. **Marginal contribution to variance.** $\dfrac{\partial \sigma_p^2}{\partial w_n} = 2w_n\sigma_n^2 + 2\sum_{m \neq n} w_m \sigma_{nm} = 2\sum_m w_m \sigma_{nm} = 2\,\mathrm{Cov}[R_n, R_p] = 2\sigma_{np}$.
4. **Marginal contribution to volatility.** From $\frac{\partial \sigma_p^2}{\partial w_n} = 2\sigma_p \frac{\partial \sigma_p}{\partial w_n}$: $\dfrac{\partial \sigma_p}{\partial w_n} = \dfrac{\sigma_{np}}{\sigma_p}$.

In matrix form: the vector of covariances with the portfolio is $V\boldsymbol{w}$.

In plain words

An asset's risk inside a portfolio is **not** its own volatility; it is how much it *moves with the portfolio* (σ_{np}/σ_p). A very volatile asset that is uncorrelated with the portfolio adds almost no risk at the margin.

Python exercise 3: where does the risk of a 60/40 portfolio come from?

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(33)
T = 240
vol = np.array([0.16, 0.06]) / np.sqrt(12)          # stocks, bonds
C = np.array([[1, 0.1], [0.1, 1]])
R = rng.multivariate_normal([0.006, 0.003], C*np.outer(vol, vol), size=T)
V = np.cov(R, rowvar=False)
w = np.array([0.6, 0.4])
var_p = w @ V @ w
cov_np = V @ w                       # covariance of each asset with the portfolio
marginal = cov_np / np.sqrt(var_p)   # d sigma_p / d w_n
share = w * cov_np / var_p           # share of portfolio risk (sums to 1)
print(pd.DataFrame({"weight": w, "marginal": marginal, "risk share": share},
                   index=["stocks", "bonds"]).round(3))
```

```
        weight  marginal  risk share
stocks     0.6     0.044       0.925
bonds      0.4     0.005       0.075
```

Stocks are 60% of the money but 92.5% of the risk: a "balanced" 60/40 portfolio is almost a pure bet on stocks. Risk-parity portfolios (Chapter 6) equalise risk shares.

3.7.1 The beta relation

Definition: Return-to-risk ratio (RRR)

$$RRR_n = \frac{\text{marginal return}}{\text{marginal risk}} = \frac{\partial \mathbb{E}[R_p]/\partial w_n}{\partial \sigma_p/\partial w_n} = \frac{\mathbb{E}[R_n] - R_f}{\sigma_{np}/\sigma_p}.$$

Claim: for any frontier portfolio p, every asset has the same RRR, equal to the portfolio's:

$$\frac{\mathbb{E}[R_n] - R_f}{\sigma_{np}/\sigma_p} = \frac{\mathbb{E}[R_p] - R_f}{\sigma_{pp}/\sigma_p} = \frac{\mathbb{E}[R_p] - R_f}{\sigma_p}.$$

Intuition

If one asset had a higher return per unit of marginal risk, you could buy a bit more of it and sell a bit of another: the portfolio's return would rise for the same risk, so the portfolio would not have been on the frontier. At the optimum, all marginal "prices of risk" are equalised (like equalising marginal utility per euro across goods in microeconomics).

▶ **Key result: beta pricing relation**

Rearranging,

$$\mathbb{E}[R_n] - R_f = \beta_{np}(\mathbb{E}[R_p] - R_f), \qquad \beta_{np} = \frac{\sigma_{np}}{\sigma_p^2} = \frac{\mathrm{Cov}[R_n, R_p]}{\mathrm{Var}[R_p]},$$

for any frontier portfolio p (other than R_f). β_{np} = amount of systematic risk; $\mathbb{E}[R_p] - R_f$ = price per unit of risk. **The CAPM is this relation with p = market**; it holds only if the market is on the frontier.

Additional note (not on the slides)

Quick proof with the tangency portfolio. $\boldsymbol{w}_T \propto V^{-1}\boldsymbol{e}$ means $V\boldsymbol{w}_T = c\,\boldsymbol{e}$ for some $c > 0$, i.e. $\sigma_{nT} = c\,(\mathbb{E}[R_n] - R_f)$ for every n. Multiply by $w_{T,n}$ and sum: $\sigma_T^2 = c\,(\mathbb{E}[R_T] - R_f)$. Divide the two: $\frac{\sigma_{nT}}{\sigma_T^2} = \frac{\mathbb{E}[R_n]-R_f}{\mathbb{E}[R_T]-R_f}$, i.e. $\mathbb{E}[R_n] - R_f = \beta_{nT}(\mathbb{E}[R_T] - R_f)$. □

Python exercise 4: checking the beta relation

```
import numpy as np
import pandas as pd
N = 5
mu = np.array([0.05, 0.06, 0.08, 0.09, 0.11])          # excess means
```

```
vol = np.array([0.12, 0.15, 0.18, 0.22, 0.25])
C = 0.3*np.ones((N, N)) + 0.7*np.eye(N)
V = C * np.outer(vol, vol)
x = np.linalg.solve(V, mu)
w = {"tangency": x / x.sum(), "1/N": np.full(N, 1/N)}
beta = {p: V @ w[p] / (w[p] @ V @ w[p]) for p in w}    # beta of each asset w.r.t. p
prem = {p: w[p] @ mu for p in w}                       # premium of p
print(pd.DataFrame({"E[R]-Rf": mu,
                    "beta x prem (tangency)": beta["tangency"]*prem["tangency"],
                    "beta x prem (1/N)": beta["1/N"]*prem["1/N"]}).round(4))
```

```
   E[R]-Rf  beta x prem (tangency)  beta x prem (1/N)
1     0.05                    0.05             0.0443
2     0.06                    0.06             0.0586
3     0.08                    0.08             0.0743
4     0.09                    0.09             0.0971
5     0.11                    0.11             0.1157
```

Exact for the tangency portfolio (on the frontier); not for 1/N (not on the frontier).

3.8 Portfolio optimisation with respect to a benchmark

Fund managers are judged against a **benchmark** (the investor's alternative is a cheap index fund). Because mean returns are noisy, investors focus on **tracking error volatility**.

▶ **Key result: tracking error**

$$TE = R_A - R_B = (\boldsymbol{w}_A - \boldsymbol{w}_B)^\top \boldsymbol{R}, \qquad \mathbb{E}[TE] = (\boldsymbol{w}_A - \boldsymbol{w}_B)^\top \boldsymbol{\mu},$$
$$\mathrm{Vol}(TE) = \sqrt{(\boldsymbol{w}_A - \boldsymbol{w}_B)^\top V (\boldsymbol{w}_A - \boldsymbol{w}_B)}.$$

Manager's problem (Roll 1992, **TEV criterion**):

$$\max_{\boldsymbol{w}_A} (\boldsymbol{w}_A - \boldsymbol{w}_B)^\top \boldsymbol{\mu} \quad \text{s.t.} \quad \sqrt{(\boldsymbol{w}_A - \boldsymbol{w}_B)^\top V (\boldsymbol{w}_A - \boldsymbol{w}_B)} \le \sigma_{\max}, \quad \boldsymbol{w}_A^\top \mathbf{1}_N = 1.$$

Same as Markowitz, but in "active" space: active return instead of total return, tracking-error volatility instead of total volatility.

Ratio	Formula		Counterpart of
Information ratio	$\dfrac{(\boldsymbol{w}_A - \boldsymbol{w}_B)^\top \boldsymbol{\mu}}{\sqrt{(\boldsymbol{w}_A - \boldsymbol{w}_B)^\top V (\boldsymbol{w}_A - \boldsymbol{w}_B)}}$ $\dfrac{\text{expected TE}}{\text{TE volatility}}$	=	Sharpe ratio (benchmark instead of R_f)
Appraisal ratio	$\dfrac{\alpha}{\sigma(\varepsilon)}$: alpha from a factor regression over the residual volatility		Treynor-like numerator, but risk = unexplained risk

Python exercise 5: tracking error, IR and appraisal ratio

Fund with true alpha 2.4%/yr and beta 1.4 vs. a benchmark with beta 1.

```
import numpy as np
rng = np.random.default_rng(35)
T = 240
m = rng.normal(0.005, 0.045, T)       # benchmark excess return
e = rng.normal(0, 0.01, T)            # manager's own risk
a = 0.002 + 1.4*m + e                 # fund excess return
active = a - m
te_vol = active.std(ddof=1)
ir = np.sqrt(12) * active.mean() / te_vol
beta, alpha = np.polyfit(m, a, 1)     # slope, intercept of the regression
resid_vol = (a - alpha - beta*m).std(ddof=2)
print("IR:", round(ir, 2), " Appraisal:", round(np.sqrt(12)*alpha/resid_vol, 2))
```

```
Beta: 1.39  alpha (annual): 0.0263
Residual volatility (annual): 0.0319   Tracking-error volatility (annual): 0.071
Information ratio: 1.04   Appraisal ratio: 0.82
```

TE volatility (7.1%) is more than twice the residual volatility (3.2%) because it includes the extra beta. The IR (1.04) rewards the fund also for the extra market exposure; the **appraisal ratio isolates skill** (0.82).

Additional note (not on the slides)

`np.polyfit(x, y, 1)` fits $y = b\,x + a$ and returns `[b, a]` (slope first!). `ddof=2` because two parameters were estimated.

3.9 Other constraints and transaction costs

Other constraints. Add to the problem: (3) $w_n \geq w_{\min}$ (long-only if $w_{\min} = 0$), (4) $w_n \leq w_{\max}$. No closed form; solve with a numerical optimiser (`scipy.optimize.minimize`, Chapter 4 appendix).

Three ways to handle transaction costs.

1. Ignore them in the optimisation, subtract them from the returns: $r^{net}_{t+1} = R_{t+1} - \sum_n |w_{t+1,n} - w_{t^+,n}| \times TC$.
2. Include them in a **myopic** optimisation (theoretically incorrect but easy):

$$\max_{\boldsymbol{w}} \; \boldsymbol{w}^\top \boldsymbol{\mu} - \frac{1}{\gamma} \boldsymbol{w}^\top V \boldsymbol{w} - TC \, \|\boldsymbol{w} - \boldsymbol{w}_0\|_1 \quad \text{s.t.} \; \sum_n w_n = 1,$$

   with $\boldsymbol{w}_0$ the portfolio before trading and $\|\boldsymbol{w} - \boldsymbol{w}_0\|_1 = \sum_n |w_n - w_{0,n}|$.
3. Include them in a **dynamic** optimisation (correct but hard; DeMiguel–Uppal 2005; not in the course).

In plain words

The $\|\boldsymbol{w}-\boldsymbol{w}_0\|_1$ penalty makes it costly to move away from what you already hold, so the optimiser trades only when the expected gain beats the cost. It creates a "no-trade zone" around the current portfolio.

3.10 Summary of modern portfolio theory

1. Investors hold **frontier portfolios**; more assets improve the frontier.
2. With a risk-free asset, frontier portfolios are combinations of R_f and the **tangency portfolio** (two-fund separation).
3. An asset's risk is its **contribution to portfolio risk**: covariances matter, individual variances barely do.
4. For any asset, risk = beta with respect to a frontier portfolio; reward per unit of risk = that portfolio's premium.

Libraries: `PyPortfolioOpt` (MV optimisation, Black–Litterman), `Riskfolio-Lib`; EDHEC's Coursera code.

Common mistakes to avoid

1. Writing the portfolio variance as $\sum w_n^2\sigma_n^2$ (forgetting the covariances).
2. Confusing $V^{-1}\mathbf{1}_N$ (raw GMV weights) with $V^{-1}\boldsymbol{\mu}$; or forgetting to normalise GMV and tangency weights so that they sum to one.
3. Thinking the tangency portfolio depends on γ or on μ_{targ}: it depends only on $\boldsymbol{\mu}$, V and R_f.
4. Using gross returns and a net risk-free rate in the same formula: $\boldsymbol{\mu} - R_f\mathbf{1}_N$ must be both gross or both net (the difference is the same).
5. Reading an asset's own volatility as its contribution to portfolio risk: the contribution is σ_{np}/σ_p.
6. In code, mixing column vectors `(N,1)` and flat arrays `(N,)`: results become matrices of the wrong shape. Pick one convention.

3.11 Self-test questions

1. Two assets, $\sigma_1 = 0.2$, $\sigma_2 = 0.3$, $\rho = 0.5$, $w = (0.6, 0.4)$. Portfolio vol? *($0.36 \cdot 0.04 + 0.16 \cdot 0.09 + 2 \cdot 0.24 \cdot 0.03 = 0.0432 \Rightarrow \sigma = 0.208$.)*
2. Equal-weighted, average variance 0.09, average covariance 0.02, $N = 10$. Portfolio variance? *($0.009 + 0.9 \cdot 0.02 = 0.027$.)* Limit as $N \to \infty$? *(0.02.)*
3. Write a_1, a_2, a_3 and the GMV weights in terms of them. *($\boldsymbol{w}_{GMV} = V^{-1}\mathbf{1}_N/a_3$.)*
4. Why does γ not appear in the tangency portfolio? *(It cancels when normalising.)*

5. With R_f, what is the Sharpe ratio of every frontier portfolio? *(The same, maximum, Sharpe ratio:* $\sqrt{(\boldsymbol{\mu} - R_f \mathbf{1}_N)^\top V^{-1} (\boldsymbol{\mu} - R_f \mathbf{1}_N)}$*.)*
6. In-sample SR 0.95, true max 0.49, true SR of estimated portfolio 0.34. Explain each.
7. An asset has $\sigma_{np} = 0$. What is its marginal contribution to portfolio volatility? *(Zero.)*
8. Difference between IR and appraisal ratio? *(IR uses total active return/TE; appraisal uses alpha/residual vol, so it removes beta exposure.)*
9. Write by hand the code for the tangency portfolio and its Sharpe ratio.

```
ex = mu - rf * np.ones(len(mu))
x = np.linalg.solve(V, ex)
w_tan = x / x.sum()
sr_max = np.sqrt(ex @ x)                  # = sqrt(ex' V^{-1} ex)
```

10. Write by hand the frontier weights for a target return `m` (no R_f).

```
o = np.ones(len(mu)); Vi_o = np.linalg.solve(V, o); Vi_mu = np.linalg.solve(V, mu)
a1, a2, a3 = mu @ Vi_mu, mu @ Vi_o, o @ Vi_o; D = a1*a3 - a2**2
w = ((a1*Vi_o - a2*Vi_mu) + (a3*Vi_mu - a2*Vi_o)*m) / D
```

3.12 To do for next class

Readings: Markowitz (1952, 1959), Merton (1972), Roll (1977, 1980); Huang–Litzenberger (1988, ch. 3); Moreira (2021, ch. 7–8); Tidy Finance, "Constrained Optimization and Backtesting". Start Assignment #2, Question 1.

References (Chapter 3)

- DeMiguel, V., and R. Uppal. 2005. Portfolio investment with the exact tax basis via nonlinear programming. *Management Science* 51(2): 277–290.
- Huang, C.-F., and R. Litzenberger. 1988. *Foundations for Financial Economics.* Prentice Hall.
- Markowitz, H. M. 1952. Portfolio selection. *JF* 7(1): 77–91; 1959. *Portfolio Selection.* Wiley.
- Merton, R. C. 1972. An analytic derivation of the efficient portfolio frontier. *JFQA* 7(4): 1851–1872.
- Roll, R. 1977. A critique of the asset pricing theory's tests. *JFE* 4(2): 129–176; 1980. Orthogonal portfolios. *JFQA* 15(5): 1005–1023; 1992. A mean/variance analysis of tracking error. *JPM* 18(4): 13–22.

Chapter 4

Mean–Variance Portfolios that Adjust for Estimation Error

In plain words

Chapter 3 gave the perfect recipe $\boldsymbol{w} = \frac{1}{\gamma} V^{-1}(\boldsymbol{\mu} - R_f \mathbf{1}_N)$, assuming we *know* $\boldsymbol{\mu}$ and V. Chapter 4 shows that with *estimated* inputs the recipe produces crazy, unstable portfolios that lose to the naive 1/N rule. Then it studies the fixes. The single idea that connects all of them is **shrinkage**: pull noisy estimates towards something simple and sensible.

4.1 What we want to do in Chapter 4

1. Show that the **out-of-sample** performance of sample-based MV portfolios is poor.
2. Study methods to improve it: first those that fix the error in $\mathbb{E}[\boldsymbol{R}]$ (Bayesian, short-sale constraints), then those that fix $V[\boldsymbol{R}]$ (GMV, Jagannathan–Ma, Ledoit–Wolf, norm constraints).

4.2 The effect of estimation error on portfolio weights (focus)

$$\boldsymbol{w} = \underbrace{\frac{1}{\gamma}}_{\text{risk tolerance}} \underbrace{(V[\boldsymbol{R}])^{-1}}_{\text{ill-conditioned}} \underbrace{(\mathbb{E}[\boldsymbol{R}] - R_f \mathbf{1}_N)}_{\text{low precision}}.$$

4.2.1 Error in $\mathbb{E}[\boldsymbol{R}]$

Merton (1980): sampling more often does not improve the mean (only the first and last prices matter; Chapter 1). With annual volatility σ_n and T years:

$$SE(\hat{\mu}_n) = \frac{\sigma_n}{\sqrt{T}}.$$

▶ **Key result: how bad is it?**

Individual stock: $\sigma_n \approx 30\%$. With 9 years, $SE = 0.30/\sqrt{9} = 10\%$, so the 95% CI is $\pm 20\%$ (a band 40% wide). With expected returns around 10%: $0.10 \pm 0.20 = [-10\%, 30\%]$. **Useless in practice.** *Bottom line: beware of any model that uses expected returns as an input.*

Python exercise 1: means are hard to estimate, volatilities are not

True $\mu = 10\%$, $\sigma = 30\%$; 5,000 samples of 5, 10, 40 years of monthly data.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(41)
mu, sigma = 0.10, 0.30
S = 5000
sim = lambda T: rng.normal(mu/12, sigma/np.sqrt(12), size=(S, T))
est = lambda R: pd.DataFrame({"mean": 12 * R.mean(axis=1),
                              "volatility": np.sqrt(12) * R.std(axis=1, ddof=1)})
ranges = pd.concat({f"{T//12} years": est(sim(T)).quantile([0.025, 0.975])
                    for T in [60, 120, 480]})
print(ranges.round(3))
print("Years needed for +-2% (95%):", round((2 * sigma / 0.02)**2))
```

```
                 mean  volatility
5 years  0.025 -0.166       0.245
         0.975  0.360       0.354
10 years 0.025 -0.084       0.262
         0.975  0.285       0.337
40 years 0.025  0.006       0.281
         0.975  0.195       0.319
Years needed to estimate the mean within 2% (95% of the time): 900
```

In plain words

With 10 years, the estimated mean can be anything between -8% and $+28\%$, while the volatility is between 26% and 34%. To pin the mean down to $\pm 2\%$ you need $(2\sigma/0.02)^2 = (0.6/0.02)^2 = 900$ years of data.

4.2.2 Error in $V[\boldsymbol{R}]$

$N(N+1)/2$ parameters: 1,275 for $N = 50$; 5,050 for $N = 100$.

- **Singular** (not invertible) when $T \leq N$: for $N = 100$, 5 years of monthly data ($T = 60$) are not enough.
- **Ill-conditioned** unless $T \gg N$: even $T = 240$ gives a poorly conditioned matrix for $N = 100$.

Result: sample-based weights are **unreasonable** (extreme), **unstable** (fluctuate over time), **inefficient** (poor out of sample).

4.2.3 A two-asset example

Same setting as Chapter 3: $\gamma = 5$, $R_f = 1.01$, both assets gross mean 1.09, vol 20%, correlation 0.99. True answer: MV weights $(0.20, 0.20)$, $w_f = 0.60$, tangency $(0.5, 0.5)$. Now suppose asset 1's mean is **estimated** as 1.10 instead of 1.09:

	True parameters	Mean of asset 1 = 1.10
MV weights	(0.201, 0.201)	(2.714, −2.286)
Risk-free weight	0.598	0.573
Tangency weights	(0.5, 0.5)	(6.353, −5.353)

In plain words

A 1% error in one mean turns a 50/50 portfolio into $+635\%/-535\%$. Why? The assets are 99% correlated, so "long asset 1, short asset 2" is almost riskless and earns the 1% difference. The optimiser loves it, but the 1% is probably just estimation noise. With real data and more assets the weights are even more extreme, and out-of-sample Sharpe ratios can be *negative* (worse than putting everything in the risk-free asset).

Worked example: why the weights explode, with a formula

Two assets with the same volatility σ and correlation ρ. Using the 2×2 inverse,

$$w_1 - w_2 = \frac{1}{\gamma} \frac{\mu_1 - \mu_2}{\sigma^2(1-\rho)}.$$

The difference between the two weights is the difference in means divided by $\sigma^2(1-\rho)$, the variance of the long–short position. With $\gamma = 5$, $\sigma = 0.2$, $\rho = 0.99$ and $\mu_1 - \mu_2 = 0.01$: $w_1 - w_2 = 0.01/(5 \times 0.04 \times 0.01) = 5$, exactly $2.714 - (-2.286)$. With $\rho = 0.5$ the same 1% error would give $w_1 - w_2 = 0.1$ only. High correlation turns small errors into huge positions.

Python exercise 2: small changes in means, large changes in weights

Five identical assets (8% excess mean, 20% vol, correlation ρ); tangency = $1/N$. Raise asset 1's mean by 1% (about one-sixth of its SE with 10 years).

```
import numpy as np
import pandas as pd
N, vol = 5, 0.20
mu = np.full(N, 0.08)
mu_new = mu + 0.01 * np.eye(N)[0]                    # asset 1: +1%
V = lambda rho: vol**2 * ((1 - rho)*np.eye(N) + rho)
tang = lambda m, rho: (lambda x: x / x.sum())(np.linalg.solve(V(rho), m))
res = pd.DataFrame({rho: tang(mu_new, rho) for rho in [0.0, 0.5, 0.8, 0.95]},
                   index=[f"asset {n}" for n in range(1, N+1)])
print(res.round(2))
```

```
correlation  0.00  0.50  0.80  0.95
asset 1      0.22  0.32  0.61  2.07
asset 2      0.20  0.17  0.10 -0.27
asset 3-5    0.20  0.17  0.10 -0.27
Standard error of the mean, 10 years of data: 0.063
```

The more similar the assets, the more the optimiser bets on differences in means that are pure estimation error.

4.3 Performance of sample-based mean–variance portfolios

4.3.1 Methodology for out-of-sample evaluation (focus)

▶ Key result: the rolling-window backtest (used in almost every paper and in Assignment 2)

1. Choose an estimation window $T^{est} < T$ (monthly data: usually 60 or 120 months).
2. Estimate the parameters on the window.
3. Compute the optimal weights with the estimated parameters.
4. Hold them for the **next** period and record the return (out of sample).
5. Roll: drop the oldest observation, add the newest, repeat until the end.
6. Compute performance metrics on the out-of-sample returns (SR, turnover, drawdowns, ...).

At month $t = 60, \ldots, T-1$, with the previous 60 months:

$$\hat{\boldsymbol{\mu}}_t = \frac{1}{60} \sum_{s=t-59}^{t} \boldsymbol{R}_s, \qquad \hat{V}_t = \frac{1}{60-1} \sum_{s=t-59}^{t} (\boldsymbol{R}_s - \hat{\boldsymbol{\mu}}_t)(\boldsymbol{R}_s - \hat{\boldsymbol{\mu}}_t)^\top,$$

$$\boldsymbol{w}_t^{MV} = \frac{1}{\gamma} \hat{V}_t^{-1} (\hat{\boldsymbol{\mu}}_t - R_f \mathbf{1}_N), \qquad R_{p,t+1}^{MV} = (1 - \mathbf{1}_N^\top \boldsymbol{w}_t^{MV}) R_f + (\boldsymbol{w}_t^{MV})^\top \boldsymbol{R}_{t+1},$$

$$\boldsymbol{w}_t^{tang} = \frac{\boldsymbol{w}_t^{MV}}{\mathbf{1}_N^\top \boldsymbol{w}_t^{MV}}, \qquad R_{p,t+1}^{tang} = (\boldsymbol{w}_t^{tang})^\top \boldsymbol{R}_{t+1},$$

$$\boldsymbol{w}_t^{EW} = \tfrac{1}{N} \mathbf{1}_N, \qquad R_{p,t+1}^{EW} = (\boldsymbol{w}^{EW})^\top \boldsymbol{R}_{t+1}, \qquad SR = \frac{12\, \mathbb{E}[R - R_f]}{\sqrt{12}\, \sigma[R - R_f]} \text{ (annualised).}$$

Then test $H_0 : SR^{MV} = SR^{EW}$ with Jobson–Korkie or a bootstrap (Chapter 2).

Python exercise 3: an out-of-sample backtest

50 years of monthly excess returns on 10 assets (one-factor model, CAPM holds); window $M = 120$; three rules: MV ($\gamma = 5$), GMV, 1/N.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(45)
N, T, M = 10, 600, 120
gamma = 5
beta = np.linspace(0.6, 1.4, N)
mu = 0.005 * beta
V = 0.045**2*np.outer(beta, beta) + np.diag(np.full(N, 0.06**2))
R = rng.multivariate_normal(mu, V, size=T)

inv = lambda X, b: np.linalg.solve(np.cov(X, rowvar=False), b)
rules = {"mean-variance": lambda X: inv(X, X.mean(axis=0)) / gamma,
         "GMV": lambda X: (lambda x: x / x.sum())(inv(X, np.ones(N))),
```

```
        "1/N": lambda X: np.full(N, 1 / N)}
W = {k: np.array([f(R[t-M:t]) for t in range(M, T)]) for k, f in rules.items()}
oos = {k: (w * R[M:]).sum(axis=1) for k, w in W.items()}       # out-of-sample
sr = lambda r: np.sqrt(12) * r.mean() / r.std(ddof=1)
tsr = lambda w: (lambda m, v: np.sqrt(12) * m.mean() / np.sqrt(v.mean() + m.var()))(
    w @ mu, np.einsum("tn,nk,tk->t", w, V, w))                 # true SR
to = lambda w: np.abs(np.diff(w, axis=0)).sum(axis=1).mean()   # turnover
print(pd.DataFrame({"realized SR": {k: sr(r) for k, r in oos.items()},
                    "true SR": {k: tsr(w) for k, w in W.items()},
                    "turnover": {k: to(w) for k, w in W.items()}}).round(2))
```

```
True maximum Sharpe ratio: 0.36
               realized SR  true SR  turnover
mean-variance         0.09     0.12      0.34
GMV                   0.24     0.28      0.06
1/N                   0.30     0.35      0.00
```

MV gets a third of the maximum SR and trades the most. GMV (no means) is much better. 1/N is best here because it is close to the true tangency portfolio. Each realised SR has an SE of about $1/\sqrt{40} \approx 0.16$ (40 years out of sample).

Additional note (not on the slides)

Reading the code. `R[t-M:t]` contains months $t-M, \ldots, t-1$, so the weights for month t use only the past (no look-ahead). `(w * R[M:]).sum(axis=1)` multiplies each row of weights by the *same row* of returns: row j of `W` was estimated with data up to $M+j-1$ and is applied to the return of month $M+j$. `np.einsum("tn,nk,tk->t", w, V, w)` computes $\boldsymbol{w}_t^\top V \boldsymbol{w}_t$ for every t at once.

4.3.2 Evidence: DeMiguel, Garlappi and Uppal (2009)

Six datasets (Ken French, MSCI, S&P sectors), $T^{est} = 120$ months.

	S&P sectors $N=11$	Industry $N=11$	Int'l $N=9$	Mkt+SMB/HML $N=3$	FF 1-factor $N=21$	FF 4-factor $N=24$
MV (in sample)	0.3848	0.2124	0.2090	0.2851	0.5098	0.5364
MV (out of sample)	0.0794	−0.0363	−0.0719	0.2186	−0.0684	−0.0031
1/N	0.1876	0.1353	0.1277	0.2240	0.1623	0.1753
p-value	(0.12)	(0.01)	(0.01)	(0.46)	(0.00)	(0.01)
MV turnover / 1/N turnover	38.99	607,479.61	4,236.10	2.83	9,408.16	2,672.71

▶ **Key result: conclusions**

1. Out-of-sample MV is far worse than in-sample; **negative SR in 4 of 6 datasets**.
2. **1/N beats MV in all six**, significantly at 5% in four.
3. MV turnover is enormous (up to 600,000 times that of 1/N).

Additional note (not on the slides)

How long would we need to wait for MV to beat 1/N? DGU (2009) compute that the estimation window needed for the sample-based MV strategy (and its extensions) to outperform 1/N is around **3,000 months (250 years) for 25 assets** and about **6,000 months for 50 assets**. With realistic data, 1/N is simply very hard to beat.

4.4 Models that adjust for estimation error: the menu

The problem has been known since the 1970s.

Bayesian	Diffuse priors (Barry 1974; Bawa–Brown–Klein 1979); shrinkage of means (Jobson–Korkie–Ratti 1979; Jobson–Korkie 1980; Jorion 1985, 1986); covariance estimation (Best–Grauer 1992; Chan–Karceski–Lakonishok 1999; Ledoit–Wolf 2004); Black–Litterman (1990, 1992; Chapter 5).
Non-Bayesian	Short-sale constraints (Frost–Savarino 1988; Chopra 1993; Jagannathan–Ma 2003); robust optimisation (Goldfarb–Iyengar 2003; Uppal–Wang 2003; Garlappi–Uppal–Wang 2007; Boyle et al. 2012); optimal diversification across market and estimation risk (Kan–Zhou 2005); resampling (Michaud 1998).

▶ **Key result: the central idea of the whole course**

Shrinkage. Almost every method can be read as shrinking $\hat{\boldsymbol{\mu}}$ and $\hat{V}$ towards a sensible target:

- means $\to$ the average of all means, or the market's expected return;
- covariance $\to$ a one-factor (market-model) matrix, or a matrix with all correlations equal, or a multiple of the identity.

In plain words

Think of a new restaurant with three 5-star reviews. You do not believe it is the best restaurant in town: you "shrink" the score towards the average restaurant, because three reviews are too few. The less data you have, the more you shrink. Exactly the same logic applies to sample means and covariances.

4.5 Bayesian approaches

Bayesian estimates of $\boldsymbol{\mu}$ and V come from the **predictive (posterior) distribution**, obtained by integrating the likelihood $f(\boldsymbol{R} \mid \boldsymbol{\mu}, V)$ over the parameters with respect to a prior $p(\boldsymbol{\mu}, V)$.

1. Diffuse prior (Barry 1974; Klein–Bawa 1976; Brown 1979). With $p(\boldsymbol{\mu}, V) \propto |V|^{-(N+1)/2}$ and Normal likelihood, the predictive distribution is Student-t with mean $\hat{\boldsymbol{\mu}}$ and variance $\hat{V}(1+1/T^{est})$. With $T^{est} = 60$ the correction is $1/60 < 2\%$: **practically the same as sample MV**.

2. Bayes–Stein shrinkage (Stein 1956; James–Stein 1961; Jorion 1985, 1986).

▶ **Key result: Bayes–Stein estimator of the means**

$$\hat{\boldsymbol{\mu}}_t^{BS} = (1-\hat{\psi}_t)\,\hat{\boldsymbol{\mu}}_t + \hat{\psi}_t\,\bar{\mu}_t \mathbf{1}_N,$$
$$\hat{\psi}_t = \frac{N+2}{(N+2) + T^{est}\,(\hat{\boldsymbol{\mu}}_t - \mu_t^{GMV}\mathbf{1}_N)^\top \hat{V}_t^{-1}(\hat{\boldsymbol{\mu}}_t - \mu_t^{GMV}\mathbf{1}_N)},$$

with $0 < \hat{\psi}_t < 1$, $\hat{V}_t = \frac{1}{T^{est}-N-2}\sum(\boldsymbol{R}_s - \hat{\boldsymbol{\mu}}_t)(\boldsymbol{R}_s - \hat{\boldsymbol{\mu}}_t)^\top$, and grand mean $\bar{\mu}_t = \mu_t^{GMV}$ (Jorion: the mean return of the GMV portfolio).

Intuition

$\hat{\psi}$ is large (more shrinkage) when: T^{est} is small (little data); the sample means are close to each other relative to their noise (the quadratic form is small, i.e. the differences between means are not statistically convincing); N is large (many means to estimate). In practice the error is so large that the optimal weight on the grand mean is often very high (the slides mention around 95% in applications).

Worked example: size of the Bayes–Stein shrinkage

$N = 10$ assets, $T^{est} = 60$ months, so $N+2 = 12$ and $\hat{\psi} = 12/(12+60\,q)$ where q is the quadratic form (how far the sample means are from the grand mean, measured in units of their risk):

q (dispersion of means)	0.05	0.2	1.0
$\hat{\psi}$ (weight on grand mean)	0.80	0.50	0.17

When the sample means are close together (small q) we cannot trust their differences, so we shrink a lot. Only very dispersed means survive.

Python exercise 4: Bayes–Stein shrinkage of the means

2,000 samples, 5 years, 10 assets.

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(44)
N, T, S = 10, 60, 2000
beta = np.linspace(0.6, 1.4, N)
mu = 0.005 * beta
V = 0.045**2*np.outer(beta, beta) + np.diag(np.full(N, 0.06**2))
R = rng.multivariate_normal(mu, V, size=(S, T))        # S samples of T x N
m = R.mean(axis=1)                                     # S x N sample means
D = R - m[:, None, :]
```

```
Vh = np.einsum("stn,stk->snk", D, D) / (T - N - 2)     # S covariance matrices
Vi = np.linalg.inv(Vh)
wg = Vi.sum(axis=2) / Vi.sum(axis=(1, 2))[:, None]     # GMV weights
m0 = (wg * m).sum(axis=1)                              # GMV mean = grand mean
d = m - m0[:, None]
q = np.einsum("sn,snk,sk->s", d, Vi, d)
psi = (N + 2) / ((N + 2) + T * q)                      # shrinkage factor
m_bs = (1 - psi)[:, None] * m + (psi * m0)[:, None]

mse = lambda e: (144 * ((e - mu)**2).sum(axis=1)).mean()
tsr = lambda e: (lambda w: np.sqrt(12) * (w @ mu) / np.sqrt(np.einsum("sn,nk,sk->s", w, V, w)))(
    np.einsum("snk,sk->sn", Vi, e))
est = {"sample": m, "Bayes-Stein": m_bs}
print(pd.DataFrame({"MSE": {k: mse(e) for k, e in est.items()},
                    "median true SR": {k: np.median(tsr(e)) for k, e in est.items()}}).round(3))
```

```
Average shrinkage factor psi: 0.6     True maximum Sharpe ratio: 0.36
              mean squared error   median true SR
sample                     0.138            0.085
Bayes-Stein                0.068            0.148
```

On average 60% weight on the grand mean; shrinkage **halves the MSE** of the means; the true SR rises from 0.09 to 0.15, still far below 0.36. **Shrinking the means helps but does not solve the problem.**

3. "Data-and-Model" (DM) (Pástor 2000; Pástor–Stambaugh 2000). The shrinkage target comes from an **asset-pricing model** (CAPM, FF3, FF4), and the amount of shrinkage from how strongly the investor believes it. The prior on mispricing α is Normal around 0 with $\sigma_\alpha = 1\%$ p.a.: with 95% probability mispricing is between -2% and $+2\%$. This removes the arbitrary choice of target and of ψ.

Performance (DGU 2009).

	S&P	Ind.	Int'l	Mkt+SMB/HML	FF1	FF4
1/N	0.1876	0.1353	0.1277	0.2240	0.1623	0.1753
Bayes–Stein	0.0811	−0.0319	−0.0528	**0.2536**	−0.0636	−0.0042
Data-Model ($\sigma_\alpha = 1\%$)	0.1410	−0.0475	0.1032	0.0181	0.0745	**0.2355**
Turnover / 1/N: Bayes–Stein	22.41	10,092	1,761	1.85	10,510	2,784
Turnover / 1/N: Data-Model	1.72	20,468	45.24	76.85	762.83	32.46

Bayes–Stein: negative SR in 4 of 6. Each Bayesian strategy beats 1/N in only one dataset. Turnover lower than MV but much higher than 1/N. **Conclusion: out of sample, 1/N usually beats the Bayesian strategies.**

4.6 Short-sale-constrained portfolios (focus)

In the two-asset example the problem was $+271\%/-228\%$. Requiring $w_n \geq 0$ limits how extreme weights can be. It is ad hoc but popular (and often required: pension funds by regulation; mutual funds find shorting costly). It can be shown formally to be **shrinkage**.

Strategies. MV-C (sample MV with $w \geq 0$), BS-C (Bayes–Stein with $w \geq 0$), GMV-C (next section).

4.6.1 Why short-sale constraints shrink the means

$$\max_{\boldsymbol{w}} \; R_f + \boldsymbol{w}^\top(\boldsymbol{\mu} - R_f \mathbf{1}_N) - \frac{\gamma}{2}\boldsymbol{w}^\top V \boldsymbol{w} \qquad \text{s.t. } w_n \geq 0 \; \forall n.$$

Lagrangian with multipliers $\boldsymbol{\lambda}_{ss} = (\lambda_1, \ldots, \lambda_N)^\top \geq \mathbf{0}$:

$$\begin{aligned} L &= R_f + \boldsymbol{w}^\top(\boldsymbol{\mu} - R_f \mathbf{1}_N) - \frac{\gamma}{2}\boldsymbol{w}^\top V \boldsymbol{w} + \boldsymbol{w}^\top \boldsymbol{\lambda}_{ss} \\ &= R_f + \boldsymbol{w}^\top(\underbrace{\boldsymbol{\mu} + \boldsymbol{\lambda}_{ss}}_{\text{adjusted mean}} - R_f \mathbf{1}_N) - \frac{\gamma}{2}\boldsymbol{w}^\top V \boldsymbol{w}. \end{aligned}$$

▶ **Key result: interpretation**

The constrained solution equals the *unconstrained* solution with means $\boldsymbol{\mu} + \boldsymbol{\lambda}_{ss}$. The constraint binds ($\lambda_n > 0$) for assets with **low** μ_n (those the optimiser wanted to short), and their mean is **raised** to $\mu_n + \lambda_n$. Low means are pushed up towards the others: **shrinkage of expected returns towards the average.**

Additional note (not on the slides)

Complementary slackness (KKT). For each asset either $w_n > 0$ and $\lambda_n = 0$ (constraint not binding, mean untouched) or $w_n = 0$ and $\lambda_n \geq 0$ (just enough to make the optimiser stop wanting to go short).

Performance (DGU 2009).

	S&P	Ind.	Int'l	Mkt+SMB/HML	FF1	FF4
MV (out of sample)	0.0794	−0.0363	−0.0719	0.2186	−0.0684	−0.0031
1/N	0.1876	0.1353	0.1277	0.2240	0.1623	0.1753
MV-C	0.0892	0.0678	0.0848	0.1084	0.1977	0.2024
BS-C	0.1075	0.0819	0.0848	0.1514	0.1955	0.2062
Turnover / 1/N: MV-C	4.53	7.17	7.23	4.12	17.53	13.82
Turnover / 1/N: BS-C	3.64	7.22	6.10	3.65	17.32	13.07

▶ **Key result: conclusions on short-sale constraints**

- Constrained beats unconstrained in 5 of 6 datasets (not with $N = 3$).
- **Very effective at reducing turnover** (from thousands of times 1/N to 4–18 times).
- Less successful at improving SR: 1/N still wins in 4 of 6 (constraints win only in FF1 and FF4).

4.7 Global-minimum-variance portfolios (focus)

In plain words

Even after fixing them, the estimated means are too noisy. So **drop them completely**: assume all assets have the same expected return. Then the target-return constraint disappears and we only minimise risk. Risk (covariances) can be estimated much more precisely than means. GMV = shrinking all expected returns 100% towards the same value.

Evidence that even adjusted means perform poorly: DGU (2009), Jacobs–Müller–Weber (2014).

Five variants.

1. **GMV** with the sample covariance (Jorion 1985, 1986).
2. **Factor-structure covariance** (Green–Hollifield 1992; Chan–Karceski–Lakonishok 1999): V built from a $K \times K$ factor covariance, $N \times K$ loadings, N residual variances. Performs worse than 3–5 (Jagannathan–Ma 2003); factor models later.
3. **GMV with short-sale constraints** (Jagannathan–Ma 2003): indirect shrinkage of V.
4. **Ledoit–Wolf shrinkage** (2003, 2004a,b, 2017, 2022): explicit shrinkage of V.
5. **Norm constraints** (DeMiguel, Garlappi, Nogales, Uppal 2009): generalised shrinkage.

4.7.1 Variant 1: the GMV portfolio

▶ **Key result: GMV weights**

$$\min_{\boldsymbol{w}} \ \boldsymbol{w}^\top V \boldsymbol{w} \quad \text{s.t.} \quad \boldsymbol{w}^\top \mathbf{1}_N = 1 \qquad \Rightarrow \qquad \boldsymbol{w}_{GMV} = \frac{V^{-1}\mathbf{1}_N}{\mathbf{1}_N^\top V^{-1}\mathbf{1}_N}.$$

Numerator: "raw" weights, inversely related to risk. Denominator: their sum, so that they add up to 1.

Additional note (not on the slides)

Derivation. $L = \boldsymbol{w}^\top V \boldsymbol{w} + \lambda(1 - \boldsymbol{w}^\top \mathbf{1}_N)$; FOC $2V\boldsymbol{w} = \lambda \mathbf{1}_N \Rightarrow \boldsymbol{w} = \frac{\lambda}{2} V^{-1}\mathbf{1}_N$; the constraint gives $\frac{\lambda}{2} = 1/(\mathbf{1}_N^\top V^{-1}\mathbf{1}_N)$.

Example. Two assets with means 1.10 and 1.09 (the "estimation error" case), vol 20%, correlation 0.99: $\boldsymbol{w}_{GMV} = (0.5, 0.5)$, exactly the true answer. **GMV is immune to errors in expected returns.**

Zero correlations. If V is diagonal, $V^{-1} = \text{diag}(1/\sigma_1^2, \ldots, 1/\sigma_N^2)$ and

$$w_n = \frac{1/\sigma_n^2}{\sum_{m=1}^{N} 1/\sigma_m^2} \qquad \text{(inverse-variance weights).}$$

Example: $V = \text{diag}(0.0625, 0.1225, 0.36) \Rightarrow \boldsymbol{w} = (0.594,\ 0.303,\ 0.103)$. In code:

```
w = np.diag(Vinv) / np.sum(np.diag(Vinv))    # only valid when V is diagonal
```

When does GMV earn a high Sharpe ratio? It overweights low-volatility (and, with correlations, low-beta) assets (Scherer 2011; Clarke–de Silva–Thorley 2010). So GMV does well when **low-volatility / low-beta stocks earn high returns** (the "low-volatility anomaly"; Dangl–Kashofer 2013). Results are very sensitive to how returns and volatility are measured (Plyakha, Uppal, Vilkov 2021).

When are GMV weights reasonable? When V is estimated precisely. Recent realised volatility (e.g. last 30 days) forecasts future volatility well (Ghysels–Santa-Clara–Valkanov 2006; Andersen–Benzoni 2010); option-implied volatilities and correlations help too (Driessen–Maenhout–Vilkov 2009; DeMiguel–Plyakha–Uppal–Vilkov 2013). Remaining problem: $\hat{V}$ is **ill-conditioned** $\Rightarrow$ variants 3, 4, 5. DGNU (2009) show all three are forms of shrinkage/regularisation. Practitioners mostly use Ledoit–Wolf; Amundi sells an ETF based on DGNU (2009).

4.7.2 Variant 3: GMV with short-sale constraints (Jagannathan–Ma 2003)

$$\min_{\boldsymbol{w}} \ \boldsymbol{w}^\top \hat{V} \boldsymbol{w} \qquad \text{s.t.} \quad \boldsymbol{w}^\top \mathbf{1}_N = 1, \quad \boldsymbol{w} \geq \mathbf{0}.$$

▶ **Key result: Jagannathan–Ma shrinkage interpretation**

The solution coincides with the *unconstrained* GMV computed with

$$\hat{V}_{JM} = \hat{V} - \boldsymbol{\lambda}_{ss} \mathbf{1}_N^\top - \mathbf{1}_N \boldsymbol{\lambda}_{ss}^\top,$$

where $\lambda_{ss,n} = 0$ if $w_n > 0$ and $\lambda_{ss,n} \geq 0$ if the constraint binds ($w_n = 0$). If asset n's constraint binds, its covariance with every other asset is reduced by λ_n (and its variance by $2\lambda_n$): it looks **less risky**, so its weight rises from negative to exactly zero.

Example ($N = 3$, only asset 3 wants to go short). $\lambda_1 = \lambda_2 = 0$, $\lambda_3 > 0$:

$$\hat{V}_{JM} = \begin{pmatrix} \sigma_{11} & \sigma_{12} & \sigma_{13} - \lambda_3 \\ \sigma_{12} & \sigma_{22} & \sigma_{23} - \lambda_3 \\ \sigma_{13} - \lambda_3 & \sigma_{23} - \lambda_3 & \sigma_{33} - 2\lambda_3 \end{pmatrix}.$$

Intuition

Why does it help? Assets that the unconstrained GMV shorts are typically those with high estimated covariances with the others, and high sample covariances are partly *over*-estimated noise. Cutting them is exactly what a good estimator would do. Jagannathan and Ma's title says it: "why imposing the wrong constraints helps". **Drawback:** many weights end up exactly zero (poor diversification); in the slides' picture, the constrained optimum sits on the edge of the triangle of feasible weights.

Python exercise 5: short-sale constraints as shrinkage

200 samples of 5 years, 25 assets.

```
import numpy as np
import pandas as pd
from scipy.optimize import minimize
rng = np.random.default_rng(46)
N, T, S = 25, 60, 200
beta = rng.uniform(0.5, 1.5, N)
V = 0.045**2*np.outer(beta, beta) + np.diag(rng.uniform(0.04, 0.10, N)**2)
one = np.ones(N)
gmv = lambda C: (lambda x: x / x.sum())(np.linalg.solve(C, one))
budget = {"type": "eq", "fun": lambda w: w.sum() - 1}
gmv_c = lambda C: minimize(lambda w: w @ C @ w, one / N, method="SLSQP",
                           bounds=[(0, 1)] * N, constraints=[budget],
                           options={"ftol": 1e-12}).x
Vh = [np.cov(rng.multivariate_normal(np.zeros(N), V, size=T), rowvar=False) for s in range(S)]
W = {"GMV": np.array([gmv(C) for C in Vh]),
     "GMV, no short sales": np.array([gmv_c(C) for C in Vh])}
vol = lambda w: np.sqrt(12 * np.einsum("sn,nk,sk->s", w, V, w))   # TRUE volatility
print(pd.DataFrame({"true vol": {k: vol(w).mean() for k, w in W.items()},
                    "min weight": {k: w.min(axis=1).mean() for k, w in W.items()},
                    "max weight": {k: w.max(axis=1).mean() for k, w in W.items()},
                    "zeros": {k: (np.abs(w) < 1e-6).sum(axis=1).mean() for k, w in W.items()}}).
    round(3))
```

```
True GMV volatility: 0.113      1/N volatility: 0.161
                     true vol  min weight  max weight  zeros
GMV                     0.148      -0.169       0.295   0.00
GMV, no short sales     0.131       0.000       0.243  14.22
```

Estimated GMV: 14.8% true vol (true minimum 11.3%) because it shorts to exploit estimation error. No short sales: 13.1%, with about 14 of 25 weights at zero.

Performance (DGU 2009).

	S&P	Ind.	Int'l	Mkt+SMB/HML	FF1	FF4
MV (out of sample)	0.0794	−0.0363	−0.0719	0.2186	−0.0684	−0.0031
MV-C	0.0892	0.0678	0.0848	0.1084	0.1977	0.2024
1/N	0.1876	0.1353	0.1277	0.2240	0.1623	0.1753
GMV	0.0820	0.1554	0.1490	0.2493	0.2778	−0.0183
GMV-C	0.0834	0.1425	0.1501	0.2493	0.1546	0.3580
Turnover / 1/N: GMV	6.54	21.65	7.30	1.11	45.47	6.83
Turnover / 1/N: GMV-C	2.47	2.58	2.27	1.11	3.93	1.76

▶ **Key result: conclusions on GMV**

- **Ignoring expected returns pays**: GMV almost always beats MV; GMV-C beats MV-C in 4 of 6.
- Effect of the short-sale constraint on GMV's SR is mixed (small in 1–4, negative with $N = 21$, strongly positive with $N = 24$).
- But it **cuts turnover**: GMV-C is only 1.1–3.9 times 1/N.

4.7.3 Variant 4: Ledoit–Wolf shrinkage of the covariance matrix

▶ **Key result: Ledoit–Wolf estimator**

$$\hat{V}_{LW} = (1 - \psi)\,\hat{V} + \psi\,\hat{V}_{target}, \qquad 0 \le \psi \le 1.$$

ψ (often called α in papers and code) is chosen to minimise the expected squared **Frobenius norm** $\|\hat{V}_{LW} - V\|_F^2$, where $\|A\|_F = \sqrt{\sum_n \sum_m a_{nm}^2}$.

Shrinkage targets.

1. **LW 2003**: the one-factor (market-model) covariance matrix.
2. **LW 2004b** ("Honey, I shrunk the sample covariance matrix"): constant correlation (all pairwise correlations = average correlation; sample variances unchanged).
3. **LW 2004a**: $\hat{V}_{target} = \frac{\text{trace}(\hat{V})}{N} I_N = \bar{\lambda} I_N$, the average variance on the diagonal ($\bar{\lambda}$ = average eigenvalue, since the trace is the sum of eigenvalues).

▶ **Key result: why identity-target shrinkage fixes the condition number**

Shrinking towards $\bar{\lambda} I_N$ changes every eigenvalue $\lambda_n \mapsto (1-\psi)\lambda_n + \psi\bar{\lambda}$ and leaves the eigenvectors unchanged. Large eigenvalues fall, small ones rise ⇒ **lower condition number**; the average eigenvalue (total variance) does not change. Equivalent to an ℓ_2-penalised maximum-likelihood estimator.

$N = 3$ **example (LW 2004a).**

$$\hat{V}_{LW} = (1 - \psi) \begin{pmatrix} \hat{\sigma}_{11} & \hat{\sigma}_{12} & \hat{\sigma}_{13} \\ \hat{\sigma}_{12} & \hat{\sigma}_{22} & \hat{\sigma}_{23} \\ \hat{\sigma}_{13} & \hat{\sigma}_{23} & \hat{\sigma}_{33} \end{pmatrix} + \psi\, \frac{\hat{\sigma}_{11} + \hat{\sigma}_{22} + \hat{\sigma}_{33}}{3}\, I_3.$$

Off-diagonal covariances are multiplied by $(1 - \psi)$; the diagonal is pulled towards the average variance.

In plain words

With $\psi = 0$ you get the sample GMV; with $\psi = 1$ (and target $\bar{\lambda} I$) all assets look identical and uncorrelated, so GMV becomes **1/N**. Ledoit–Wolf picks the point in between that is, on average, closest to the truth. So GMV-LW lies between the sample GMV and 1/N.

```
from sklearn.covariance import LedoitWolf
lw = LedoitWolf().fit(X)          # X: T x N array of returns
V_lw = lw.covariance_             # shrunk covariance (identity-type target)
psi = lw.shrinkage_               # estimated shrinkage intensity
w = np.linalg.solve(V_lw, np.ones(X.shape[1])); w /= w.sum()
```

Other code: OAS (Oracle Approximating Shrinkage, Chen–Wiesel–Eldar–Hero 2010) in scikit-learn.

Worked example: Ledoit–Wolf with two assets

Sample covariance $\hat{V} = \begin{pmatrix} 0.04 & 0.018 \\ 0.018 & 0.01 \end{pmatrix}$ (vols 20% and 10%, correlation 0.9). Target: average variance $(0.04 + 0.01)/2 = 0.025$ times I. With $\psi = 0.5$:

$$\hat{V}_{LW} = 0.5\hat{V} + 0.5 \begin{pmatrix} 0.025 & 0 \\ 0 & 0.025 \end{pmatrix} = \begin{pmatrix} 0.0325 & 0.009 \\ 0.009 & 0.0175 \end{pmatrix}.$$

	implied correlation	condition number	GMV weights
Sample $\hat{V}$	0.90	30.9	$(-0.57,\ 1.57)$
Ledoit–Wolf	0.38	2.8	$(0.27,\ 0.73)$

Shrinkage lowers the correlation, makes the matrix well conditioned, and removes the short position. In practice ψ is not chosen by hand: Ledoit–Wolf estimate it from the data (`LedoitWolf().fit(X).shrinkage_`).

Python exercise 6: Ledoit–Wolf shrinkage with 50 assets

```
import numpy as np
import pandas as pd
from sklearn.covariance import LedoitWolf
rng = np.random.default_rng(47)
N, S = 50, 200
beta = rng.uniform(0.5, 1.5, N)
V = 0.045**2*np.outer(beta, beta) + np.diag(rng.uniform(0.04, 0.10, N)**2)
one = np.ones(N)
gmv = lambda C: (lambda x: x / x.sum())(np.linalg.solve(C, one))
vol = lambda w: np.sqrt(12 * w @ V @ w)
one_run = lambda X: (lambda C, L: [vol(gmv(C)), vol(gmv(L)), np.linalg.cond(C), np.linalg.cond(L)
    ])(
    np.cov(X, rowvar=False), LedoitWolf().fit(X).covariance_)
cols = ["vol, sample", "vol, LW", "cond, sample", "cond, LW"]
res = pd.DataFrame({f"T = {T}": np.mean([one_run(rng.multivariate_normal(np.zeros(N), V, size=T))
                                         for s in range(S)], axis=0) for T in [60, 120, 240]},
                   index=cols).T
print(res)
```

```
True GMV volatility: 0.093   1/N volatility: 0.169   Condition number of true V: 74
          vol, sample  vol, LW  cond, sample  cond, LW
T = 60          0.235    0.121          3887       101
T = 120         0.122    0.111           263       112
T = 240         0.105    0.103           132        98
```

With $T = 60 \approx N$: sample condition number $\approx$ 3,900 and GMV true vol 23.5% (*worse than 1/N*). LW: condition number $\approx$ 100, vol 12.1% (close to 9.3%). The gain shrinks as T grows.

Nonlinear shrinkage (LW 2017, 2020, 2022): shrinks each eigenvalue differently. Gains over linear shrinkage are modest, so not in the course (possible master's project).

4.7.4 Variant 5: norm-constrained portfolios (DGNU 2009)

$$\min_{\boldsymbol{w}} \ \boldsymbol{w}^\top \hat{V} \boldsymbol{w} \qquad \text{s.t.} \quad \boldsymbol{w}^\top \mathbf{1}_N = 1, \quad \|\boldsymbol{w}\| \le \delta.$$

Norm	Definition	Effect
1-norm	$\|\boldsymbol{w}\|_1 = \sum_n \lvert w_n \rvert \le \delta$	Limits total gross exposure (longs + shorts). Feasible set is a diamond/polygon $\Rightarrow$ corner solutions, **some weights exactly zero**. With $\delta = 1$ it is the short-sale constraint.
2-norm	$\|\boldsymbol{w}\|_2 = \left(\sum_n w_n^2\right)^{1/2} \le \delta$	Feasible set is a circle/ball $\Rightarrow$ **rarely exact zeros**, well diversified. Smallest possible 2-norm with $\sum w = 1$ is 1/N.
A-norm	$\|\boldsymbol{w}\|_A = (\boldsymbol{w}^\top A \boldsymbol{w})^{1/2} \le \delta$	General version.

Additional note (not on the slides)

Why the 2-norm constraint is shrinkage. Adding the constraint $\boldsymbol{w}^\top \boldsymbol{w} \le \delta^2$ with multiplier $\nu \ge 0$ gives the Lagrangian $\boldsymbol{w}^\top \hat{V} \boldsymbol{w} + \nu(\boldsymbol{w}^\top \boldsymbol{w} - \delta^2) + \cdots = \boldsymbol{w}^\top (\hat{V} + \nu I)\boldsymbol{w} + \ldots$, so the solution is the unconstrained GMV with $\hat{V} + \nu I$: the sample covariance plus a multiple of the identity, which is the same idea as Ledoit–Wolf's identity target. (Same family as ridge regression; the 1-norm is the LASSO family.)

Choosing δ: cross-validation. Unlike LW (distributional assumptions), let the data choose δ, *using only historical data* (no look-ahead):

1. split the estimation sample into 5 folds; define a grid of δ values;
2. for each δ, estimate on 4 folds and evaluate on the held-out fold (e.g. minimum volatility or maximum SR);
3. rotate the folds; pick the δ with the best average hold-out performance.

Code in scikit-learn; you do not need to learn cross-validation for this course.

4.7.5 Performance of Ledoit–Wolf and norm-constrained portfolios

Sharpe ratio	10Ind	48Ind	6FF	25FF	500CRSP
1/N	0.2541	0.2508	0.2563	0.2565	0.3326
GMV-C	0.2852	0.2914	0.2629	0.2720	0.3985
LW-diagonal	0.2962	0.2620	0.3226	0.3974	0.4086
LW-1-factor	0.2902	0.2544	0.3296	0.3927	0.4500
NC-1-norm	0.2890	0.2831	0.3374	0.3553	0.3706
NC-2-norm	**0.3193**	0.2891	**0.3922**	**0.4278**	**0.4672**
Turnover					
1/N	0.0232	0.0311	0.0155	0.0174	0.0595
GMV	0.1656	0.8286	0.2223	0.7953	0.7769
GMV-C	0.0552	0.0741	0.0461	0.0841	0.4222
LW-diagonal	0.1132	0.4029	0.0905	0.3144	0.5594
LW-1-factor	0.1428	0.4034	0.1455	0.4893	0.5463
NC-1-norm	0.6013	0.8232	1.0064	0.9767	0.9753
NC-2-norm	1.0177	2.7556	1.6594	3.6275	1.0443

2-norm-constrained portfolios have the best SR, close to LW. Turnover (excluding 1/N): lowest for GMV-C, then LW-diagonal, LW-1-factor; norm-constrained portfolios have the highest. Here the GMV-type strategies beat 1/N.

4.8 The big comparison at a glance

Strategy	Uses	Fixes	Out-of-sample SR	Turnover
MV (sample)	$\hat{\boldsymbol{\mu}}$, $\hat{V}$	nothing	very poor, often < 0	enormous
Bayes–Stein	shrunk $\hat{\boldsymbol{\mu}}$, $\hat{V}$	means (partly)	poor	very high
Data-Model	model-based $\hat{\boldsymbol{\mu}}$	means	mixed	high
MV-C / BS-C	$\hat{\boldsymbol{\mu}}$, $\hat{V}$, $w \geq 0$	means (shrinkage)	better than MV	moderate
GMV	$\hat{V}$ only	means (100%)	good	moderate
GMV-C	$\hat{V}$, $w \geq 0$	means + covariances	good	low
GMV-LW	$\hat{V}_{LW}$	means + covariances	good	low–moderate
GMV-NC (2-norm)	$\hat{V}$, $\|\boldsymbol{w}\|_2 \leq \delta$	means + covariances	best	high
1/N	nothing	everything (no estimation)	hard to beat	minimal

4.9 Appendix: numerical optimisation with `scipy`

In plain words

When there are inequality constraints ($w_n \geq 0$, $w_n \leq 0.5$) there is no formula: we ask a numerical solver to search for the best weights. The solver only **minimises**, so to maximise utility we minimise *minus* utility. You need: an objective function, a starting guess (1/N), and optionally `bounds` (one interval per weight) and `constraints` (equalities such as $\sum w = 1$).

4.9.1 Mean–variance utility (with risk-free asset)

```
import numpy as np
from scipy.optimize import minimize
ones = np.ones((3, 1))
gamma = 2.715745          # equivalent to mu_targ = 30% in Chapter 3
Rf = 1.065
V = np.array([[0.0625, 0.0700, 0.1050],
              [0.0700, 0.1225, 0.0840],
              [0.1050, 0.0840, 0.3600]])
mu = np.array([[1.20], [1.30], [1.40]])
Vinverse = np.linalg.inv(V)

# Analytic (no constraints)
w_mvp = (1/gamma) * Vinverse @ (mu - Rf * ones)    # [-1.0041, 1.0052, 0.4010]
w_riskFree = 1 - ones.T @ w_mvp                    # 0.5979
w_tangency = w_mvp / (ones.T @ w_mvp)              # [-2.4972, 2.5000, 0.9972]

# Numerical: minimise MINUS utility
def objective(weights):
    portfolio_return = weights @ (mu - Rf * ones).ravel()
    portfolio_risk = weights @ V @ weights
    return (gamma/2) * portfolio_risk - portfolio_return
initial_weights = np.ones(3) / 3
result = minimize(objective, initial_weights, method="SLSQP", options={"ftol": 1e-12})
print(result.x)                                    # same as analytic

# With short-sale constraint: add bounds (0, upper)
bounds = tuple((0, 999) for _ in range(3))
result = minimize(objective, initial_weights, method="SLSQP", bounds=bounds,
                  options={"ftol": 1e-12})
w = result.x                                       # [0.000, 0.5612, 0.2117]
print(1 - w.sum(), w / w.sum())                    # rf 0.2271; tangency [0, 0.7261, 0.2739]
```

Additional note (not on the slides)

In the slides `weights.T @ (mu - Rf*ones)` mixes a 1-D array with a 3×1 column; it works but returns a 1-element array. Using `.ravel()` (as above) keeps everything 1-D and returns a scalar, which is what `minimize` expects. With MV utility no budget constraint is needed: the rest goes to the risk-free asset.

4.9.2 GMV: analytic, numerical, with bounds

Here we need an explicit **equality constraint** $\sum w = 1$.

```
w_gmv = (Vinverse @ ones) / (ones.T @ Vinverse @ ones)   # [1.5636, -0.3206, -0.2430]

def objective(weights, V):
    return weights @ V @ weights
constraints = ({"type": "eq", "fun": lambda weights: np.sum(weights) - 1})
initial_weights = np.ones(len(V)) / len(V)
result = minimize(objective, initial_weights, args=(V,), method="SLSQP",
                  constraints=constraints, options={"ftol": 1e-12})
# -> [1.5636, -0.3206, -0.2430], variance 0.04977

bounds = tuple((0, 999) for _ in range(len(V)))           # no short sales
result = minimize(objective, initial_weights, args=(V,), method="SLSQP",
                  bounds=bounds, constraints=constraints, options={"ftol": 1e-12})
# -> [1, 0, 0], variance 0.0625  (everything in asset 1: not diversified!)

bounds = tuple((0, 0.50) for _ in range(len(V)))          # 0 <= w <= 50%
result = minimize(objective, initial_weights, args=(V,), method="SLSQP",
                  bounds=bounds, constraints=constraints, options={"ftol": 1e-12})
# -> [0.5, 0.4944, 0.0056], variance 0.0812  (better diversified)
```

▶ **Key result: `minimize` cheat sheet**

- `minimize(fun, x0, args=(...), method="SLSQP", bounds=..., constraints=..., options="ftol":1e-12)`
- `bounds`: list/tuple of `(low, high)`, one per weight; `None` means no bound.
- `constraints`: dict(s) with `"type": "eq"` (must equal 0) or `"ineq"` (must be ≥ 0).
- Result: `result.x` (weights), `result.fun` (objective value), `result.success`.
- Always check `result.success` and compare with the analytic solution when one exists.

Norms (aside). $\|\boldsymbol{x}\|_2 = \sqrt{x_1^2 + \cdots + x_N^2}$ (Euclidean); $\|\boldsymbol{x}\|_1 = |x_1| + \cdots + |x_N|$ (Manhattan/taxicab); $\|\boldsymbol{x}\|_p = (\sum |x_n|^p)^{1/p}$; $\|\boldsymbol{x}\|_\infty = \max_n |x_n|$. Frobenius norm of a matrix: $\|A\|_F = \sqrt{\sum_{n,m} a_{nm}^2}$, useful to measure the distance between two matrices. In NumPy: `np.linalg.norm(x, 1)`, `np.linalg.norm(x)`, `np.linalg.norm(A, "fro")`.

4.10 Takeaways of Chapter 4

1. Means are estimated so imprecisely ($SE = \sigma/\sqrt{T}$) that MV weights built on them are extreme, unstable, and lose out of sample.
2. Sample covariance matrices are ill-conditioned when N is large relative to T; V^{-1} amplifies errors.
3. Out of sample, **1/N beats sample MV** in all six DGU datasets, and Bayesian fixes do not change that.
4. **Shrinkage** is the unifying idea: Bayes–Stein (means), short-sale constraints (means and covariances), GMV (100% shrinkage of means), Ledoit–Wolf and norm constraints (covariances).
5. GMV-type strategies (GMV-C, LW, 2-norm) are the ones that can beat 1/N; constraints are excellent at cutting turnover.

6. With inequality constraints, use `scipy.optimize.minimize`.

Common mistakes to avoid

1. Believing an in-sample Sharpe ratio: it is biased upwards (overfitting).
2. Forgetting that the GMV needs *no* expected returns: putting $\hat{\boldsymbol{\mu}}$ in the GMV formula.
3. Imposing $w \geq 0$ and concluding the problem is solved: constraints mainly cut turnover; they do not make the means precise.
4. Using `minimize` without the equality constraint $\sum w = 1$ for GMV (the solver then returns $\boldsymbol{w} = \mathbf{0}$, which has zero variance).
5. Not checking `result.success`, or starting the solver from a bad point.
6. Choosing δ (norm constraint) or ψ using the whole sample, including the out-of-sample period (look-ahead bias).
7. Comparing strategies on Sharpe ratios alone, without turnover and costs.

4.11 Self-test questions

1. A stock has $\sigma = 40\%$. Years of data for a 95% CI of $\pm 5\%$ on the mean? *($(2 \cdot 0.4/0.05)^2 = 256$ years.)*
2. Why do higher correlations make MV weights more sensitive to errors in means? *(Long–short spreads become almost riskless, so the optimiser levers up on tiny mean differences.)*
3. What does the short-sale constraint do to the expected return of an asset whose constraint binds? *(Raises it by λ_n: shrinkage towards the average.)*
4. What does Jagannathan–Ma's $\hat{V}_{JM}$ do to asset n when its constraint binds? *(Reduces its covariances by λ_n and its variance by $2\lambda_n$.)*
5. Ledoit–Wolf with target $\bar{\lambda} I$ and $\psi = 1$: what is the GMV? *(1/N.)*
6. What happens to the eigenvalues under LW identity shrinkage? *($\lambda \mapsto (1-\psi)\lambda + \psi\bar{\lambda}$.)*
7. 1-norm vs. 2-norm constraint: which gives exact zeros? *(1-norm.)*
8. Why must δ be chosen without look-ahead bias? *(Otherwise the backtest uses future information and overstates performance.)*
9. Why does GMV overweight low-volatility stocks? When does it earn a high SR?
10. Write by hand the long-only GMV with `scipy`.

```
from scipy.optimize import minimize
N = V.shape[0]
res = minimize(lambda w: w @ V @ w, np.ones(N)/N, method="SLSQP",
               bounds=[(0, 1)]*N,
               constraints=[{"type": "eq", "fun": lambda w: w.sum() - 1}])
w_gmv_c = res.x
```

11. Write by hand the Bayes–Stein means given `mu_hat`, `V_hat`, `T`.

```
N = len(mu_hat); o = np.ones(N)
Vi = np.linalg.inv(V_hat)
w_g = Vi @ o / (o @ Vi @ o); mu_g = w_g @ mu_hat
```

```
d = mu_hat - mu_g
psi = (N + 2) / ((N + 2) + T * d @ Vi @ d)
mu_bs = (1 - psi) * mu_hat + psi * mu_g
```

4.12 To do for next class

- Read DeMiguel, Garlappi, Uppal (2009), "Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?" *RFS* 22(5).
- Moreira (2021), ch. 9–10; Tidy Finance, "Constrained Optimization and Backtesting".
- Work on Assignment #2 (due 16 Nov, 22:00 CET).

References (Chapter 4, selection)

- DeMiguel, V., L. Garlappi, F. J. Nogales, and R. Uppal. 2009. A generalized approach to portfolio optimization: improving performance by constraining portfolio norms. *Management Science* 55(5): 798–812.
- DeMiguel, V., L. Garlappi, and R. Uppal. 2009. Optimal versus naive diversification: how inefficient is the 1/N portfolio strategy? *RFS* 22(5): 1915–1953.
- Jagannathan, R., and T. Ma. 2003. Risk reduction in large portfolios: why imposing the wrong constraints helps. *JF* 58(4): 1651–1684.
- James, W., and C. Stein. 1961. Estimation with quadratic loss. *Proc. 4th Berkeley Symposium*: 361–379.
- Jorion, P. 1985. International portfolio diversification with estimation risk. *J. Business* 58(3): 259–278; 1986. Bayes–Stein estimation for portfolio analysis. *JFQA* 21(3): 279–292.
- Ledoit, O., and M. Wolf. 2003. *JEF* 10(5): 603–621; 2004a. *J. Multivariate Analysis* 88: 365–411; 2004b. Honey, I shrunk the sample covariance matrix. *JPM* 30(4): 110–119; 2017. *RFS* 30(12): 4349–4388; 2022. *J. Fin. Econometrics* 20(1): 187–218.
- Merton, R. C. 1980. On estimating the expected return on the market. *JFE* 8(4): 323–361.
- Pástor, L. 2000. Portfolio selection and asset pricing models. *JF* 55: 179–223.
- Pástor, L., and R. F. Stambaugh. 2000. Comparing asset pricing models: an investment perspective. *JFE* 56(3): 335–381.

References (Chapter 1)

- Brugière, P. 2020. *Quantitative Portfolio Management*. Springer.
- Chen, A. Y., and T. Zimmermann. 2022. Open source cross-sectional asset pricing. *Critical Finance Review* 11(2): 207–264.
- Dick-Nielsen, J., P. Feldhütter, L. H. Pedersen, and C. Stolborg. 2023. Corporate bond factors: replication failures and a new framework. SSRN 4586652.
- Dick-Nielsen, J., and T. K. Poulsen. 2019. How to clean academic TRACE data. SSRN 3456082.
- Dickerson, A., P. Mueller, and C. Robotti. 2023. Priced risk in corporate bonds. *Journal of Financial Economics*.
- Gu, S., B. Kelly, and D. Xiu. 2020. Empirical asset pricing via machine learning. *Review of Financial Studies* 33(5): 2223–2273.
- Hou, K., C. Xue, and L. Zhang. 2015. Digesting anomalies: an investment approach. *Review of Financial Studies* 28(3): 650–705.
- Scheuch, C., S. Voigt, P. Weiss, and C. Frey. 2024. *Tidy Finance with Python*. Chapman & Hall/CRC.
- Welch, I., and A. Goyal. 2008. A comprehensive look at the empirical performance of equity premium prediction. *Review of Financial Studies* 21(4): 1455–1508.

Appendix A

Assignment guides

In plain words

These guides explain, question by question, *what* is asked, *which theory* you need, *how* to code it, and *what to watch out for*. The assignments are not graded (full credit if submitted), but each in-class quiz only counts if the matching assignment is submitted, and the final exam has one question per assignment, **with code written by hand**. So use the code below to understand, then make sure you can rewrite the key lines yourself and explain the results in your own words.

A.1 Submission checklist (applies to every assignment)

- Groups of exactly 3 or 4. Email to `Raman.Uppal@edhec.edu`, subject `QPM 2026-2027: Assignment n`. Deadline 22:00 CET; late = 0.
- File name `QPM-2026-Assignment-n-A-B-C-D.ipynb` (last names).
- Before sending: Colab *Runtime* → *Restart session and run all*; Jupyter *Kernel* → *Restart Kernel and Run All Cells*. No errors.
- Mention any outside help (AI included): no penalty.

Section 0 template (Markdown cell).

```
# 0. Submission information
**Submission date:** 10 November 2026
**QPM 2026-2027: Assignment 1**
**Group members:** BIANCHI Anna, MASTROGIACOMO Paolo, MULLER Jonas, ROSSI Luca
**Comments/challenges:** Some S&P 500 tickers failed to download from Yahoo
(ignored, as allowed). We used ChatGPT/Claude to debug the download code.
```

Then, for each question: a Markdown cell with the question text, followed by code cells and a Markdown cell with your interpretation.

A.2 Assignment 1 (Lectures 1–2; due 11 November)

Q1.1 Download daily FAANG prices, January 2015 – December 2020

Theory: Chapter 1, "Stock-price data". Use **adjusted** prices (splits and dividends).

```
import numpy as np
import pandas as pd
import yfinance as yf
from scipy import stats

faang = ["META", "AMZN", "AAPL", "NFLX", "GOOG"]
prices = yf.download(tickers=faang, start="2015-01-01", end="2021-01-01",
                     auto_adjust=False, progress=False)["Adj Close"]
prices = prices[faang]           # keep the column order
print(prices.shape)
prices.head()
```

Additional note (not on the slides)

- `end` is **exclusive** in `yfinance`: `end="2021-01-01"` includes 31 Dec 2020.
- `auto_adjust=False` keeps the `"Adj Close"` column; with the default (`True`) the `"Close"` column is already adjusted.
- This is *wide* format (dates × tickers), which is fine here because the next questions are matrix calculations. For tidy format: `prices.stack().rename("adjusted").reset_index()`.

Q1.2 First and second moments of returns

Theory: Chapter 1, returns and sample moments. *Means*, *variances*, *covariances* of (daily, simple) returns.

```
returns = prices.pct_change(fill_method=None).dropna()   # daily simple returns
mean_d = returns.mean()                                  # daily means
var_d = returns.var()                                    # daily variances (ddof=1)
cov_d = returns.cov()                                    # 5 x 5 covariance matrix
corr_d = returns.corr()

moments = pd.DataFrame({"mean (daily)": mean_d, "variance (daily)": var_d,
                        "mean (ann.)": 252 * mean_d, "vol (ann.)": np.sqrt(252 * var_d)})
print(moments.round(5)); print(cov_d.round(6)); print(corr_d.round(2))
```

In plain words

Report daily numbers (what is asked) and also annualised ones (easier to read: mean $\times 252$, volatility $\times\sqrt{252}$). Comment on the correlation matrix: FAANG stocks are positively correlated (common "tech" factor), which limits diversification among them (Chapter 3).

Q1.3 Skewness, excess kurtosis; are daily returns Normal?

Theory: Chapter 1, "Are returns Normal?". Under Normality $\hat{S} \sim \mathcal{N}(0, 6/T)$, $\hat{K} \sim \mathcal{N}(3, 24/T)$; Jarque–Bera.

```
T = len(returns)
dist = pd.DataFrame({
    "skewness": returns.skew(),
```

```
    "excess kurtosis": returns.kurt(),              # pandas .kurt() is EXCESS kurtosis
    "z skew": returns.skew() / np.sqrt(6 / T),
    "z ex-kurt": returns.kurt() / np.sqrt(24 / T),
    "JB stat": [stats.jarque_bera(returns[c]).statistic for c in returns],
    "JB p-value": [stats.jarque_bera(returns[c]).pvalue for c in returns]})
print(dist.round(3))
```

▶ **Key result: what you should find and how to answer**

Daily returns are **not Normal**: excess kurtosis is large (fat tails), the z-statistics are far above 1.96 and the JB p-values are ≈ 0. Expect some skewness too (sign can differ across stocks; big single-day jumps after earnings, e.g. Netflix, Meta). The Normal distribution underestimates the frequency of extreme days (Chapter 1, Python exercise 5).

Additional note (not on the slides)

`pandas .kurt()` and `scipy.stats.kurtosis(x)` (default `fisher=True`) both return **excess** kurtosis (Normal = 0). `scipy.stats.kurtosis(x, fisher=False)` returns kurtosis (Normal = 3). pandas uses bias-corrected estimators; differences with scipy are tiny for large T.

Q2.1 S&P 500 constituents, month-end adjusted prices 2000–2022

Theory: Chapter 1, "From a single stock to multiple stocks".

```
import requests
from io import StringIO

url = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"
html = requests.get(url, headers={"User-Agent": "Mozilla/5.0"}).text  # avoids 403
sp500 = pd.read_html(StringIO(html))[0]
tickers = sp500["Symbol"].str.replace(".", "-", regex=False).tolist()  # BRK.B -> BRK-B
print(len(tickers), tickers[:10])

start_date, end_date = "2000-01-01", "2022-12-31"
raw = yf.download(tickers=tickers, start=start_date, end=end_date,
                  auto_adjust=False, progress=False)["Adj Close"]

monthly = raw.resample("ME").last()                       # month-end prices
monthly = monthly.dropna(axis=1, how="all")               # drop all-NaN columns
monthly = monthly.loc[:, monthly.isna().sum() <= 100]     # drop > 100 missing months
print(monthly.shape)                                      # (276 months, ~4xx stocks)
```

Additional note (not on the slides)

- Yahoo uses `-` instead of `.` in tickers (BRK-B, BF-B).
- Downloading about 500 tickers $\times$ 23 years takes a few minutes; failed tickers are printed and can be ignored (the assignment allows it).
- `"ME"` = month end (pandas $\geq$ 2.2; in older versions use `"M"`). Alternative: the option `interval="1mo"` of `yf.download`, but its dates are the *first* day of each month; resampling daily data is cleaner.

- Save the raw download too (`raw.to_csv("sp500_daily.csv")`) so you do not need to re-download for Assignment 2.

Q2.2 Simple monthly returns (and save them)

```
ret = monthly.pct_change(fill_method=None).iloc[1:]   # first row is NaN
print(ret.index[0], ret.index[-1], ret.shape)         # 2000-02-29 ... 2022-12-31
ret.to_csv("sp500_monthly_returns.csv")
# Later: ret = pd.read_csv("sp500_monthly_returns.csv", index_col=0, parse_dates=True)
```

Additional note (not on the slides)

In Colab, files in the session disappear when it restarts. Save to Google Drive (`from google.colab import drive; drive.mount("/content/drive")`) or download the CSV.

Q2.3 Annualised mean, volatility and Sharpe ratio

Theory: Chapter 2, Sharpe ratio; Chapter 1, Fama–French data. The Sharpe ratio uses **excess** returns $r_t - RF_t$ (monthly RF from Ken French).

```
import pandas_datareader as pdr
ff = pdr.DataReader("F-F_Research_Data_Factors", "famafrench",
                    start=start_date, end=end_date)[0] / 100     # percent -> decimal
rf = ff["RF"]                                                    # PeriodIndex (monthly)

ret_p = ret.copy()
ret_p.index = ret_p.index.to_period("M")          # align on months
excess = ret_p.sub(rf, axis=0)                    # subtract RF from every column

stats_sp = pd.DataFrame({
    "mean (ann.)": 12 * ret.mean(),
    "vol (ann.)": np.sqrt(12) * ret.std(),
    "Sharpe (ann.)": np.sqrt(12) * excess.mean() / excess.std()})
print(stats_sp.describe().round(3))                          # cross-sectional summary
print(stats_sp.sort_values("Sharpe (ann.)", ascending=False).head(10).round(3))
```

Additional note (not on the slides)

Ken French dates are monthly *periods* (`2000-01`), Yahoo month-end dates are timestamps (`2000-01-31`): converting both to periods with `.to_period("M")` is the safest way to align them. Annualise SR with $\sqrt{12}$ (Chapter 2; exact only without autocorrelation).

Q2.4 Should weights be chosen only from the stocks' Sharpe ratios?

No. Arguments to develop (use the course):

1. **Correlations are ignored.** The SR is meant for efficient portfolios, not individual assets (Chapter 2). Portfolio risk depends on covariances (Chapter 3: diversification formula, marginal contribution σ_{np}/σ_p). Two high-SR stocks that are highly correlated add little; a low-SR stock that hedges the others can be valuable. The optimal rule is $\boldsymbol{w} \propto V^{-1}(\boldsymbol{\mu} - R_f \mathbf{1}_N)$, not "rank by SR". (Only if all correlations were zero would weights be $\propto (\mu_n - R_f)/\sigma_n^2 = SR_n/\sigma_n$.)
2. **Estimation error.** Each SR is estimated with $SE \approx \sqrt{(1 + SR^2/2)/T}$; with about 23 years the SE of an annual SR is about 0.21, so most differences between stocks are not significant (Chapter 2). Ranking by *in-sample* SR is fitting noise (Chapter 4: in-sample vs. out-of-sample).
3. **Look-ahead bias.** These SRs use data from 2000–2022; choosing weights with them and evaluating on the same period is in-sample.
4. **Survivorship bias.** We used *today's* S&P 500 constituents: firms that survived and grew. Their past SRs are biased upwards (Chapter 1, Python exercise 7).
5. **Multiple testing.** Picking the best of ∼480 stocks will find high SRs by luck alone (Chapter 2, Python exercise 5).
6. Other: non-Normal returns (SR ignores skewness/kurtosis), turnover and transaction costs.

Additional note (not on the slides)

Where $SE \approx 0.21$ comes from. $T = 275$ months, monthly SR about 0.1–0.2: $SE_{monthly} = \sqrt{(1 + 0.5 \cdot 0.15^2)/275} \approx 0.061$; annualised $\times\sqrt{12} \approx 0.21$.

A.3 Assignment 2 (Lectures 3–4; due 16 November)

In plain words

You run a real **rolling-window backtest** (Chapter 4 methodology) on 10 stocks with four strategies: MVP and GMV without constraints (Question 1), and MVP-C and GMV-C with $w \geq 0$ (Question 2). Then you compare out-of-sample Sharpe ratios and explain them with the theory of Chapter 4.

Q1.1 Data: 10 stocks, February 2000 – December 2022, $R_f = 0$

```
import numpy as np
import pandas as pd
from scipy.optimize import minimize

ret = pd.read_csv("sp500_monthly_returns.csv", index_col=0, parse_dates=True)
tickers = ["MMM", "AOS", "ABT", "ADM", "ADBE", "ADP", "AES", "AFL", "A", "AKAM"]
R = ret.loc["2000-02":"2022-12", tickers]
print(R.shape, R.isna().sum().sum())      # (275, 10) and 0 missing values
```

With $R_f = 0$, excess returns = returns and $SR = \sqrt{12}\,\bar{r}/\sigma_r$.

Q1.2 Estimation window of 60 months: MVP and GMV, fully invested

Theory: Chapter 3 (tangency portfolio), Chapter 4 (GMV).

$$\text{MVP: } \boldsymbol{w} = \frac{1}{\gamma}\hat{V}^{-1}\hat{\boldsymbol{\mu}} \xrightarrow{\text{rescale}} \boldsymbol{w}_{MVP} = \frac{\hat{V}^{-1}\hat{\boldsymbol{\mu}}}{\mathbf{1}_N^\top \hat{V}^{-1}\hat{\boldsymbol{\mu}}} \quad (\gamma \text{ cancels}), \qquad \text{GMV: } \boldsymbol{w}_{GMV} = \frac{\hat{V}^{-1}\mathbf{1}_N}{\mathbf{1}_N^\top \hat{V}^{-1}\mathbf{1}_N}.$$

```
T_est = 60
N = R.shape[1]
ones = np.ones(N)

def w_mvp(mu, V):                        # tangency with rf = 0
    x = np.linalg.solve(V, mu)
    return x / x.sum()

def w_gmv(V):
    x = np.linalg.solve(V, ones)
    return x / x.sum()

first = R.iloc[:T_est]                   # Feb 2000 - Jan 2005: estimation sample
mu_hat, V_hat = first.mean().values, first.cov().values
print(pd.DataFrame({"MVP": w_mvp(mu_hat, V_hat), "GMV": w_gmv(V_hat)},
                   index=tickers).round(3))
```

Q1.3 – Q1.4 Rolling window and out-of-sample returns

At each month $t = 60, \ldots, T-1$: estimate on months $t-60, \ldots, t-1$, compute the weights, apply them to the return of month t (i.e. weights chosen at the end of month $t-1$ earn the return of month t: this is "$t+1$" in the assignment's notation).

```
def run_backtest(R, strategies, T_est=60):
    weights = {k: [] for k in strategies}
    oos = {k: [] for k in strategies}
    dates = []
    for t in range(T_est, len(R)):
        window = R.iloc[t - T_est:t]          # past 60 months only (no look-ahead)
        mu_hat = window.mean().values
        V_hat = window.cov().values
        r_next = R.iloc[t].values             # next month's returns
        for k, rule in strategies.items():
            w = rule(mu_hat, V_hat)
            weights[k].append(w)
            oos[k].append(w @ r_next)         # out-of-sample portfolio return
        dates.append(R.index[t])
    oos = pd.DataFrame(oos, index=dates)
    W = {k: pd.DataFrame(v, index=dates, columns=R.columns) for k, v in weights.items()}
    return oos, W

strategies_q1 = {"MVP": lambda mu, V: w_mvp(mu, V),
                 "GMV": lambda mu, V: w_gmv(V)}
oos1, W1 = run_backtest(R, strategies_q1)
print(oos1.shape)                              # 215 out-of-sample months (Feb 2005 - Dec 2022)
```

In plain words

The loop, line by line.

- `for t in range(T_est, len(R))`: t goes from 60 (the 61st month, February 2005) to the last month. At each t we pretend we are at the *end of month* $t-1$.

- `window = R.iloc[t - T_est:t]`: the 60 rows $t-60, \ldots, t-1$. Python slices exclude the end, so month t is *not* in the window: no look-ahead.
- `mu_hat`, `V_hat`: estimates from the window only.
- `rule(mu_hat, V_hat)`: the strategy's weights, decided with past data.
- `w @ r_next`: the return of month t earned by those weights, i.e. the out-of-sample return.
- The loop runs $275 - 60 = 215$ times, so you get 215 out-of-sample returns, from February 2005 to December 2022.

Q1.5 Sharpe ratios: which is higher, and why?

```
sharpe = lambda r: np.sqrt(12) * r.mean() / r.std(ddof=1)
def summarize(oos, W):
    return pd.DataFrame({
        "mean (ann.)": 12 * oos.mean(),
        "vol (ann.)": np.sqrt(12) * oos.std(ddof=1),
        "Sharpe (ann.)": oos.apply(sharpe),
        "avg max |w|": {k: W[k].abs().max(axis=1).mean() for k in W},
        "avg turnover": {k: W[k].diff().abs().sum(axis=1).mean() for k in W}})
print(summarize(oos1, W1).round(3))
```

▶ Key result: how to explain the result (expected: GMV > MVP)

- MVP uses $\hat{\boldsymbol{\mu}}$, which is estimated very imprecisely ($SE = \sigma/\sqrt{5} \approx 13\%$ per year for a 30%-vol stock with a 5-year window). $\hat{V}^{-1}$ amplifies those errors $\Rightarrow$ extreme, unstable weights (look at "avg max $|w|$" and turnover).
- GMV ignores means (shrinks them 100% to a common value) and relies only on $\hat{V}$, which is estimated much more precisely.
- This is the DGU (2009) evidence and Chapter 4, Python exercise 3. Optionally test the difference with the Jobson–Korkie–Memmel test (Chapter 2).

▶ Key result: a trap specific to rescaling the MVP weights

The rescaled MVP divides by $\mathbf{1}_N^\top \hat{V}^{-1}\hat{\boldsymbol{\mu}}$. When this number is **close to zero** the weights explode; when it is **negative** the "tangency" portfolio flips sign (you end up *short* the efficient portfolio, i.e. on the inefficient lower half of the frontier). With 5-year windows this happens in some months. In a simulation of this exact design, the sum was negative in 25 of 215 months, the average largest weight was ±950%, and the annual volatility of MVP returns was above 2,000%. Check it in your data and **mention it** in your answer: it is a direct consequence of estimation error in the means.

```
denoms = [np.linalg.solve(R.iloc[t-60:t].cov().values, R.iloc[t-60:t].mean().values).sum()
          for t in range(T_est, len(R))]
print("months with negative 1'V^{-1}mu:", sum(d < 0 for d in denoms),
      " smallest |denominator|:", round(min(abs(d) for d in denoms), 4))
```

Q2.1 – Q2.3 MVP-C and GMV-C (no short sales), rolling window

Theory: Chapter 4, short-sale constraints and appendix (`scipy`).

MVP-C: $\max_{\boldsymbol{w}\geq\mathbf{0}} \boldsymbol{w}^\top\hat{\boldsymbol{\mu}} - \frac{\gamma}{2}\boldsymbol{w}^\top\hat{V}\boldsymbol{w}$ then rescale; GMV-C: $\min_{\boldsymbol{w}} \boldsymbol{w}^\top\hat{V}\boldsymbol{w}$ s.t. $\mathbf{1}_N^\top\boldsymbol{w}=1,\ \boldsymbol{w}\geq\mathbf{0}$.

```
def w_gmv_c(V):
    cons = {"type": "eq", "fun": lambda w: w.sum() - 1}
    res = minimize(lambda w: w @ V @ w, ones / N, method="SLSQP",
                   bounds=[(0, 1)] * N, constraints=[cons],
                   options={"ftol": 1e-12, "maxiter": 1000})
    return res.x

def w_mvp_c(mu, V, gamma=1.0):
    obj = lambda w: gamma / 2 * w @ V @ w - w @ mu       # minus MV utility
    res = minimize(obj, ones / N, method="SLSQP", bounds=[(0, None)] * N,
                   options={"ftol": 1e-12, "maxiter": 1000})
    w = np.clip(res.x, 0, None)
    if w.sum() < 1e-10:            # all estimated means <= 0: optimum is 100% risk-free
        return w_gmv_c(V)          # fallback needed to be "fully invested" (say so!)
    return w / w.sum()

strategies_q2 = {"MVP-C": lambda mu, V: w_mvp_c(mu, V),
                 "GMV-C": lambda mu, V: w_gmv_c(V)}
oos2, W2 = run_backtest(R, strategies_q2)
print(summarize(oos2, W2).round(3))
print("average number of zero weights:",
      {k: (W2[k] < 1e-6).sum(axis=1).mean() for k in W2})
```

Additional note (not on the slides)

Why γ does not matter for MVP-C. The feasible set $\{\boldsymbol{w}\geq\mathbf{0}\}$ is a cone: if $\boldsymbol{w}^*$ solves the problem for $\gamma=1$, then $\boldsymbol{w}^*/\gamma$ solves it for any γ (multiply the FOCs by $1/\gamma$). After rescaling to sum to one, the weights are identical. So MVP-C = the long-only maximum-Sharpe portfolio. (Do not add the constraint $\sum w=1$ inside the MV-utility problem: with R_f available the leftover goes to the risk-free asset; rescale afterwards as asked.) Always check `res.success`.

Q2.4 MVP-C vs. GMV-C

Often (not always) GMV-C has the higher SR, and almost always the lower turnover and volatility. If your data say the opposite, that is a legitimate result: report it and explain it (see the caveats in Q2.5). Arguments: MVP-C still uses the noisy means (the constraint only limits the damage, Chapter 4: "constraints are less successful at improving SR"); GMV-C uses no means and the constraint shrinks the covariance matrix (Jagannathan–Ma). MVP-C tends to concentrate in the few stocks with the highest recent average returns (many zero weights, little diversification).

Q2.5 Which of the four is best, and why?

Put the four in one table:

```
oos_all = pd.concat([oos1, oos2], axis=1)
W_all = {**W1, **W2}
table = summarize(oos_all, W_all)
print(table.round(3))
print("1/N Sharpe (benchmark):", round(sharpe(R.iloc[T_est:].mean(axis=1)), 3))
```

▶ **Key result: structure of a strong answer**

1. **Report** the four SRs (and vol, turnover, max weight). MVP is almost surely the worst. Among the other three the ranking depends on the sample: GMV-C or GMV is the most common winner (DGU 2009; Jagannathan–Ma 2003), but MVP-C can win in a period when the stocks with high past average returns kept doing well. Do not force the "textbook" answer: explain yours.
2. **Means vs. no means:** strategies that ignore $\hat{\boldsymbol{\mu}}$ (GMV, GMV-C) beat those that use it (MVP, MVP-C): expected returns cannot be estimated precisely (Merton 1980; $SE = \sigma/\sqrt{T}$).
3. **Constraints as shrinkage:** $w \geq 0$ is equivalent to shrinking the means (MV) or the covariances (GMV; $\hat{V}_{JM} = \hat{V} - \boldsymbol{\lambda}\mathbf{1}_N^\top - \mathbf{1}_N\boldsymbol{\lambda}^\top$), and cuts turnover a lot.
4. **Caveats:** 10 stocks, one sample period (2005–2022, includes 2008 and 2020), survivorship bias (today's constituents), no transaction costs; SR differences may not be significant (SE of each annual SR $\approx \sqrt{12 \cdot (1 + SR_m^2/2)/215} \approx 0.24$; use JKM or a bootstrap).
5. Optional: add 1/N and Ledoit–Wolf GMV as benchmarks.

Additional note (not on the slides)

Optional extension: GMV with Ledoit–Wolf. `from sklearn.covariance import LedoitWolf`; inside the loop use `V_hat = LedoitWolf().fit(window.values).covariance_`. With $N = 10$ and $T^{est} = 60$ the gain is small (the matrix is not badly conditioned); it matters much more for large N.

Appendix B

Formula sheet (Chapters 1–4)

Chapter 1 – Data and returns

Gross / net / excess	$R_t = P_t/P_{t-1}$; $r_t = R_t - 1$; $R_{t,n} - R_{t,f}$ (gross excess = net excess)
With dividends	$R_t = (P_t + D_t)/P_{t-1}$
Compound	$R_t(h) = \prod_{i=0}^{h-1} R_{t-i}$; $\ln R_t(h) = \sum_{i=0}^{h-1} \ln R_{t-i} = p_t - p_{t-h}$
Aggregation	across assets: $R_p = \sum w_n R_n$ (gross); over time: sum of log returns
Annualise (monthly)	mean $\times 12$, vol $\times\sqrt{12}$, variance $\times 12$ (daily: 252)
Arith. vs geom.	$\bar{r}_{arith} - \bar{r}_{geom} \approx \sigma^2/2$
Lognormal	$\ln R \sim \mathcal{N}(\mu, \sigma^2)$: $\mathbb{E}[R] = e^{\mu+\sigma^2/2}$, $V[R] = e^{2\mu+\sigma^2}(e^{\sigma^2} - 1)$, $\mathbb{E}[X^m] = e^{m\mu+m^2\sigma^2/2}$
Skew / kurtosis	$S = \mathbb{E}[(X-\mu)^3]/\sigma^3$, $K = \mathbb{E}[(X-\mu)^4]/\sigma^4$; Normal: $S = 0$, $K = 3$; $\hat{S} \sim \mathcal{N}(0, 6/T)$, $\hat{K} \sim \mathcal{N}(3, 24/T)$
Jarque–Bera	$JB = T[\hat{S}^2/6 + (\hat{K}-3)^2/24] \sim \chi^2_2$
SE of mean	$\sigma/\sqrt{\text{years}}$ (independent of sampling frequency)
Cov. parameters	$N(N+1)/2$
Condition number	$\lambda_{\max}/\lambda_{\min}$

Chapter 2 – Performance

Out of sample	$R_{t+1,p} = \boldsymbol{w}_t^\top \boldsymbol{R}_{t+1}$, $\boldsymbol{w}_t$ uses info up to t only
Sharpe	$SR = \mathbb{E}[R_p - R_f]/\sigma(R_p - R_f)$; annual $= \sqrt{12}\times$ monthly (IID)
Lo (2002)	$\eta(q) = q/\sqrt{q + 2\sum_{k=1}^{q-1}(q-k)\rho^k}$
Sortino	$(\mathbb{E}[R_p] - R_{target})/\sqrt{DV}$, $DV = \frac{1}{T}\sum \min(R_{t,p} - R_{target}, 0)^2$
Treynor	$\mathbb{E}[R_p - R_f]/\beta_{p,mkt}$, $\beta = C[R_p, R_m]/V[R_m]$
M^2 / RAP	$SR_p \times \sigma(R_B)$
Information ratio	$\mathbb{E}[R_p - R_B]/\sigma(R_p - R_B)$
Jensen's alpha	$(R_p - R_f) - \beta_{p,mkt}(R_{mkt} - R_f)$ (or multifactor)
Max drawdown	$\max_{\tau_1<\tau_2}\{0, 1 - V(\tau_2)/V(\tau_1)\}$
VaR / CVaR	$\text{VaR}_\alpha = -q_\alpha$; $\text{CVaR}_\alpha = -\mathbb{E}[R \mid R \le q_\alpha]$
Drifted weights	$w_{t^+,n} = w_{t,n}(1 + r_{t+1,n})/(1 + r_{t+1,p})$
Turnover	$\sum_n \lvert w_{t+1,n} - w_{t^+,n}\rvert$; annual $= 12\times$ average monthly
Net return	$R_{t+1,p}\big(1 - \kappa\sum_n \lvert w_{t+1,n} - w_{t^+,n}\rvert\big) - 1$
Test of a mean	$z = (\bar{X} - \mu_0)/(\sigma/\sqrt{T})$; reject if $\lvert z\rvert > 1.96$ (5%, two-sided); $p = 2\Phi(-\lvert z\rvert)$
Delta method	$\sqrt{T}(f(\hat{\theta}) - f(\theta)) \to \mathcal{N}(0, \nabla f^\top \Omega \nabla f)$
SE of SR	$\sqrt{(1 + SR^2/2)/T}$
JKM test	$V[\widehat{SR}_n - \widehat{SR}_m] = \frac{1}{T}[2 - 2\rho + \frac{1}{2}(SR_n^2 + SR_m^2 - 2SR_nSR_m\rho^2)]$
Bootstrap p	$\hat{p} = (1 + \sum_b \mathbb{1}\{\lvert S_b^*\rvert \ge \lvert S_{obs}\rvert\})/(B+1)$, $S_b^* = (\widehat{\Delta SR}^{*b} - \widehat{\Delta SR})/\widehat{SE}^{*b}$
Bonferroni	$p_{adj} = \min(1, K \times p)$

Chapter 3 – Mean–variance theory

Portfolio moments	$\mu_p = \boldsymbol{w}^\top \boldsymbol{\mu}$, $\sigma_p^2 = \boldsymbol{w}^\top V \boldsymbol{w}$
Diversification	$\sigma_p^2 = \frac{1}{N}\overline{\text{var}} + (1 - \frac{1}{N})\overline{\text{cov}} \to \overline{\text{cov}}$
"a" numbers	$a_1 = \boldsymbol{\mu}^\top V^{-1}\boldsymbol{\mu}$, $a_2 = \boldsymbol{\mu}^\top V^{-1}\mathbf{1}_N$, $a_3 = \mathbf{1}_N^\top V^{-1}\mathbf{1}_N$, $D = a_1 a_3 - a_2^2$
Frontier (no R_f)	$\boldsymbol{w} = \frac{1}{D}(a_1 V^{-1}\mathbf{1}_N - a_2 V^{-1}\boldsymbol{\mu}) + \frac{1}{D}(a_3 V^{-1}\boldsymbol{\mu} - a_2 V^{-1}\mathbf{1}_N)\mu_{targ}$
Frontier variance	$\sigma^2 = (a_3\mu^2 - 2a_2\mu + a_1)/D$
GMV	$\boldsymbol{w} = V^{-1}\mathbf{1}_N/a_3$, $\mu_{GMV} = a_2/a_3$, $\sigma^2_{GMV} = 1/a_3$
Frontier (with R_f)	$\boldsymbol{w} = \frac{\mu_{targ} - R_f}{(\boldsymbol{\mu} - R_f\mathbf{1}_N)^\top V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)} V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)$
MV utility	$\max\ \boldsymbol{w}^\top(\boldsymbol{\mu} - R_f\mathbf{1}_N) - \frac{\gamma}{2}\boldsymbol{w}^\top V\boldsymbol{w} \Rightarrow \boldsymbol{w} = \frac{1}{\gamma}V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)$
Tangency	$\boldsymbol{w}_T = V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)/\mathbf{1}_N^\top V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)$ (no γ)
Max Sharpe	$SR_T = \sqrt{(\boldsymbol{\mu} - R_f\mathbf{1}_N)^\top V^{-1}(\boldsymbol{\mu} - R_f\mathbf{1}_N)}$
Marginal contributions	$\partial\mathbb{E}[R_p]/\partial w_n = \mathbb{E}[R_n] - R_f$; $\partial\sigma_p^2/\partial w_n = 2\sigma_{np}$; $\partial\sigma_p/\partial w_n = \sigma_{np}/\sigma_p$
Risk share	$w_n\sigma_{np}/\sigma_p^2$ (sums to 1); vector of σ_{np} is $V\boldsymbol{w}$
Beta relation	$\mathbb{E}[R_n] - R_f = \beta_{np}(\mathbb{E}[R_p] - R_f)$, $\beta_{np} = \sigma_{np}/\sigma_p^2$, p on the frontier
Tracking error	$TE = (\boldsymbol{w}_A - \boldsymbol{w}_B)^\top \boldsymbol{R}$; vol $= \sqrt{(\boldsymbol{w}_A - \boldsymbol{w}_B)^\top V(\boldsymbol{w}_A - \boldsymbol{w}_B)}$
Appraisal ratio	$\alpha/\sigma(\varepsilon)$
Market weights	$w_n = num_n P_n / \sum_m num_m P_m$

Chapter 4 – Estimation error

Years for $\pm x$ (95%)	$(2\sigma/x)^2$
Diffuse prior	predictive variance $\hat{V}(1 + 1/T^{est})$
Bayes–Stein	$\hat{\boldsymbol{\mu}}^{BS} = (1 - \hat{\psi})\hat{\boldsymbol{\mu}} + \hat{\psi}\mu^{GMV}\mathbf{1}_N$, $\hat{\psi} = \frac{N+2}{(N+2)+T(\hat{\boldsymbol{\mu}} - \mu^{GMV}\mathbf{1}_N)^\top \hat{V}^{-1}(\hat{\boldsymbol{\mu}} - \mu^{GMV}\mathbf{1}_N)}$
Short sales ⇒ means	constrained MV = unconstrained MV with $\boldsymbol{\mu} + \boldsymbol{\lambda}_{ss}$
Jagannathan–Ma	$\hat{V}_{JM} = \hat{V} - \boldsymbol{\lambda}\mathbf{1}_N^\top - \mathbf{1}_N\boldsymbol{\lambda}^\top$
GMV diagonal V	$w_n = (1/\sigma_n^2)/\sum_m(1/\sigma_m^2)$
Ledoit–Wolf	$\hat{V}_{LW} = (1 - \psi)\hat{V} + \psi\hat{V}_{target}$; target $\bar{\lambda}I$: $\lambda_n \mapsto (1 - \psi)\lambda_n + \psi\bar{\lambda}$
Frobenius norm	$\|A\|_F = \sqrt{\sum a_{nm}^2}$
Norm constraints	$\|\boldsymbol{w}\|_1 = \sum \|w_n\| \le \delta$; $\|\boldsymbol{w}\|_2 = \sqrt{\sum w_n^2} \le \delta$; $\|\boldsymbol{w}\|_A = \sqrt{\boldsymbol{w}^\top A\boldsymbol{w}} \le \delta$

Appendix C

Glossary (English – Italian)

English	Italiano	Meaning
Asset universe	universo investibile	set of assets one can choose from
Backtest	backtest / verifica storica	simulate a strategy on past data
Benchmark	benchmark / indice di riferimento	portfolio against which a manager is judged
Beta	beta	sensitivity to a portfolio (market) return; systematic risk
Bootstrap	bootstrap / ricampionamento	resampling the data to get the distribution of a statistic
Condition number	numero di condizionamento	$\lambda_{\max}/\lambda_{\min}$; how close a matrix is to singular
Covariance matrix	matrice di varianza-covarianza	V or Σ
Diversification	diversificazione	reducing risk by combining assets
Drawdown	drawdown / perdita dal massimo	fall from a previous peak of wealth
Efficient frontier	frontiera efficiente	portfolios with max return for each level of risk
Estimation error	errore di stima	difference between estimate and true parameter
Estimation window	finestra di stima	past observations used to estimate (T^{est})
Excess return	rendimento in eccesso	return minus risk-free rate
Expanding window	finestra espandibile	estimation sample that grows over time
Expected shortfall / CVaR	expected shortfall / CVaR	average loss in the worst $q\%$ of cases
Fat tails	code spesse	more extreme outcomes than the Normal
Global minimum-variance (GMV)	portafoglio a varianza minima globale	lowest-risk fully invested portfolio
Gross / net return	rendimento lordo / netto	P_t/P_{t-1} vs. $P_t/P_{t-1}-1$
In-sample / out-of-sample	nel campione / fuori campione	evaluated on estimation data vs. on new data
Kurtosis	curtosi	fourth standardised moment (Normal = 3)
Lagrange multiplier	moltiplicatore di Lagrange	shadow price of a constraint
Look-ahead bias	distorsione da informazione futura	using information not available at decision time
Market capitalisation	capitalizzazione di mercato	shares outstanding × price
Mean–variance utility	utilità media-varianza	$\mu_p - \frac{\gamma}{2}\sigma_p^2$
Multiple testing	test multipli	many tests inflate false discoveries
Myopic / dynamic portfolio	portafoglio miope / dinamico	one-period vs. multi-period optimisation
Norm constraint	vincolo di norma	limit on $\|\boldsymbol{w}\|$
Power of a test	potenza del test	$1 - P(\text{Type II error})$
Proportional transaction cost	costo di transazione proporzionale	cost κ per unit traded

English	Italiano	Meaning
Risk aversion / tolerance	avversione / tolleranza al rischio	γ / $1/\gamma$
Risk-free asset	attività priva di rischio	R_f
Rolling window	finestra mobile	fixed-length estimation sample that moves forward
Sharpe ratio	indice di Sharpe	excess return per unit of volatility
Short sale	vendita allo scoperto	negative weight
Short-sale constraint	vincolo di non negatività	$w_n \geq 0$ (long-only)
Shrinkage	shrinkage / contrazione	pulling an estimate towards a target
Skewness	asimmetria	third standardised moment
Standard error	errore standard	standard deviation of an estimator
Survivorship bias	distorsione da sopravvivenza	using only firms that survived
Systematic / unsystematic risk	rischio sistematico / idiosincratico	non-diversifiable / diversifiable risk
Tangency portfolio	portafoglio di tangenza	max-Sharpe portfolio of risky assets
Tidy (long) / wide format	formato lungo / largo	one row per (date, asset) vs. one column per asset
Tracking error	tracking error / errore di replica	portfolio minus benchmark return
Turnover	turnover / rotazione del portafoglio	amount traded at rebalancing
Two-fund separation	separazione a due fondi	every efficient portfolio mixes two funds
Type I / Type II error	errore di I / II tipo	false rejection / failure to reject
Value at Risk (VaR)	valore a rischio	loss exceeded with probability α
Volatility	volatilità	standard deviation of returns